SCRIBE: Low-memory SNARKs via Read-Write Streaming

Anubhav Baweja
abaweja@upenn.edu
UPenn

Pratyush Mishra
prat@upenn.edu
UPenn

Tushar Mopuri
tmopuri@upenn.edu
UPenn

Karan Newatia
knewatia@upenn.edu
UPenn

Steve Wang
qwang97@upenn.edu
UPenn

July 29, 2026

Abstract

Succinct non-interactive arguments of knowledge (SNARKs) enable a prover to produce a short and efficiently verifiable proof of the validity of an arbitrary NP statement. Recent constructions of efficient SNARKs have led to interest in using them for a wide range of applications, but unfortunately, deployment of SNARKs in these applications faces a key bottleneck: SNARK provers require a prohibitive amount of time and memory to generate proofs for even moderately large statements. While there has been progress in reducing prover time, prover memory remains an issue.

In this work, we describe SCRIBE, a new *low-memory* SNARK that can efficiently prove large statements even on cheap consumer devices such as smartphones by leveraging a plentiful, but heretofore unutilized, resource: disk storage. Instead of storing its (large) intermediate state in RAM, SCRIBE's prover instead stores it on disk. To ensure that accesses to state are efficient, we design SCRIBE's prover in a *read-write streaming model* of computation that allows the prover to read and modify its state only in a streaming manner.

We implement and evaluate SCRIBE's prover, and show that, on commodity hardware, it can easily scale to circuits of size 2^{28} gates while using less than $750\,\mathrm{MB}$ of memory and incurring only 10% proving latency overhead compared to a state-of-the-art memory-intensive baseline (HyperPlonk [EUROCRYPT 2023]) that requires much more memory.

Contents

1 Introduction

SNARKs enable a prover to convince a verifier of the validity of correct program execution via a *succinct* proof that can be checked much more quickly than running the program itself. There has been much recent interest in the construction of efficient SNARKs for a wide range of applications, including blockchain rollup systems [Whi18] and cryptocurrency bridges [Xie+22]. Many of these applications require proving the correctness of large computations. For instance, both the rollup and bridge applications require proving satisfiability of circuits with billions of gates. Unfortunately, existing SNARKs incur large time and space overheads when proving large computations, requiring the use of powerful machines to generate proofs. For instance, recent industry benchmarks rely on server-class machines with powerful GPUs[1] and hundreds of gigabytes of RAM[2] to prove such statements.

Much effort [BCGJM18; Set20; CBBZ23; GLSTW23; HLP24; Pol; AST24] has been devoted to reducing the time overhead of SNARKs. While these efforts have greatly reduced prover latency, they do not address the high memory overheads. This overhead occurs because the size of the prover's internal state scales linearly with the size of the computation, as opposed to scaling with the (potentially smaller) space complexity of the computation. For example, the HyperPlonk SNARK [CBBZ23] requires over 16 GB of RAM to prove that 10 kB of data was hashed correctly with SHA256.

This has motivated recent efforts to reduce these memory requirements. These efforts proceed by reducing the size of the prover's internal state via disparate techniques such as complexity-preserving SNARKs [BC12; BCCT13; BBHV22], streaming SNARKs [BHRRS20; BHRRS21; BCHO22; ZCLKZ24], and recursive composition [BCTV14; BCLMS21; BDFG21; KST22]. However, as we detail in Section 3, these approaches achieve lower memory usage only by sacrificing asymptotic and concrete prover time. Moreover, some approaches even seem to require an inherent space-time tradeoff [BBHV22; CM24].

In sum, we do not have concretely efficient SNARKs that can prove large computations on commodity devices quickly without requiring a prohibitive amount of memory.

1.1 Our results

In this work, we tackle the foregoing problem via a new approach: instead of trying to reduce the size of the prover's internal state, we propose to instead change where it is stored and how it is accessed by the prover. We formalize our approach via a new way to model low-memory algorithms, and construct in this model a new SNARK, Scribe, that effectively scales to large computations even on commodity devices. We detail our contributions below.

Read-write streaming. We introduce a new algorithm design framework which we call the *read-write streaming model*. Algorithms in this model have access to a small amount of random-access memory (e.g., RAM), and a large amount of external storage (e.g., disk) that stores *streams* containing the algorithm's state. These streams can be read and modified only sequentially from beginning to end.

Our model is motivated by the observation that while RAM is expensive and limited, disk storage is plentiful and cheap, and so it is natural to store an algorithm's (possibly large) internal state on disk. However, this incurs fresh challenges of its own: for efficiency, disk operations read and write data in large blocks, and each operation is much more costly than a RAM operation. Hence, a naive attempt to port an algorithm to our model could incur significant latency overheads.

[1] risczero.com/blog/beating-moores-law-with-zkvm-1-0 .

[2] blog.succinct.xyz/sp1-is-live/.

To avoid this, our model restricts the algorithm's disk accesses to follow a predictable and data-independent *streaming* access pattern. This enables numerous systems optimizations such as prefetching, pipelining, and caching that mask I/O costs. We formalize read-write streams and algorithms that use them, provide efficiency measures for such algorithms, and describe how to efficiently compose them to minimize time and space overheads. We use this model to construct a new 'read-write streaming SNARK' that we describe next.

Scribe: a linear-time read-write streaming SNARK. We construct Scribe, a SNARK for arithmetic circuit satisfiability with a read-write streaming prover. To prove satisfiability of a circuit of size N, Scribe's prover requires: (i) $O(N)$ cryptographic (group) operations and $O(N)$ field operations, (ii) $O(\log N)$ random-access memory, and (iii) $O(N)$ external storage. Scribe is based on the HyperPlonk SNARK [CBBZ23], and preserves the latter's prover and verifier time complexity and succinct proof size, while reducing the prover's random-access memory from $O(N)$ to $O(\log N)$.

We obtain Scribe by adapting the popular "Polynomial IOP + Polynomial Commitment $\rightarrow$ SNARK" paradigm [CHMMVW20; BFS20] to the read-write streaming model. To do so, we first construct read-write streaming versions of polynomial IOPs (PIOPs) that are commonly used as building blocks in the literature, and show how to combine these to obtain a read-write streaming prover for HyperPlonk's PIOP. We also construct read-write streaming provers for a variety of popular polynomial commitment schemes [PST13; BGH19; WTSTW18; BMMTV21; Lee21]. All our PIOP and PC constructions preserve the time complexity of their non-streaming counterparts, while reducing their random-access space to logarithmic. We additionally provide algorithmic optimizations that minimize the amount of I/O required for key building blocks of our PIOPs and PC schemes.

Implementation. We implement Scribe as a Rust library based on the `arkworks` framework [con22]. Our implementation uses CPU RAM for random-access, and relies on disk storage for externally stored streams. To efficiently work with these streams, we develop new abstractions over file I/O that enable optimizations such as prefetching, batch operations, and parallelization. We provide details about our implementation and optimizations in Section 7.

Evaluation. We perform a thorough evaluation of Scribe's performance, and show that it can scale to prove circuits of size 2^{28} gates in just $1.5\,\mathrm{h}$ on a server-class machine. Our evaluation demonstrates that assuming reasonable hardware, read-write streaming imposes minimal overheads (10%) over the memory-intensive baseline of HyperPlonk. Furthermore, compared to a prior state-of-the-art low-memory SNARK that operates in the same application regime [BCHO22], Scribe improves proving latency by almost an order of magnitude.

We also evaluate Scribe's performance on a commodity smartphone, and show that it can scale to prove computations of size 2^{24} gates, $32\times$ larger than the memory-intensive baseline. We believe that this is the largest circuit that has been proven on a smartphone-class device to date.

2 Techniques

We describe the main ideas underlying Scribe.We begin by describing notation and background in Section 2.1, including background on HyperPlonk, the starting point for our construction of Scribe.

Next, in Section 2.3, we introduce the notion of read-write (RW) streaming algorithms. We show in Section 2.4 how to construct RW streaming prover algorithms for 'PIOP + PC' SNARKs. We then proceed to construct RW-streaming PIOPs and PC schemes in Sections 2.5, 2.6 and 2.8. Finally, in Section 7, we describe our implementation of Scribe, and evaluate its performance in Section 8. The RW streaming algorithms we construct will often be derived in a simple way from their non-streaming counterparts. We view this simplicity as a strength of our approach, as it shows that our model is expressive enough to capture existing state-of-the-art algorithms, while also showing that these algorithms can be implemented with little random-access space with minimal time overhead.

2.1 Notation

We introduce some notation that we will use in the rest of this paper.

Vector and set notation. We use $[a_1, \ldots, a_n]$ to denote *ordered* sets, and denote the set $[1, 2, \ldots, n]$ by $[n]$. Let S be a finite set. An N-dimensional vector of elements is denoted by $\boldsymbol{x} \in S^N$, and x_i denotes the i-th element of the vector. Vectors are indexed as $\boldsymbol{x} = [x_i]_{i=1}^N = [x_1, x_2, \ldots, x_N]$.[3] We denote matrices by $\mathbf{M} \in S^{N \times N}$, where $\boldsymbol{M}_i$ represents the i-th row of the matrix. Given a vector $\boldsymbol{x} = [x_i]_{i=1}^N$, we use $\boldsymbol{x}_{[i:j]}$ to denote the vector $[x_k]_{k=i}^j$. We write $x \xleftarrow{\$} S$ to denote sampling x uniformly at random from a finite set S.

Vector partitions. For a vector $\boldsymbol{x} \in \mathbb{F}^N$, $\boldsymbol{x}_E = [x_{2i}]_{i=0}^{N/2-1}$ and $\boldsymbol{x}_O = [x_{2i+1}]_{i=0}^{N/2-1}$ are vectors containing the even-indexed and odd-indexed elements of $\boldsymbol{x}$ respectively, while $\boldsymbol{x}_L = [x_i]_{i=0}^{N/2-1}$ and $\boldsymbol{x}_R = [x_i]_{i=N/2}^{N-1}$ are vectors containing the left and right halves of $\boldsymbol{x}$ respectively.[4]

Binary representation. Given integers i and n, $\mathsf{bin}_n(i)$ represents the n-bit MSB binary decomposition of i, and $\mathsf{bin}_n(i, j)$ represents the j-th bit of $\mathsf{bin}_n(i)$. We omit n when it is clear from context. We denote by $\mathsf{int}(\boldsymbol{x})$ the inverse operation which maps $\boldsymbol{x} = \mathsf{bin}_n(v)$ to v in the obvious way.

Multilinear polynomials. A polynomial is *multilinear* if the degree of each variable in every monomial is at most 1. A multilinear polynomial f is uniquely defined by its evaluations $\boldsymbol{f} := [f(\mathsf{bin}_n(i))]_{i=0}^{2^n-1}$ over the n-dimensional boolean hypercube $\{0,1\}^n$. We sometimes denote a polynomial $f(X_1, \ldots, X_n)$ by $f(\boldsymbol{X})$ when n is clear from context.

Multilinear extension. The *multilinear extension* of a vector $\boldsymbol{v} \in \mathbb{F}^{2^n}$ is the unique multilinear polynomial $\hat{v}(\boldsymbol{X}) \in \mathbb{F}[X_1, \ldots, X_n]$ such that $\hat{v}(\boldsymbol{i}) = v_{\mathsf{int}(\boldsymbol{i})}$ for all $\boldsymbol{i} \in \{0,1\}^n$.

Lagrange basis polynomial. The *Lagrange basis polynomial* $\mathrm{eq}_n(\boldsymbol{X}, \boldsymbol{Y}) := \prod_{i=1}^n (X_i Y_i + (1-X_i)(1-Y_i))$ checks that $\boldsymbol{X} = \boldsymbol{Y}$ for any $\boldsymbol{X}, \boldsymbol{Y} \in \{0,1\}^n$. We omit n when it is clear.

Oracle access. For an n-variate polynomial $p \in \mathbb{F}[\boldsymbol{X}]$, $[\![p]\!]$ denotes an *oracle* for p that can be queried at any point $\boldsymbol{x} \in \mathbb{F}^n$ to receive the evaluation $p(\boldsymbol{x}) \in \mathbb{F}$.

Multisets. We use the double braces notation $\{\!\{\cdot\}\!\}$ to represent multisets.

Algebraic notation. We use additive notation for groups. We use bilinear groups sampled by an algorithm $\mathsf{SampleGrp}$ that outputs $(\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G, H)$ where $\mathbb{F}$ is a prime field of size q, $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ are groups of order q, $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is a bilinear map, G generates $\mathbb{G}_1$, and H generates $\mathbb{G}_2$.

[3] We make an exception when indexing vectors corresponding to coefficients of multilinear polynomials; these are zero-indexed as $\boldsymbol{x} = [x_i]_{i=0}^{N-1}$.

[4] These definitions assume that N is even and the vector is indexed from 0, which is true whenever they are used.

Inner products. We use three types of inner products: (a) the scalar product $\langle \boldsymbol{x}, \boldsymbol{y} \rangle_{\mathbb{F}} := \sum_{i=1}^{N} x_i y_i$, (b) the group inner product $\langle \boldsymbol{x}, \boldsymbol{Y} \rangle_{\mathbb{G}} := \sum_{i=1}^{N} x_i \cdot Y_i$, and (c) the pairing inner product $\langle \boldsymbol{X}, \boldsymbol{Y} \rangle_e := \sum_{i=1}^{N} e(X_i, Y_i)$. We omit subscripts when it is clear from context which inner product is being used.

2.2 Starting point: HyperPlonk

To construct an efficient read-write streaming SNARK, our starting point will be the memory-*intensive* SNARK of HyperPlonk [CBBZ23]. We choose this starting point because:

- asymptotically, HyperPlonk achieves *cryptographic* linear time[5]: it requires just $O(N)$ field operations and $O(N)$ group operations to prove satisfaction of a circuit of size N.
- concretely, HyperPlonk achieves better prover times than almost all prior SNARKs, and additionally supports attractive features such as custom gates [GW19].

SNARKs from Polynomial IOPs and PC schemes. HyperPlonk and Scribe are obtained via a popular methodology [CHMMVW20; BFS20] which constructs SNARKs from two ingredients: Polynomial Interactive Oracle Proofs (PIOPs) and Polynomial Commitment (PC) schemes.

- *PIOPs* are interactive proofs where the prover's messages are *polynomial oracles*. The verifier does not read these messages in their entirety, but instead queries these polynomials at evaluation points of its choice. The verifier also often has oracle access to polynomial representations of the NP statement being proved; such PIOPs are called holographic PIOPs [CHMMVW20]. For example, a PIOP for circuit satisfiability provides an oracle representation of the circuit to the verifier.
- *PC schemes* are commitment schemes which enable the prover to commit to a polynomial p, and then later 'open' the commitment to prove the claim "$p(z) = v$", where z is an evaluation point, and v is the claimed value of $p(z)$.

The 'PIOP + PC' methodology composes these ingredients to construct succinct arguments as follows. First, if the PIOP verifier is supposed to have oracle access to a polynomial encoding of the NP statement, then the argument's preprocessing phase commits to this polynomial using the PC scheme, and provides this commitment to the argument verifier. The argument prover $\mathcal{P}$ and verifier $\mathcal{V}$ then engage in an interaction: in each round, $\mathcal{P}$ invokes the PIOP prover P for that round, commits to the polynomials produced by P via the PC scheme, and sends these commitments to $\mathcal{V}$. $\mathcal{V}$ then invokes the PIOP verifier V to compute its message for that round, and sends this to $\mathcal{P}$. At the end of the interaction, when V wishes to query its polynomial oracles at certain evaluation points, $\mathcal{V}$ sends these points to $\mathcal{P}$, which replies with the corresponding evaluation values and evaluation proofs for these using the PC scheme. A SNARK can then be constructed by invoking the Fiat–Shamir transform [FS86] on this interactive argument.

HyperPlonk [CBBZ23] constructs a SNARK via this methodology by constructing a PIOP for circuit satisfiability. The PIOP relies on *multilinear* polynomial oracles, and so the corresponding PC scheme used in HyperPlonk is one that supports committing to these. We briefly recall HyperPlonk's constructions of these ingredients, starting with an overview of their circuit representation.

HyperPlonk's circuit representation. Consider an arithmetic circuit $C : \mathbb{F}^n \to \mathbb{F}$ with m gates, each of which is a fan-in 2 addition or multiplication gate.[6] HyperPlonk represents C as three vectors $\boldsymbol{\ell}, \boldsymbol{r}, \boldsymbol{o} \in \mathbb{F}^m$,

[5] In this paper, we say that an algorithm requires 'cryptographic X time' if it performs $O(X)$ group operations, Some prior work omits the distinction between cryptographic and non-cryptographic operations, but this is inaccurate [GLSTW23] because some group operations, in particular multi-scalar multiplications, are asymptotically super-linear even with the best known algorithms [Pip80].

[6] HyperPlonk supports circuits that have higher arity gates that enforce complex logic, such as evaluating a degree-d polynomial over the gate inputs. We omit these details for brevity, but the HyperPlonk circuit relation definition in Definition 5.19 supports such 'custom gates'.

where ℓ_i, r_i, o_i denote the left input, right input, and output of the i-th gate in the circuit, respectively. The circuit is satisfied if and only if the following constraints are satisfied:

1. *Gate satisfaction constraints*: enforce that the gate operations are applied correctly. If the i-th gate is an add gate, then $o_i = \ell_i + r_i$; if it is a multiply gate, then $o_i = \ell_i \cdot r_i$.
2. *Wiring constraints*: enforce that wires between gates are connected correctly. E.g., if the output of gate j is the left input of gate i, then a wiring constraint enforces $\ell_i = o_j$.

To encode this circuit representation in polynomial form, HyperPlonk associates with each circuit the following vectors: (a) a *selector* $\boldsymbol{s} \in \mathbb{F}^m$ such that s_i is 0 if the i-th gate is an add gate, and is 1 if it is a multiply gate, and (b) *permutation* vectors $\boldsymbol{\pi}, \boldsymbol{\sigma} \in \mathbb{F}^m$ that encode the wiring constraints as follows: if the left input of gate i is the output of gate j, then $\pi_i = j$, and if instead the right input is the output of gate j then $\sigma_i = j$.[7]

HyperPlonk's PIOP. In a preprocessing phase, C is arithmetized by encoding $\boldsymbol{s}$, $\boldsymbol{\pi}$, $\boldsymbol{\sigma}$ as multilinear polynomials $\hat{s}, \hat{\pi}, \hat{\sigma}$.[8] The PIOP prover receives these polynomials as explicit input, while the PIOP verifier is given oracle access to them.

During the interactive phase, the PIOP prover receives as additional input the witness vectors $\boldsymbol{\ell}, \boldsymbol{r}, \boldsymbol{o}$, and proceeds as follows. The PIOP prover computes the multilinear extensions $\hat{\ell}$, $\hat{r}$, $\hat{o}$ of the witness vectors, and sends oracles for these to the PIOP verifier. The PIOP prover and verifier than engage in two subPIOPs corresponding to each of the checks above.

1. *Gate satisfaction*: The gate constraints are satisfied if and only if for all $i \in \{0, \ldots, m-1\}$, it holds that $s_i \cdot (o_i - \ell_i - r_i) + (1 - s_i) \cdot (o_i - \ell_i \cdot r_i) = 0$. It leverages this idea by encoding these checks as the polynomial $p = \hat{s} \cdot (\hat{o} - \hat{\ell} - \hat{r}) + (1 - \hat{s}) \cdot (\hat{o} - \hat{\ell} \cdot \hat{r})$, and enforcing that $p(\boldsymbol{i}) = 0$ for all $\boldsymbol{i} \in \{0,1\}^{\log m}$. This check is done via a *zerocheck* PIOP that enforces that a given polynomial evaluates to zero at all points in the boolean hypercube.
2. *Wiring*: The wiring constraints are satisfied if and only if for all $i \in [m]$, the left input ℓ_i of the i-th gate is the output o_{π_i} of the π_i-th gate, and the right input r_i of the i-th gate is the output o_{σ_i} of the σ_i-th gate. That is, $\boldsymbol{\ell}$ is a permutation of $\boldsymbol{o}$ with respect to $\boldsymbol{\pi}$, and similarly for $\boldsymbol{r}$ with respect to $\boldsymbol{\sigma}$. These 'permutation checks' are done via a *permcheck* PIOP that uses $\hat{\pi}, \hat{\sigma}$ to ensure that $\hat{\ell}(\boldsymbol{i}) = \hat{o}(\mathsf{bin}(\hat{\pi}(\boldsymbol{i})))$ and $\hat{r}(\boldsymbol{i}) = \hat{o}(\mathsf{bin}(\hat{\sigma}(\boldsymbol{i})))$, for every $\boldsymbol{i} \in \{0,1\}^{\log m}$.

Both these subPIOPs in turn depend on the PIOP for the *sumcheck* relation as noted in Fig. 1. We provide details about these PIOPs in Sections 2.5 and 2.6; as we explain there, existing prover algorithms for these PIOPs achieve the optimal linear time, but also require a linear amount of random-access space.

HyperPlonk's PC scheme. Chen et al. use the PST multilinear PC scheme [PST13]. We provide details about this scheme in Section 2.9.1, but note here that while both the commitment and opening algorithms require only a linear number of field operations and a linear-sized multi-scalar multiplication, they also require a linear amount of space.

2.3 Read-write streams

As explained in the introduction, we will rely on a new model of 'read-write' streaming to construct SNARK provers that can efficiently prove large statements on devices with limited memory by relying on external memory. We describe our model below, including descriptions of read-write streams, algorithms that use

[7] For brevity, we omit a discussion of how HyperPlonk's representation handles inputs to the circuit and assume that every gate is an 'internal' gate with both left and right inputs. Our formal definition in Definition 5.19 lifts these restrictions and considers a standard circuit model.

[8] Formally, this preprocessing step is performed by the PIOP *indexer*, which we have not described here for brevity. Our formal definition of PIOPs in Section 5.1.1 describes the indexer's task in detail.

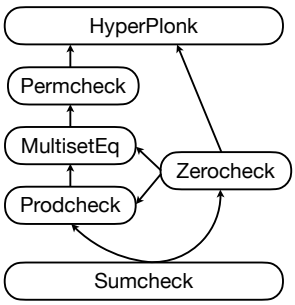

Figure 1: Components of the HyperPlonk PIOP.

these, efficiency measures for these algorithms, and how one can compose read-write streaming algorithms. We provide formal definitions in Section 4, and focus here on high-level intuition.

Read-write streams. A stream consists of a tape containing elements from some alphabet Σ, a *pointer* to a position on the tape, and a *mode* that is either 'read' or 'write'. If the stream's mode is 'read', then an algorithm can sequentially read elements from the stream (starting from the location pointed to by the pointer), while if its mode is 'write', the algorithm can sequentially write elements to the stream. More precisely, our model supports the following operations on streams:

- $\mathsf{init}(\cdot)$, which initializes a stream in write mode and allocates space for it,
- $\mathsf{restart}()$, which resets the pointer to the beginning of the stream,
- $\mathsf{read}()$, which reads a stream element from the location pointed to by the pointer (incrementing the pointer) if the stream is in read mode,
- $\mathsf{write}(\cdot)$, which writes the input value to the location pointed to by the pointer (incrementing the pointer) if the stream is in write mode.

Read-write streaming algorithms. A *read-write* (RW) streaming algorithm $\mathcal{A}$ has the following interface and behavior. It obtains its input via a stream $\mathbf{I}$ in read mode, and writes its output to a stream $\mathbf{O}$ in write mode. It has random-access to a small number of *registers*, and has read-write streaming access to a small number of intermediate read-write streams that are stored in a large external storage.[9] We denote an invocation of $\mathcal{A}$ on input $\mathbf{I}$ that produces output $\mathbf{O}$ as $\mathcal{A}(\mathbf{I}) \mapsto \mathbf{O}$.

Efficiency measures. The time complexity $t_{\mathcal{A}}$ of a read-write streaming algorithm $\mathcal{A}$ is defined as the total number of operations on registers plus the total number of stream operations performed during execution. The *random-access* space complexity $m_{\mathcal{A}}$ of $\mathcal{A}$ is the maximum number of registers it uses during execution, and the *streaming* space complexity $s_{\mathcal{A}}$ is the maximum number of elements it stores in its intermediate read-write streams during execution.

Composing read-write streaming algorithms. A read-write streaming algorithm $\mathcal{A}$ can invoke another read-write streaming algorithm $\mathcal{B}$ as a subroutine. We call this process *composition* of read-write streaming algorithms, and at a high level, it works as follows. $\mathcal{A}$ allocates two intermediate streams: $\mathbf{I}$, the input stream of $\mathcal{B}$, and $\mathbf{O}$, the output stream of $\mathcal{B}$. $\mathcal{A}$ writes the input for $\mathcal{B}$ to $\mathbf{I}$, and then invokes $\mathcal{B}$ on $\mathbf{I}$. Once $\mathcal{B}$ terminates, $\mathcal{A}$ can read the output in $\mathbf{O}$ by re-interpreting it as a read stream, and can continue the rest of its computation. Composition is well-behaved with respect to time and space efficiency, as we show in Lemma 2.1.

[9] Concretely, the number of registers and streams is at most logarithmic in the size of the input. Our model can support a logarithmic number of input streams without overhead by viewing the single input stream as the concatenation of these multiple streams, and by performing a single pass over it to copy each stream's contents to different intermediate streams. A similar statement holds for multiple output streams.

Lemma 2.1 (informal)**.** *Let $\mathcal{A}$ be a read-write streaming algorithm that invokes another read-write streaming algorithm $\mathcal{B}$ as a subroutine L times. Denote by $t'_{\mathcal{A}}$, $m'_{\mathcal{A}}$, and $s'_{\mathcal{A}}$ the time complexity, random-access space complexity, and streaming space complexity, respectively, of $\mathcal{A}$ when* ignoring the cost of running $\mathcal{B}$*. Then the time complexity of $\mathcal{A}$ is $t_{\mathcal{A}} = t'_{\mathcal{A}} + L \cdot t_{\mathcal{B}}$, the random-access space complexity is $m_{\mathcal{A}} = m'_{\mathcal{A}} + m_{\mathcal{B}}$, and the streaming space complexity is $s_{\mathcal{A}} = s'_{\mathcal{A}} + s_{\mathcal{B}}$.*

In the foregoing lemma, the space complexity costs of $\mathcal{A}$ do not incur a factor of L multiplicative blowup with respect to the same costs for $\mathcal{B}$; $\mathcal{A}$ can reclaim the space used internally by each invocation of $\mathcal{B}$ after it finishes executing.

The notion of composition can be straightforwardly generalized to support multiple subroutine calls, with the corresponding changes to total time and space complexities.

Remark 2.2 (applicability of read-write streaming)**.** While in this paper we primarily instantiate RW streams as files on disk storage, the RW streaming model is flexible enough to capture other settings where there is an imbalance between small (but fast) and large (but slow) memory. We provide some examples below, specialized for our use case of SNARK proving.

- Specialized hardware provers: there has been much recent industrial interest in hardware-accelerated proving (e.g., via GPUs, FPGAs, or custom ASICs). Unfortunately, such specialized hardware tends to have small on-device memory, which bottlenecks the size of the computations that can be proven. The CPU or other host device, on the other hand, has access to large CPU RAM, but moving data between the two types of memory is expensive. An RW streaming prover is a natural fit for this setting: the prover can run on the hardware device, and use the host device's memory as a large external storage. The streaming access pattern is entirely predictable, so the host device can efficiently pre-fetch data from its slow memory.
- Hardware-constrained devices: users of private cryptocurrencies cannot utilize hardware wallets to create private transactions without incurring privacy leakage. This is because these transactions contain SNARK proofs that cannot be proven in the wallet's limited memory, and so today wallets outsource proof generation to a host device, leaking sensitive transaction details in the process. However, if these SNARKs were equipped with an RW streaming prover, then hardware wallets could prove these SNARKs themselves by utilizing the host device's memory as external storage, and encrypting the contents of streams before sending them to the host device.

We leave it to future work to explore these applications of our RW streaming model.

2.3.1 Common RW streaming subroutines

We define below some helper functions that we use in our streaming algorithms in the rest of the overview.

- Zip takes as input k streams $\mathbf{R}_1, \ldots, \mathbf{R}_k$ of length N containing k ordered sets, and outputs a stream $\mathbf{W}$ whose i-th element is a tuple of k elements, where the j-th element of the tuple is the i-th element of $\mathbf{R}_j$.
- Reduce takes as input a stream $\mathbf{R}$ of length N, an accumulator s, and a function f, and produces a single element that is the result of reducing the elements of $\mathbf{R}$ using f.
- Map takes as input a stream $\mathbf{R}$ of length N and a function f, and outputs a stream $\mathbf{W}$ which contains the result of applying f to each element of $\mathbf{R}$.
- SplitLSB takes as input a stream $\mathbf{R}$ of length $N = 2^n$ and a parameter k, and splits $\mathbf{R}$ into 2^k output streams, each with $N/2^k$ elements, such that the i-th output stream contains the elements of $\mathbf{R}$ whose indices have the lowest k bits equal to i. SplitEO is the special case when $k = 1$ that splits $\mathbf{R}$ into two streams containing even and odd-numbered elements. SplitMSB and SplitLR are defined analogously with respect to the k most significant bits.

$\mathsf{Zip}(\mathbf{R}_1, \ldots, \mathbf{R}_k) \mapsto \mathbf{W}$: 1. For i in $[1, \ldots, \mathbf{R}_1.\mathsf{len}()]$: 2. $\quad \mathbf{W}.\mathsf{write}((\mathbf{R}_1.\mathsf{read}(),$ 3. $\quad\quad \ldots, \mathbf{R}_k.\mathsf{read}()))$.	$\mathsf{Reduce}(\mathbf{R}, f, e) \mapsto s$: 1. $s \leftarrow e$. 2. For i in $[1, \ldots, \mathbf{R}.\mathsf{len}()]$: 3. $\quad s \leftarrow f(s, \mathbf{R}.\mathsf{read}())$.
$\mathsf{Map}(\mathbf{R}, f) \mapsto \mathbf{W}$: 1. For i in $[1, \ldots, \mathbf{R}.\mathsf{len}()]$: 2. $\quad \mathbf{W}.\mathsf{write}(f(\mathbf{R}.\mathsf{read}()))$.	$\mathsf{SplitLSB}(\mathbf{R}, k) \mapsto (\mathbf{W}_1, \ldots, \mathbf{W}_{2^k})$: 1. For i in $[1, \ldots, \mathbf{R}.\mathsf{len}()/2^k]$: 2. $\quad$ For j in $[1, \ldots, 2^k]$: $\mathbf{W}_j.\mathsf{write}(\mathbf{R}.\mathsf{read}())$.

2.4 RW streaming SNARKs from RW streaming PIOPs and PC schemes

Let P be a RW streaming PIOP prover, and let PC be an RW streaming PC scheme. Then we can construct the RW streaming argument prover $\mathcal{P}$ in a manner analogous to the non-streaming case (Section 2.2): $\mathcal{P}$ receives on its input stream (s) any preprocessed polynomials and the NP witness and initializes P with these. In each round of the protocol, $\mathcal{P}$ invokes P with the latest verifier message to obtain a stream containing the current round polynomials. $\mathcal{P}$ then passes this stream to $\mathsf{PC.Commit}$ to obtain commitments to the round polynomials, and sends these to $\mathcal{V}$. At the end of the interaction, $\mathcal{P}$ invokes $\mathsf{PC.Open}$ with input streams comprised of both the preprocessed polynomials and the round polynomials produced during the interactive phase. $\mathcal{P}$ assembles the resulting opening proof and the commitments into the final SNARK proof and outputs this.

Efficiency. The argument prover $\mathcal{P}$ calls 3 subroutines: the PIOP Prover P, and the PC scheme algorithms $\mathsf{PC.Commit}$ and $\mathsf{PC.Open}$ Applying the version of Lemma 2.1 that supports multiple subroutines gives the following lemma:

Lemma 2.3. *Consider the following ingredients:*

- *A PIOP whose prover, on inputs of size N, requires time $t_{\mathsf{PIOP}}(N)$, random-access space $m_{\mathsf{PIOP}}(N)$, and streaming space $s_{\mathsf{PIOP}}(N)$. The PIOP prover produces c polynomials and the PIOP verifier queries o polynomials.*
- *A PC scheme whose algorithms have the following complexities on inputs of size N.*
  - $\mathsf{PC.Commit}$ *requires time $t_{\mathsf{C}}(N)$, random-access space $m_{\mathsf{C}}(N)$, and streaming space $s_{\mathsf{C}}(N)$.*
  - $\mathsf{PC.Open}$ *requires time $t_{\mathsf{O}}(N)$, random-access space, $m_{\mathsf{O}}(N)$, and streaming space $s_{\mathsf{O}}(N)$.*

Then there exists a read-write streaming argument whose prover $\mathcal{P}$ has the following efficiency properties:

- *Time* $= t_{\mathsf{PIOP}}(N) + \mathsf{c} \cdot t_{\mathsf{C}}(N) + \mathsf{o} \cdot t_{\mathsf{O}}(N)$.
- *Random-access space* $= m_{\mathsf{PIOP}}(N) + m_{\mathsf{C}}(N) + m_{\mathsf{O}}(N)$.
- *Streaming space* $= s_{\mathsf{PIOP}}(N) + s_{\mathsf{C}}(N) + s_{\mathsf{O}}(N)$.

We note that the construction underlying Lemma 2.3 enables optimizations that are not possible in the read-only setting; we detail these in Section 3. Applying Lemma 2.3 to the RW streaming PIOP and PC scheme[10] that we construct in Sections 2.6 and 2.8 leads to the following corollary:

Corollary 2.4. *There exists a RW streaming argument prover $\mathcal{P}$ for circuit satisfiability that requires $O(N)$ cryptographic time, $O(\log N)$ random-access space, and $O(N)$ streaming space, where N is the size of the circuit.*

Proof. The prover for the PIOP of Section 2.6 requires $O(N)$ time, $O(\log N)$ random-access space, and $O(N)$ streaming space and the verifier queries $L = 6$ polynomials, while the commitment and opening

[10] We say that a PIOP or argument is read-write streaming if it has a read-write streaming prover algorithm, and that a PC scheme is read-write streaming if it has read-write streaming commitment and opening algorithms.

algorithms of the PC scheme in Section 2.8 both require $O(N)$ cryptographic time, $O(\log N)$ random-access space, and $O(N)$ streaming space. □

Remark 2.5. The foregoing discussion does not specify how RW streams for the witness are produced. In Section 7 we show how to do so for the HyperPlonk relation so that the random-access space scales with the space complexity of the computation, instead of the size of the corresponding circuit.

2.5 Read-write streaming sumcheck

A core component of many PIOPs (including ours and HyperPlonk's) is the sumcheck protocol [LFKN92]. In this section, we show how to construct linear-time read-write streaming provers for the sumcheck protocol when specialized to sums of products of multilinear polynomials (this is without loss of generality). We begin below with an overview of existing (non-streaming) algorithms, and then show how to adapt these to the read-write streaming setting in Section 2.5.1.

Background: sumcheck. The sumcheck problem requires a prover P to convince a verifier V that $\sum_{\boldsymbol{b}\in\{0,1\}^n} f(\boldsymbol{b}) = \sigma$, where $f(\boldsymbol{X})$ is an n-variate polynomial of individual degree at most d over a field $\mathbb{F}$, and $\sigma \in \mathbb{F}$ is a claimed sum. The sumcheck protocol [LFKN92] is a protocol for this problem.

PIOP 1: SUMCHECK

$\langle \mathsf{P}(f), \mathsf{V}([\![f]\!], d, \sigma)\rangle$:
For i in $[n, \ldots, 1]$:
1. P sends to V the following univariate degree d polynomial:
   $a_i(X_i) := \sum_{\boldsymbol{b}\in\{0,1\}^{i-1}} f(\boldsymbol{b}, X_i, r_{i+1}, \ldots, r_n)$.
2. V checks that a_i is of degree at most d.
3. If $i = n$, V sets $\sigma_i := \sigma$; else it sets $\sigma_i := a_{i-1}(r_{i-1})$.
4. V checks if $a_i(0) + a_i(1) = \sigma_i$.
5. V samples $r_i \xleftarrow{\$} \mathbb{F}$.
6. If $i = 1$, V asserts $a_1(r_1) = f(r_1, \ldots, r_n)$; else it sends r_i to P.

Like prior sumcheck-based SNARKs [Set20; CBBZ23], we assume that f is a sum of products of at most d-many multilinear polynomials $p_1, \ldots, p_d$. That is, $f := h(p_1, \ldots, p_d)$, where h is a multilinear polynomial with ℓ monomials. We refer to polynomials f of this form as (d, ℓ)-sumprod polynomials.

P's input is represented by $\boldsymbol{p}_1, \ldots, \boldsymbol{p}_d$: the evaluations of the multilinear polynomials $p_1, \ldots, p_d$ over the boolean hypercube $\{0,1\}^n$ ordered lexicographically. Below we recap Thaler's [Tha13; XZZPS19] linear-time prover for the sumcheck PIOP for product of d multilinear polynomials (so $\ell = 1$), as it will be the basis for our RW streaming sumcheck prover. We omit the details of the work of V since they are unchanged. At a high level, in each round Thaler's algorithm first interpolates the polynomial a_i (Step 4 and the Sum helper function), and then eliminates the i-th variable via the FoldInPlace helper.

$\mathsf{P}(\boldsymbol{p}_1, \ldots, \boldsymbol{p}_d)$:
1. Initialize table $\boldsymbol{T}_j \leftarrow \boldsymbol{p}_j$ for all $j \in [d]$.
2. For i in $[n, \ldots, 1]$:
3. Define $N_i := N/2^{n-i}$.
4. Set $S \leftarrow \mathsf{Sum}(N_i, d, \boldsymbol{T}_1, \ldots, \boldsymbol{T}_d)$.
5. Send $[a_i(s) := S[s]]_{s=0}^{d}$ to V.
6. If $i \neq 1$:
7. Receive challenge $r_i \xleftarrow{\$} \mathbb{F}$ from V.
8. $\mathsf{FoldInPlace}(N_i, 1 - r_i, r_i, \boldsymbol{T}_j)$ for all $j \in [d]$.

$\mathsf{Sum}(N, d, \boldsymbol{T}_1, \ldots, \boldsymbol{T}_d) \to S$:
1. Set $S \leftarrow [0]_{s=0}^{d}$.
2. For j in $[1, \ldots, N/2]$ and s in $[0, \ldots, d]$:
3. $S[s] \leftarrow S[s] + \prod_{i=1}^{d} ((1-s)\cdot T_i[2j] + s \cdot T_i[2j+1])$.

$\mathsf{FoldInPlace}(N, \alpha, \beta, \boldsymbol{T})$:
1. For j in $[1, \ldots, N/2]$, set $T[j] \leftarrow \alpha \cdot T[2j] + \beta \cdot T[2j+1]$.

The foregoing algorithm requires a table of size $N = 2^n$ for each multilinear polynomial, and at the i-th iteration, it performs $N_i = N/2^{n-i}$ reads and writes from this table. This leads to the claimed time complexity of $O(d \cdot N)$, but also, unfortunately, to a space complexity of $O(d \cdot N)$.

2.5.1 Linear-time RW streaming sumcheck

Prior work indicates that algorithms for sumcheck with random-access space $O(\text{polylog } N)$ must take time $\Omega(N \log N / \log \log N)$ [Baw+25]. We design our RW streaming algorithm to avoid this lower bound.

Our algorithm. In each iteration of the algorithm from Section 2.5, all operations except Sum and FoldInPlace require $O(1)$ time and space. We design RW streaming algorithms for these subroutines, and then construct a RW streaming version of the sumcheck prover P using these. P first initializes $\boldsymbol{T}_1, \ldots, \boldsymbol{T}_d$ as RW streams. Then, it invokes the foregoing algorithms to compute streams of the folded tables. (a) RWSum reads from the table T in a streaming manner to compute $a_i(0)$ and $a_i(1)$, and (b) RWFoldInPlace reads from *and* writes to the table T in a streaming manner using an intermediate stream. The resulting algorithm works as follows, where we highlight the streaming operations in blue:

Read-write Streaming Algorithm 1: SUMCHECK for (d, ℓ)-sumprod polynomials

$\mathsf{P}(\mathbf{R}_1, \ldots, \mathbf{R}_d)$:
1. Initialize tables $\boldsymbol{T}_i \leftarrow \mathbf{R}_i$.
2. For each i in $[n, \ldots, 1]$:
3. Initialize running sums: $S \leftarrow [0]_{j=0}^{d}$.
4. For each monomial $(c \cdot X_{j_1} \cdot X_{j_2} \cdot \ldots \cdot X_{j_k})$ in h:
5. Obtain $d+1$ evaluations for the round polynomial of the monomial: set $E \leftarrow \mathsf{RWSum}(d, \boldsymbol{T}_{j_1}, \ldots, \boldsymbol{T}_{j_k})$.
6. Add each evaluation to the corresponding running sum: For s in $[0, \ldots, d]$: set $S[s] \leftarrow S[s] + c \cdot E[s]$.
7. Restart the streams that were used for this monomial: $\boldsymbol{T}_{j_1}.\mathsf{restart}(), \ldots, \boldsymbol{T}_{j_k}.\mathsf{restart}()$.
8. Send $[a_i(s) := S[s]]_{s=0}^{d}$ to V.
9. If $i \neq 1$:
10. Receive verifier challenge $r_i \xleftarrow{\$} \mathbb{F}$ from V.
11. Fold streams: $\mathsf{RWFoldInPlace}(1 - r_i, r_i, \boldsymbol{T}_1, \ldots, \boldsymbol{T}_d)$.

$\mathsf{RWSum}(d, \boldsymbol{T}_1, \ldots, \boldsymbol{T}_k) \mapsto S$:
1. Set $S \leftarrow [0]_{s=0}^{d}$.
2. For i in $[1, \ldots, \mathbf{R}_1.\mathsf{len}()/2]$:
3. For j in $[1, \ldots, k]$: $a_{L,j} \leftarrow \boldsymbol{T}_j.\mathsf{read}(); a_{R,j} \leftarrow \boldsymbol{T}_j.\mathsf{read}()$.
4. For s in $[0, \ldots, d]$: $S[s] \leftarrow S[s] + \prod_{j=1}^{k} ((1-s) \cdot a_{L,j} + s \cdot a_{R,j})$.

$\mathsf{RWFoldInPlace}(\alpha, \beta, \boldsymbol{T}_1, \ldots, \boldsymbol{T}_d)$:
1. Allocate intermediate RW stream: $\mathbf{W}.\mathsf{init}(\boldsymbol{T}_i.\mathsf{len}()/2)$.
2. For i in $[1, \ldots, d]$:
3. Set $\mathbf{W}_i \leftarrow \mathsf{Map}((a, b) \mapsto \alpha \cdot a + \beta \cdot b) \rightsquigarrow \mathsf{Zip} \rightsquigarrow \mathsf{SplitEO} \rightsquigarrow \boldsymbol{T}_i$.
4. Set $\boldsymbol{T}_i \leftarrow \mathbf{W}_i.\mathsf{swapmode}()$.

It is straightforward to see that this algorithm preserves the desired time complexity of $O(d \cdot \ell \cdot N)$ and reduces the random-access space complexity to just $O(d + \ell)$.

Optimizations. If two monomials in h share a common term p_j, we do not need to make two passes over $\mathbf{R}_j$ to compute the monomials' contributions to the round polynomial. Instead, we can read from $\mathbf{R}_j$ once and store the value in RAM. This value can be used to compute both contributions. This enables a single pass over each $\mathbf{R}_j$, as opposed to worst-case ℓ passes. Additionally, if one of the input polynomials is eq,

then P can generate its evaluations in a *read-only* streaming manner in $O(N)$ time and $O(\log N)$ space, thus reducing I/O costs. See Sections 2.6.1 and 5.2 for details.

2.6 Read-write streaming PIOPs

With our RW streaming sumcheck PIOP in hand, we are now equipped to construct read-write streaming variants of the various PIOPs underlying the HyperPlonk PIOP.

2.6.1 Zerocheck

Given a polynomial p, the zerocheck PIOP [CBBZ23] aims to prove the following claim: "for all $\boldsymbol{x} \in \{0,1\}^n$: $p(\boldsymbol{x}) = 0$".

Reduction to sumcheck. The PIOP reduces the zerocheck claim to the sumcheck claim "$\sum_{\boldsymbol{b}\in\{0,1\}^{n-1}} p(\boldsymbol{b}) \cdot \mathrm{eq}(\boldsymbol{r}, \boldsymbol{b}) = 0$", where $\boldsymbol{r}$ is a random point sent by the verifier. (Recall that $\mathrm{eq}(\boldsymbol{r}, \boldsymbol{b}) = \prod_{i=1}^{n}((1-r_i)(1-b_i)+r_i b_i)$.)

PIOP 2: ZEROCHECK

$\langle \mathsf{P}(\boldsymbol{p}), \mathsf{V}([\![p(\boldsymbol{X})]\!])\rangle$:

1. V samples $\boldsymbol{r} \xleftarrow{\$} \mathbb{F}^n$ and sends it to P.
2. P and V engage in a sumcheck of the product of $p(\boldsymbol{X})$ and $\mathrm{eq}(\boldsymbol{r}, \boldsymbol{X})$, with the target $\sigma = 0$.

Read-write streaming PIOP for zerocheck. We show how to create a streaming prover P for the zerocheck PIOP by designing a *read-only* streaming algorithm that generates the evaluations of the Lagrange basis polynomial, $[\mathrm{eq}(\boldsymbol{r}, \boldsymbol{b})]_{\boldsymbol{b}\in\{0,1\}^n}$. P then uses the read-write streaming sumcheck PIOP to prove the desired sumcheck claim.[11] The time and space complexity of this PIOP are determined by the cost of sumcheck and the cost of generating a stream for the evaluations of the Lagrange basis polynomials over $\{0,1\}^n$. The former requires $O(N)$ time with read-write streams as shown in Section 2.5.1, and we show below how to achieve the latter in $O(N)$ time with just read-only streams.

Computing the Lagrange basis polynomial. Given any $\boldsymbol{b} \in \{0,1\}^n$, computing $\mathrm{eq}(\boldsymbol{r}, \boldsymbol{b})$ takes $O(\log N)$ time. Therefore, at first glance it may seem like producing evaluations over the entire hypercube would require $O(N \log N)$ time. We achieve $O(N)$ time by fleshing out an observation from an unpublished manuscript of Vu in 2013. The resulting algorithm proceeds as follows. First, it invokes an initialization procedure $\mathsf{Eq.Init}(\boldsymbol{r})$ that, on input $\boldsymbol{r}$, initializes a state in $O(\log N)$ time. Then, to generate the stream $[\mathrm{eq}(\boldsymbol{r}, \boldsymbol{b})]_{\boldsymbol{b}\in\{0,1\}^n}$, it sequentially invokes $\mathsf{Eq.Next}()$, which updates this state and outputs $\mathrm{eq}(\boldsymbol{r}, \boldsymbol{b})$, in $O(1)$ amortized time.

$\mathsf{Eq.Init}(\boldsymbol{r})$:

1. Define $S := \{i \in [n] : r_i \in \{0,1\}\}$.
2. Set $\mathsf{start} \leftarrow 0$ and $\mathsf{out} \leftarrow 0$.
3. Set $\mathsf{prod} \leftarrow \prod_{i\notin S}(1-r_i)$. // (running evaluation of eq)
4. Set $\boldsymbol{b} \leftarrow [0]_{i=1}^n$ and store $\boldsymbol{r}$.

$\mathsf{Eq.Next}()$:

1. If $\mathsf{bin}(\boldsymbol{b}, i) = \mathsf{bin}(\boldsymbol{r}, i)$ for all $i \in S$: // ($O(1)$ time)
2. If $\mathsf{start} = 0$:
3. Set $\mathsf{start} \leftarrow 1$; $\mathsf{out} \leftarrow \mathsf{prod}$. // (first non-zero eval)
4. Else, for j in $[n, \ldots, 1] \setminus S$:
5. If $\mathsf{bin}(\boldsymbol{b}, j) = 0$: set $\mathsf{prod} \leftarrow \mathsf{prod} \cdot (1-r_j)/r_j$.
6. Else, set $\mathsf{prod} \leftarrow \mathsf{prod} \cdot r_j/(1-r_j)$; $\mathsf{out} \leftarrow \mathsf{prod}$; break.
7. Else, set $\mathsf{out} \leftarrow 0$. // (there exists $i \in S$ s.t. $r_i \neq b_i$)
8. Increment (the integer represented by) $\boldsymbol{b}$. // ($O(1)$ time)
9. Output out.

[11] We provide a concrete optimization of this reduction via the *Lagrange sumcheck protocol* in Section 5.2.

We first analyze the case where $\boldsymbol{r} \in (\mathbb{F} \setminus \{0,1\})^n$, implying that $S = \varnothing$. Eq.Init initializes $\boldsymbol{b}$ with $[0]_{i=1}^n$ and prod with $\mathrm{eq}(\boldsymbol{r}, [0]_{i=1}^n) = \prod_{i=1}^n (1 - r_i)$, and Eq.Next then maintains the following invariant: $\mathsf{prod} = \prod_{i \in [n]} (r_i b_i + (1 - r_i)(1 - b_i))$, while incrementing $\boldsymbol{b}$.

While a single call to Eq.Next can, in the worst case, cost $O(n) = O(\log N)$ time, but over N invocations, the cost amortizes to $O(1)$ per invocation. This is because each multiplication with $r_i/(1 - r_i)$ or $(1 - r_i)/r_i$ corresponds to flipping the i-th bit of $\boldsymbol{b}$. For each $i \in [n]$, this occurs at most 2^{n-i} times, and therefore the total number of flips (and hence the total number of multiplications) required to compute the evaluations of eq is at most $2N$. As an additional optimization, Steps 1 and 8 can be performed in $O(1)$ time by storing $\boldsymbol{b}$ as an integer and using bitwise operations.

For the other case where some $r_i \in \{0,1\}$, we cannot divide by either r_i or by $(1 - r_i)$, so we need to specially handle this. If $\boldsymbol{b}$ and $\boldsymbol{r}$ are equal on all the indices in S, then $\prod_{i \in S}((1 - r_i)(1 - b_i) + r_i b_i) = 1$, and therefore we simply return $\mathrm{eq}(\boldsymbol{r}, \boldsymbol{b}) = \prod_{i \notin S}((1 - r_i)(1 - b_i) + r_i b_i)$, which we iteratively update as required. If $\boldsymbol{b}$ and $\boldsymbol{r}$ are not equal on the indices in S, there exists an index $i \in S$ such that $r_i \neq b_i$, and therefore $\mathrm{eq}(\boldsymbol{r}, \boldsymbol{b})$ evaluates to 0.

Remark 2.6 (comparison to Vu's procedure)**.** Vu gives a high level description of the foregoing algorithm, but this description omits numerous details, and in particular does not handle the case where some of the r_i's are boolean. The latter is important for supporting batch opening of multiple polynomials at different points [CBBZ23].

2.6.2 Prodcheck

A prodcheck claim for two n-variate multilinear polynomials p and q says that the product of their evaluations over the hypercube $\prod_{\boldsymbol{x} \in \{0,1\}^n} (p(\boldsymbol{x})/q(\boldsymbol{x}))$ equals a claimed product $\sigma \in \mathbb{F}$. PIOPs for prodcheck claims are a fundamental tool for constructing PIOPs for more complex statements, including multiset-equality-check and permcheck.

Reduction to zerocheck. HyperPlonk relies on the prodcheck PIOP introduced by Setty and Lee [SL20]. In this PIOP, the prover P computes and sends to the verifier an $(n+1)$-variate polynomial ν such that $\nu(0, \boldsymbol{X}) := p(\boldsymbol{X})/q(\boldsymbol{X})$ and $\nu(1, \boldsymbol{X}) := \nu(\boldsymbol{X}, 0) \cdot \nu(\boldsymbol{X}, 1)$. P and V then engage in a zerocheck PIOP for the claims "$\nu(0, \boldsymbol{X}) \cdot q(\boldsymbol{X}) - p(\boldsymbol{X}) = 0$" and "$\nu(1, \boldsymbol{X}) - \nu(\boldsymbol{X}, 0) \cdot \nu(\boldsymbol{X}, 1) = 0$", which together ensure that ν is constructed correctly from p and q, and hence $\nu(1, 1, \ldots, 1, 0) = \prod_{\boldsymbol{x} \in \{0,1\}^n} p(\boldsymbol{x})/q(\boldsymbol{x})$. If this holds, then by construction of ν, the verifier V can query $\nu(1, 1, \ldots, 1, 0)$ and check that it equals σ to verify the prodcheck claim.

Because each zerocheck PIOP only requires $O(N)$ time and $O(\log N)$ random-access space, the only remaining challenge is to compute ν with the same complexity. The standard algorithm for computing ν does so recursively: for each $\boldsymbol{x} \in \{0,1\}^n$, let $\nu^{(0)}(\boldsymbol{x}) = p(\boldsymbol{x})/q(\boldsymbol{x})$, and, for each $i \in \{0, \ldots, n-1\}$ and each $\boldsymbol{x} \in \{0,1\}^{n-i}$, set $\nu^{(i+1)}(\boldsymbol{x}) = \nu^{(i)}(1, \boldsymbol{x}) \cdot \nu^{(i)}(0, \boldsymbol{x})$. Then ν is the concatenation of the $\nu^{(i)}$'s. With $O(N)$ space, this requires $O(N)$ time. What about with less space?

Barrier to read-only streaming. A streaming variant of the foregoing algorithm would only be able to produce a stream for ν by producing, in order, streams for $\nu^{(0)}, \nu^{(1)}, \ldots, \nu^{(n-1)}$. While this is straightforward for $\nu^{(0)}$, producing the stream for $\nu^{(i+1)}$ requires computing the product of two evaluations of $\nu^{(i)}$, which in turn would either require recomputing $\nu^{(i)}$ (and hence every $\nu^{(j)}$ for $j < i$), or storing it in memory. As the latter is not possible in the low-memory regime, we must use the first approach which requires $O(N \log N)$ time overall.

Remark 2.7. An alternative approach outputs a stream containing the entries of the $\nu^{(i)}$'s, but in an interleaved form. The idea is to make explicit the tree induced by the recursion, and then, instead of exploring the tree in

a breadth-first manner (i.e., producing the leaves $\nu^{(0)}$, then the layer above it, and so on), we explore it in a depth-first manner. This approach achieves $O(N)$ time with $O(\log N)$ space, but it produces ν's evaluations in an interleaved manner, which is incompatible with our zerocheck and sumcheck PIOPs.

Producing $\boldsymbol{\nu}$ using RW streams. In the read-write setting, we *can* store the evaluations of $\nu^{(i)}$ on disk, and therefore we can produce the stream for ν in $O(N)$ time and $O(\log N)$ random-access space. We defer to Section 5.3 a detailed description of this algorithm and the overall PIOP. We describe the algorithm below.

$\mathsf{P}(\boldsymbol{p}, \boldsymbol{q})$:
1. Initialize a read-stream $\boldsymbol{\nu}^{(0)} := \boldsymbol{p}/\boldsymbol{q}$.
2. For i in $[1, \ldots, n-1]$:
3. Initialize a write-stream $\boldsymbol{\nu}^{(i)}$ of size 2^{n-i}.
4. For j in $[1, \ldots, 2^{n-i}]$:
5. $a \leftarrow \boldsymbol{\nu}^{(i-1)}.\mathsf{read}()$; $b \leftarrow \boldsymbol{\nu}^{(i-1)}.\mathsf{read}()$.
6. $\boldsymbol{\nu}^{(i)}.\mathsf{write}(a \cdot b)$.
7. Re-initialize $\boldsymbol{\nu}^{(i)}$ as a read-stream.
8. Return read-stream $\boldsymbol{\nu} :=$ $(\boldsymbol{\nu}^{(0)} \;||\; \boldsymbol{\nu}^{(1)} \;||\; \ldots \;||\; \boldsymbol{\nu}^{(n-1)})$.

Further details are provided in Section 5.3.2.

2.6.3 Multiset-equality-check

Given two n-variate multilinear polynomials p and q, the multiset-equality-check PIOP aims to prove that the evaluation tables over the boolean hypercube of p and q are equal as multisets. More precisely, it aims to prove the following claim: "$\{\!\{p(\boldsymbol{x}) : \boldsymbol{x} \in \{0,1\}^n\}\!\} = \{\!\{q(\boldsymbol{x}) : \boldsymbol{x} \in \{0,1\}^n\}\!\}$".

Reduction to prodcheck. This claim is proven via a reduction to a prodcheck claim: V sends a random challenge $\alpha \in \mathbb{F}$ to P, and then P and V engage in a prodcheck PIOP for the claim that $\prod_{\boldsymbol{x} \in \{0,1\}^n} (p(\boldsymbol{x}) + \alpha)/(q(\boldsymbol{x}) + \alpha) = 1$.

Streaming PIOP. The prodcheck PIOP is invoked on the polynomials $(p(\boldsymbol{x}) + \alpha)_{\boldsymbol{x} \in \{0,1\}^n}$ and $(q(\boldsymbol{x}) + \alpha)_{\boldsymbol{x} \in \{0,1\}^n}$. Streaming access to these polynomials can be easily simulated with streaming access to p and q, and therefore the reduction is in fact read-only streaming. In order to be more concretely efficient and avoid disk accesses, we leverage read-only streaming reductions whenever possible. Therefore, the reduction requires $O(N)$ time, and only $O(1)$ additional space (both streaming and random-access). Further details are provided in Section 5.3.3.

Multiset-equality-check with log derivatives. We also present an RW streaming version of this PIOP that relies on the PIOP of Haböck [Hab22], which reduces a multiset-equality instance to sumcheck and zerocheck instances directly. We also discuss some concrete optimizations for this PIOP through a reduction to a *sumcheck for rational functions* as described in Appendix A.

2.6.4 Permcheck

Given two n-variate multilinear polynomials p and q, and a permutation $\pi : \{0,1\}^n \to \{0,1\}^n$, the permcheck PIOP proves that "$p(\boldsymbol{x}) = q(\pi(\boldsymbol{x}))$ for all $\boldsymbol{x} \in \{0,1\}^n$".

Reduction to multiset-equality-check. This claim is proven via a reduction to a multiset-equality-check claim: V sends a random challenge $\beta \in \mathbb{F}$ to P, and then P and V engage in a multiset-equality-check PIOP for the claim $\{p(\boldsymbol{x}) + \beta \cdot \mathsf{int}(\boldsymbol{x})\}_{\boldsymbol{x}\in\{0,1\}^n} = \{q(\boldsymbol{x}) + \beta \cdot \mathsf{int}(\pi(\boldsymbol{x}))\}_{\boldsymbol{x}\in\{0,1\}^n}$.[12]

Streaming PIOP. Similar to the reduction from multiset-equality-check to prodcheck, the read-streams for both $\{p(\boldsymbol{x}) + \beta \cdot \mathsf{int}(\boldsymbol{x})\}_{\boldsymbol{x}\in\{0,1\}^n}$ and $\{p(\boldsymbol{x}) + \beta \cdot \mathsf{int}(\boldsymbol{x})\}_{\boldsymbol{x}\in\{0,1\}^n}$ can be simulated efficiently in a read-only streaming manner. For any $\boldsymbol{x} \in \{0,1\}^n$, P can compute $p(\boldsymbol{x}) + \beta \cdot \mathsf{int}(\boldsymbol{x})$ and $q(\boldsymbol{x}) + \beta \cdot \mathsf{int}(\pi(\boldsymbol{x}))$ in constant time and space since it has access to $p(\boldsymbol{x}), q(\boldsymbol{x}), \mathsf{int}(\boldsymbol{x}), \mathsf{int}(\pi(\boldsymbol{x}))$. Therefore, both streams can be produced in $O(N)$ time, and only $O(1)$ additional space (both streaming and random-access). Further details are provided in Section 5.3.4.

2.7 PIOP for HyperPlonk

Recall from Section 2.2 that HyperPlonk's PIOP reduces circuit satisfiability to a permcheck claim and a zerocheck claim. We show in Section 5.3 how to construct a RW streaming prover for this PIOP that requires $O(N)$ time, $O(\log N)$ random-access space, and $O(N)$ streaming space complexity and satisfies Corollary 2.4. The time and space complexities of the aforementioned PIOPs are summarized in Table 1.

PIOP	time	space	
		random-access	streaming
multilinear sumcheck	$O(N)$	$O(1)$	$O(N)$
(d,ℓ)-sumprod sumcheck	$O(d\ell N)$	$O(d+\ell)$	$O(dN)$
(d,ℓ)-sumprod zerocheck	$O(d\ell N)$	$O(d+\ell+\log N)$	$O(dN)$
prodcheck	$O(N)$	$O(\log N)$	$O(N)$
multiset-equality	$O(N)$	$O(\log N)$	$O(N)$
permcheck	$O(N)$	$O(\log N)$	$O(N)$
HyperPlonk	$O(N)$	$O(\log N)$	$O(N)$

Table 1: Efficiency of our read-write streaming PIOPs. All complexities are in terms of number of field elements/operations. Verifier time, query complexity, and soundness error are the same as their HyperPlonk counterparts [CBBZ23].

2.8 Read-write streaming polynomial commitment schemes

We construct read-write streaming variants of a number of popular multilinear polynomial commitment schemes, that preserve cryptographic linear-time complexity while reducing the random-access space required to just $O(\log N)$. Our results are summarized in Table 2.

We now provide high-level overviews of the techniques used to obtain the results in Table 2. We begin in Section 2.9.1 by describing the read-write streaming variant of the scheme of Papamanthou, Shi, and Tamassia (PST) [PST13]. Then, in Section 2.9.2, we show how to obtain a cryptographic linear-time RW streaming prover for generalized inner product arguments [BMMTV21], and then show how to use the latter to construct RW streaming variants of the schemes of Bowe et al. [BGH19] (Section 2.9.3), and of Wahby et

[12] The stream of $\mathsf{int}(\boldsymbol{x})$ can be easily simulated because it consists of integers from 0 to $N-1$. P also receives $\hat{\pi}$ as input, which is the multilinear extension of $\tilde{\pi}$, where $\tilde{\pi} : \{0,1\}^n \to \mathbb{F}$ is obtained by casting the output of π to an element of $\mathbb{F}$.

scheme	SRS size	time		streaming space		check time	proof size
		commit	open	commit	open		
PST13 [PST13]	$O(N)$	$O(N)$	$O(N)$	$O(1)$	$O(N)$	$O(\log N)$	$O(\log N)$
Hyrax [WTSTW18]	$O(1)$	$O(N)$	$O(N)$	$O(1)$	$O(\sqrt{N})$	$O(\sqrt{N})$	$O(\sqrt{N})$
Multilinear Halo [BGH19; Set20]	$O(1)$	$O(N)$	$O(N)$	$O(1)$	$O(N)$	$O(N)$	$O(\log N)$
BMMTV21 [BMMTV21]	$O(\sqrt{N})$	$O(N)$	$O(N)$	$O(\sqrt{N})$	$O(\sqrt{N})$	$O(\log N)$	$O(\log N)$
Dory [Lee21]	$O(\sqrt{N})$	$O(N)$	$O(N)$	$O(\sqrt{N})$	$O(\sqrt{N})$	$O(\log N)$	$O(\log N)$

Table 2: Efficiency of our read-write streaming polynomial commitment schemes for n-variate multilinear polynomials, where $N = 2^n$. All sizes are specified in number of group elements, and all time complexities in number of group operations.

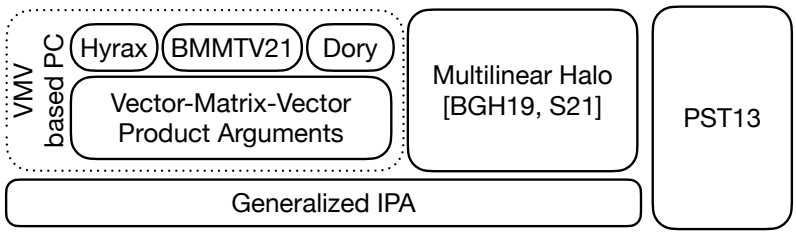

Figure 2: Our read-write streaming PC schemes.

al. [WTSTW18], Bünz et al. [BMMTV21] and of Lee [Lee21] (Section 2.9.4). In all cases, we focus on the commitment and opening algorithms, as these are the ones that are invoked by the RW streaming SNARK prover.

2.9 Building block: multi-scalar multiplication

A key component of all the aforementioned constructions is a *multi-scalar multiplication* (MSM) operation, which computes $\sum_{i=1}^{N} a_i \cdot G_i$ for given scalars (field elements) $a_1, \ldots, a_N$ and group elements $G_1, \ldots, G_N$. Given streaming access to $\boldsymbol{a}$ and $\boldsymbol{G}$, clearly this operation can be performed in a read-only streaming manner in cryptographic linear time: simply stream through $\boldsymbol{a}$ and $\boldsymbol{G}$ in parallel, and compute the sum incrementally. For a group of size $O(2^\lambda)$, this requires N scalar multiplications. Since each of the latter requires $O(\lambda)$ group additions, the overall cost is $O(\lambda N)$ group additions.

Fast read-only streaming MSMs. Pippenger's algorithm [Pip80] computes an MSM of size N in a group $\mathbb{G}$ of size $O(2^\lambda)$ with just $O(\lambda \cdot N / \log(\lambda \cdot N))$ group additions. We can apply this algorithm "in chunks" to construct a read-only streaming MSM algorithm that uses m random-access space and $O(\lambda N / \log(\lambda m))$ group additions as follows: define a running MSM output $\mathsf{ans} := 0$. In a streaming manner, read m field and group elements of the input at a time, compute their MSM using Pippenger's algorithm, and add the output to ans. After N/m iterations, output ans.

2.9.1 Read-write streaming variant of PST

The PST scheme [PST13] generalizes the KZG scheme [KZG10] to multilinear polynomials. We recall Libra's [XZZPS19] version of this scheme, and then describe how to make it RW streaming.

Setup. Sample a bilinear group $\langle \mathsf{group} \rangle = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, q, G, H, e) \leftarrow \mathsf{SampleGrp}(1^\lambda)$, and output the commitment key $\mathsf{ck} := ([\mathsf{ck}_j]_{j=0}^{n-1})$, where $\mathsf{ck}_j := [\mathrm{eq}_j(\boldsymbol{\alpha}_{[n-j+1:n-1]}, \boldsymbol{i}) \cdot G]_{\boldsymbol{i} \in \{0,1\}^j}$, and $\boldsymbol{\alpha} \xleftarrow{\$} \mathbb{F}^n$ is uniformly random. Each ck_j enables committing to a j-variate multilinear polynomial.[13]

Commit. Given the evaluations $\boldsymbol{p}$ of an n-variate multilinear polynomial p, compute $C := \langle \boldsymbol{p}, \mathsf{ck}_n \rangle$ via a streaming MSM.

Opening. For an n-variate multilinear polynomial $p(X_1, \ldots, X_n)$ and an evaluation point $\boldsymbol{z} \in \mathbb{F}^n$, PST relies on the fact that $p(\boldsymbol{z}) = v$ if and only if there exist polynomials $q_1(X_2, \ldots, X_n), q_2(X_3, \ldots, X_n), \ldots, q_{n-1}(X_n), q_n$ such that

$$p(\boldsymbol{X}) - v = \sum_{i=1}^{n} q_i(X_{i+1}, \ldots, X_n) \cdot (X_i - z_i) \ .$$

PST leverages this fact as follows: to prove that $p(\boldsymbol{z}) = v$, Open computes commitments $\pi_1, \ldots, \pi_n$ to the n 'witness' polynomials $q_1, \ldots, q_n$ with respect to keys $\mathsf{ck}_{n-1}, \ldots, \mathsf{ck}_0$ respectively. Since these polynomials are respectively of sizes $2^{n-1}, 2^{n-2}, \ldots, 1$, these commitments can be computed in overall cryptographic linear-time, and so we are left to reason about the complexity of computing the witness polynomials.

Zhang et al. [ZGKPP18, Appendix G] demonstrate a linear-time algorithm for computing these polynomials that relies on the following decomposition of the multilinear polynomial p:

$$\begin{aligned} p(X_1, \ldots, X_n) &= g(X_2, \ldots, X_n) + X_1 \cdot h(X_2, \ldots, X_n) \\ &= (g(X_2, \ldots, X_n) + z_1 \cdot h(X_2, \ldots, X_n)) \\ &\qquad + (X_1 - z_1) \cdot h(X_2, \ldots, X_n) \\ &:= r_1(X_2, \ldots, X_n) + (X_1 - z_1) \cdot q_1(X_2, \ldots, X_n) \end{aligned}$$

With q_1 in hand, Open proceeds to compute q_2 by recursively invoking the foregoing decomposition on the 'remainder' polynomial $r_1(X_2, \ldots, X_n)$. Then for each $j \in \{2, 3, \ldots, n\}$, Open recursively invokes the foregoing decomposition on the 'remainder' polynomial $r_{j-1}(X_j, \ldots, X_n)$ to obtain the witness polynomial $q_j(X_{j+1}, \ldots, X_n)$ and the next remainder polynomial $r_j(X_{j+1}, \ldots, X_n)$. Zhang et al. [ZGKPP18] show how to compute this decomposition in linear time.

Limitations of read-only streaming. It is straightforward to construct a read-only streaming algorithm for Commit, but doing the same for Open is more difficult. A naive adaptation of the algorithm of Zhang et al. would require $O(N \log N)$ time because, for each $i \in [\log N]$, it would have to compute from scratch and in linear time, the evaluations $\boldsymbol{q}_i$ of q_i.

We can avoid this by changing the order in which we produce $\boldsymbol{q}_i$ (similar to our streaming prodcheck algorithm in Section 2.6.2). Namely, we can view the computation that produces these polynomials as implicitly traversing a binary tree. The naive adaptation of the algorithm of Zhang et al. traverses this tree in a breadth-first manner, computing all the evaluations of a particular $\boldsymbol{q}_i$ before moving on to the next one. Traversing this tree in depth-first manner would reduce this cost to $O(N)$ time overall. Unfortunately, this approach produces evaluations of the q_i's in an interleaved manner (e.g. $q_1[0], q_2[0], \cdots$), and committing to these in a read-only streaming manner requires a similarly interleaved commitment key, thus doubling overall commitment key size.

RW streaming in cryptographic linear-time. We describe a simpler, cryptographic linear-time, RW streaming Open algorithm that avoids interleaving. Our construction follows from the observation that reversing the order of variable elimination in the algorithm of Zhang et al. enables one to easily produce streams for $\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n$; indeed, the algorithm resembles $\mathsf{RWFoldInPlace}$ from Section 2.5.1.

[13] To avoid ambiguity, we explicitly define $\mathsf{ck}_0 := G$.

Let $\boldsymbol{p}$, $\boldsymbol{g}$ and $\boldsymbol{h}$ be the evaluations (over the appropriate hypercubes) of $p(X_1, \ldots, X_n)$, $g(X_1, \ldots, X_{n-1})$ and $h(X_1, \ldots, X_{n-1})$ respectively. Let $\boldsymbol{i}$ be a point in $\{0,1\}^n$. When $i_n = 0$, $p(i_1, \ldots, i_{n-1}, 0) = g(i_1, \ldots, i_{n-1})$, and when $i_n = 1$, $p(i_1, \ldots, i_{n-1}, 1) = g(i_1, \ldots, i_{n-1}) + h(i_1, \ldots, i_{n-1})$, implying that $h(i_1, \ldots, i_{n-1}) = p(i_1, \ldots, i_{n-1}, 1) - p(i_1, \ldots, i_{n-1}, 0)$, and so we can set $\boldsymbol{g} := \boldsymbol{p}_E$ and $\boldsymbol{h} := \boldsymbol{p}_O - \boldsymbol{g}$, where $\boldsymbol{p}_E$ and $\boldsymbol{p}_O$ are the even and odd halves of $\boldsymbol{p}$, respectively.

Given streaming access to $\boldsymbol{p}$, the prover writes $\boldsymbol{g}$ and $\boldsymbol{h}$ onto intermediate write streams via the foregoing identities. The prover then sets $\boldsymbol{q}_1 := \boldsymbol{h}$ and commits to it. Finally, the prover computes $\boldsymbol{r}_1 := \boldsymbol{g} + z_1 \cdot \boldsymbol{h}$ using the streams for $\boldsymbol{g}$ and $\boldsymbol{h}$ and uses it to compute $\boldsymbol{q}_2, \ldots, \boldsymbol{q}_n$.

We further reduce I/O via the following **optimizations**:

- *Batch commitments:* Often, the prover needs to commit to multiple polynomials at once. Calling Commit on each polynomial separately would incur a fresh pass over the commitment key per polynomial. We instead provide a BatchCommit algorithm that performs a single pass over the commitment key, using the group element it has read to update the commitment for each polynomial in the batch.
- *Reduced I/O for* Open*:* Instead of producing intermediate streams for $\boldsymbol{g}$ and $\boldsymbol{h}$, Open can directly produce a stream for $\boldsymbol{r}_1$ and the commitment to $\boldsymbol{q}_1$: it reads two elements of $\boldsymbol{p}$ at a time, and first combines these to produce an element of $\boldsymbol{r}_1$, and then uses the first element to update $\boldsymbol{q}_1$'s commitment.

The full details of our RW streaming construction are presented in Section 6.2.1.

2.9.2 Building block: read-write streaming inner product arguments

A core building block for many polynomial commitment schemes [BCCGP16; WTSTW18; Lee21; BMMTV21; BGH19; BCMS20] is an inner product argument (IPA) [BCCGP16; BBBPWM18] or its generalization [LMR19; BMMTV21]. Thus, if we wish to design RW streaming variants of these polynomial commitments, it is essential to first design RW streaming variants of these IPAs.

Generalized Inner Product Arguments. We construct a RW streaming prover that requires only logarithmic random access space for the generalized inner product argument (GIPA) of [BMMTV21]. For the purposes of this section, an inner product argument allows a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that the inner product $\langle \boldsymbol{w}, \boldsymbol{x} \rangle = v$, where $\boldsymbol{w} \in \mathbb{F}^N$ is a private vector committed to in a Pedersen commitment C and $\boldsymbol{x} \in \mathbb{F}^N$ is a public vector shared by both $\mathcal{P}$ and $\mathcal{V}$.[14]

At a high level, the construction works as follows: $\mathcal{P}$ and $\mathcal{V}$ both receive as input (1) a commitment key consisting of a vector of group generators $\boldsymbol{G} \in \mathbb{G}^N$, (2) the Pedersen commitment $C := \sum_{i=1}^{n} w_i \cdot G_i = \langle \boldsymbol{w}, \boldsymbol{G} \rangle$, (3) the public vector $\boldsymbol{x}$, and (4) the claimed inner product value v. $\mathcal{P}$ additionally receives as input the private witness $\boldsymbol{w}$.

$\mathcal{P}$ and $\mathcal{V}$ engage in the following interactive protocol that reduces verifying the validity of the above claim to verifying the validity of a claim of half the size. (Recall that, for a vector $\boldsymbol{a}$, $\boldsymbol{a}_E$ is the vector containing the elements of $\boldsymbol{a}$ that appear at even indices, and $\boldsymbol{a}_O$ is the vector containing elements at odd indices.)

[14] Generalized inner product arguments consider statements of a more general form: $\boldsymbol{w}, \boldsymbol{x}$ do not need to be vectors over $\mathbb{F}$. Further details can be found in Appendix B.

1. $\mathcal{P}$ computes $C_+ := \langle \boldsymbol{w}_E, \boldsymbol{G}_O \rangle$ and $C_- := \langle \boldsymbol{w}_O, \boldsymbol{G}_E \rangle$.
2. $\mathcal{P}$ computes $v_+ := \langle \boldsymbol{w}_E, \boldsymbol{x}_O \rangle$ and $v_- := \langle \boldsymbol{w}_O, \boldsymbol{x}_E \rangle$.
3. $\mathcal{P}$ sends the cross-terms C_+, C_-, v_+, v_- to $\mathcal{V}$.
4. $\mathcal{V}$ samples $\alpha \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$.
5. $\mathcal{P}$ sets $\boldsymbol{w}' := \alpha \boldsymbol{w}_E + \boldsymbol{w}_O$.
6. $\mathcal{P}$ and $\mathcal{V}$ set

$$\begin{aligned} \boldsymbol{G}' &:= \alpha^{-1} \boldsymbol{G}_E + \boldsymbol{G}_O, \\ C' &:= C + \alpha C_+ + \alpha^{-1} C_-, \\ \boldsymbol{x}' &:= \alpha^{-1} \boldsymbol{x}_E + \boldsymbol{x}_O, \\ v' &:= v + \alpha v_+ + \alpha^{-1} v_-. \end{aligned}$$

This reduction guarantees, except with negligible probability over the choice of α, that if $C' = \langle \boldsymbol{w}', \boldsymbol{G}' \rangle$ and $v' = \langle \boldsymbol{w}', \boldsymbol{x}' \rangle$, then it must have been the case that $C = \langle \boldsymbol{w}, \boldsymbol{G} \rangle$ and $v = \langle \boldsymbol{w}, \boldsymbol{x} \rangle$. Clearly the prover in this reduction runs in cryptographic linear-time as it only performs inner-products and linear combinations.

In the full GIPA, $\mathcal{P}$ and $\mathcal{V}$ run this interactive reduction recursively for $\log N$ rounds until $\boldsymbol{w}'$ is of length $O(1)$, at which point $\mathcal{P}$ can send $\boldsymbol{w}'$ to $\mathcal{V}$ in the clear and $\mathcal{V}$ can check that $C' = \langle \boldsymbol{w}', \boldsymbol{G}' \rangle$ and $v' = \langle \boldsymbol{w}', \boldsymbol{x}' \rangle$. Since the vectors in each round are respectively of sizes $2^{n-1}, 2^{n-2}, \ldots, 1$, the whole protocol can be executed in cryptographic linear-time.

Additionally, since this protocol is public-coin, it can be turned non-interactive via the Fiat–Shamir transform [FS86].

Barrier to read-only streaming. In each of the $\log N$ rounds, $\mathcal{P}$ has to compute and send the cross terms C_+, C_-, v_+, v_- to $\mathcal{V}$. This requires access to the intermediate vectors $\boldsymbol{G}'$, $\boldsymbol{x}'$ and $\boldsymbol{w}'$, and the computation of these closely resembles the Fold step of the sumcheck protocol (see Section 2.5). Indeed, just like in sumcheck, the GIPA prover 'folds' the vectors $\boldsymbol{G}$, $\boldsymbol{x}$ and $\boldsymbol{w}$ in half with respect to the challenge α. As a result, a read-only streaming version of the GIPA prover $\mathcal{P}$ would encounter the same bottleneck as the read-only streaming sumcheck prover: it would need to recompute the 'state' $\boldsymbol{G}'$, $\boldsymbol{x}'$ and $\boldsymbol{w}'$ for each round from scratch, which takes $O(N)$ work for each of the $\log N$ rounds. This is the approach taken by Block et al. [BHRRS20], and as a result they suffer from a $\log N$ overhead on the prover time.

Achieving RW streaming in cryptographic linear-time. We present a direct construction for a RW streaming GIPA prover that achieves cryptographic linear-time and logarithmic random-access space. Given streaming access to $\boldsymbol{G}$, $\boldsymbol{x}$ and $\boldsymbol{w}$, the RW streaming prover can compute the cross-terms as well as compute and write out $\boldsymbol{G}'$, $\boldsymbol{x}'$ and $\boldsymbol{w}'$ to an intermediate stream in a straightforward way using $O(N)$ group and field operations. It then iteratively applies the streaming reduction on these intermediate streams $\log N - 1$ more times, where the size of each stream is halved, resulting in a total of $O(N)$ operations. The prover only requires $O(1)$ random-access space to keep track of the pointers for the input and intermediate streams.

We note that there is another, more indirect path to obtaining a RW streaming GIPA prover: Bootle, Chiesa, and Sotiraki [BCS21] show how to interpret IPAs as *sumcheck arguments* where the prover's algorithm closely resembles the sumcheck prover's algorithm. We can exploit this interpretation by applying our techniques from Section 2.5 to obtain a RW streaming GIPA prover with the same asymptotic efficiency as our direct construction.

2.9.3 PC schemes directly from inner product arguments

Evaluating a multilinear polynomial $p(\boldsymbol{X})$ at a point $\boldsymbol{z}$ is equivalent to computing the inner product $\langle \boldsymbol{p}, \mathrm{eq}_{\boldsymbol{z}} \rangle = \sum_{\boldsymbol{b} \in \{0,1\}^n} p(\boldsymbol{b}) \mathrm{eq}(\mathbf{z}, \boldsymbol{b})$, where $\boldsymbol{p} := [p(\mathsf{bin}(i))]_{i=0}^{2^n-1}$ is the evaluation of the polynomial over

its corresponding boolean hypercube and $\mathrm{eq}_{\boldsymbol{z}} := [\mathrm{eq}(\boldsymbol{z}, \mathsf{bin}(i))]_{i=0}^{2^n-1}$ (i.e. the vector consisting of the $\boldsymbol{i}$-th Lagrange polynomials evaluated at $\boldsymbol{z}$, for all $\boldsymbol{i} \in \{0,1\}^n$). Thus, an IPA immediately implies a polynomial commitment scheme:

- PC.Setup: Sample the commitment key $\boldsymbol{G} \xleftarrow{\$} \mathbb{G}^N$.
- PC.Commit: Compute a Pedersen-style commitment to the polynomial $\boldsymbol{p}$ as $C := \langle \boldsymbol{p}, \boldsymbol{G} \rangle$.
- PC.Open: Use the GIPA to prove that $p(\boldsymbol{z}) = v$ by proving $C = \langle \boldsymbol{p}, \boldsymbol{G} \rangle$ and $v = \langle \boldsymbol{p}, \mathrm{eq}_{\boldsymbol{z}} \rangle$.

Clearly, the RW streaming IPA from Section 2.9.2 implies RW streaming versions of the commitment and opening algorithms, with the only subtlety arising in the computation of $\mathrm{eq}_{\boldsymbol{z}}$, which we demonstrated how to compute in a read-only streaming manner in Section 2.6.1.

This is precisely the PC scheme from [BGH19]. It requires linear verifier time and a linear-sized commitment key. We describe next RW streaming algorithms for PC schemes that avoid these drawbacks.

2.9.4 PC schemes from VMV products

Background: polynomials as matrices. Wahby et al. [WTSTW18] proposed an alternate recipe for constructing polynomial commitment schemes from inner product arguments, where the size of the commitment key is sublinear in the size of the polynomial being committed to. At a high level, the recipe proceeds by viewing an n-variate multilinear polynomial $p(\boldsymbol{X})$, represented by its evaluation over the boolean hypercube $\boldsymbol{p}$, as a matrix $\mathbf{M} \in \mathbb{F}^{\sqrt{N} \times \sqrt{N}}$ defined as follows:

$$M_{ij} := p_k \text{ for } k = i \cdot 2^m + j \quad ,$$

where $N := 2^n$ and $m := n/2$.

They observe that evaluating $p(X_1, \ldots, X_n)$ at a point $\boldsymbol{z} = (z_1, z_2, \ldots, z_n)$ is equivalent to computing the vector-matrix-vector (VMV) product $\boldsymbol{\ell}^\top \mathbf{M} \boldsymbol{r}$, where $\boldsymbol{\ell} := [\mathrm{eq}(\boldsymbol{z}_L, \mathsf{bin}_n(i))]_{i=0}^{\sqrt{N}-1} = \otimes_{i=1}^{m}(1 - z_i, z_i)$ and $\boldsymbol{r} := [\mathrm{eq}(\boldsymbol{z}_R, \mathsf{bin}_n(i))]_{i=0}^{\sqrt{N}-1} = \otimes_{i=m+1}^{n}(1 - z_i, z_i)$.[15]

This can be illustrated as follows: consider a multilinear polynomial in four variables $p(X_1, X_2, X_4, X_4)$. Then for any $\mathbf{z} \in \mathbb{F}^4$ we can write $p(\mathbf{z}) = \sum_{\boldsymbol{b} \in \{0,1\}^4} p(\boldsymbol{b}) \prod_{i \in [4]} (1 - z_i)^{1-b_i} z_i^{b_i}$. Now consider the following vectors:

$$\begin{aligned}
\boldsymbol{\ell} &= (1 - z_1, z_1) \otimes (1 - z_2, z_2) = ((1 - z_1)(1 - z_2), (1 - z_1)z_2, z_1(1 - z_2), z_1 z_2), \\
\boldsymbol{r} &= (1 - z_3, z_3) \otimes (1 - z_4, z_4) = ((1 - z_3)(1 - z_4), (1 - z_3)z_4, z_3(1 - z_4), z_3 z_4), \\
\mathbf{M} &= \begin{bmatrix} p_0 & p_1 & p_2 & p_3 \\ p_4 & p_5 & p_6 & p_7 \\ p_8 & p_9 & p_{10} & p_{11} \\ p_{12} & p_{13} & p_{14} & p_{15} \end{bmatrix} = \begin{bmatrix} \boldsymbol{M}_0 \\ \boldsymbol{M}_1 \\ \boldsymbol{M}_2 \\ \boldsymbol{M}_3 \end{bmatrix}
\end{aligned}$$

It can be inspected that $\boldsymbol{\ell}^\top \mathbf{M} \boldsymbol{r} = \sum_{\boldsymbol{b} \in \{0,1\}^4} p(\boldsymbol{b}) \prod_{i \in [4]} (1 - z_i)^{1-b_i} z_i^{b_i} = p(\mathbf{z})$. This idea leads to the following blueprint for building polynomial commitment schemes:

Setup. Sample the commitment key $\boldsymbol{G} \xleftarrow{\$} \mathbb{G}^{\sqrt{N}}$.

Commit. To commit to the polynomial $\boldsymbol{p}$, Commit commits to the corresponding matrix $\mathbf{M}$ by Pedersen committing to each row $\boldsymbol{M}_i$ as $C_i := \langle \boldsymbol{M}_i, \boldsymbol{G} \rangle$, obtaining $\sqrt{N}$ row commitments. Some schemes like Hyrax [WTSTW18] directly use these row commitments $\boldsymbol{C} := [C_i]_{i=0}^{\sqrt{N}-1}$ as a $\sqrt{N}$-sized commitment to the

[15] The $\otimes$ operation denotes the Kronecker product. Given vectors $\boldsymbol{x} \in \mathbb{F}^N$ and $\boldsymbol{y} \in \mathbb{F}^M$, $\boldsymbol{x} \otimes \boldsymbol{y} := [x_1 \cdot \boldsymbol{y}, x_2 \cdot \boldsymbol{y}, \ldots, x_N \cdot \boldsymbol{y}]$.

polynomial. Other schemes, like Dory [Lee21] and that of Bünz et al. (BMMTV21) [BMMTV21], further commit to these row commitments (which are group elements) via a structure-preserving commitment scheme in groups with bilinear pairings [AFGHO16] to obtain a constant-sized commitment C to the matrix $\mathbf{M}$. This step imposes marginal overhead in the prover time and commitment key size. For simplicity of exposition, below we focus on Hyrax and defer the discussion of how to achieve RW streaming algorithms for Dory and BMMTV21 to Appendices B to D.

Opening. To prove that $p(\boldsymbol{z}) = v$, Open uses a 'vector-matrix-vector' product argument that allows a prover to convince a verifier that $\boldsymbol{\ell}^\top \mathbf{M} \boldsymbol{r} = v$ for a matrix $\mathbf{M}$ committed via the commitment $\boldsymbol{C}$, where $\boldsymbol{\ell}$ and $\boldsymbol{r}$ are public vectors constructed from $\boldsymbol{z}$ as above.

It is easy to see that $\boldsymbol{\ell}^\top \mathbf{M} = \sum_{i=0}^{\sqrt{N}-1} \ell_i \cdot \boldsymbol{M}_i$ and $\boldsymbol{\ell}^\top \mathbf{M} \boldsymbol{r} = \langle \sum_{i=0}^{\sqrt{N}-1} \ell_i \cdot \boldsymbol{M}_i, \boldsymbol{r} \rangle = v$. Thus, Hyrax's vector-matrix-vector product argument proceeds by having the verifier directly compute the commitment C to the vector $\boldsymbol{\ell}^\top \mathbf{M}$ by computing the linear combination of the row commitments with the $\boldsymbol{\ell}$ vector, $C := \sum_{i=0}^{\sqrt{N}-1} \ell_i \cdot C_i$. Open then uses an IPA to produce a proof that the inner product of the resulting committed vector C with $\boldsymbol{r}$ equals the claimed evaluation v.

Since vector-matrix multiplication can be computed in time $O(N)$ for matrices of dimension $\sqrt{N} \times \sqrt{N}$, and running an IPA over vectors of length $\sqrt{N}$ only takes $O(\sqrt{N})$ cryptographic time, Open requires $O(N)$ (*non-cryptographic*) time overall.

Barrier to read-only streaming. It seems that we cannot simultaneously achieve (cryptographic) linear-time and logarithmic random-access space for both the Commit and Open algorithms of VMV PC schemes. In particular, given access to the matrix $\mathbf{M}$ in row-major order, the Commit algorithm can be implemented in a read-only streaming manner with only logarithmic random-access memory. On the other hand, Open computes the vector-matrix product $\boldsymbol{\ell}^\top \mathbf{M}$, and streaming computation of the latter requires a *column-major-order* stream of $\mathbf{M}$. Since the surrounding application (in this case the SNARK) only provides row-major access to $\mathbf{M}$, Open would need to compute the transpose of $\mathbf{M}$ to obtain column-major access to $\mathbf{M}$, and even the best *in-place* (i.e. non-streaming) algorithms for this task require $O(N)$ time and space.

Achieving RW streaming in cryptographic linear-time. We now outline how to construct a RW streaming algorithm for Open that achieves linear-time and logarithmic random-access space by leveraging just *two* intermediate write-streams of size $\sqrt{N}$. Our algorithm avoids the need for matrix transposition entirely.

Given streaming access to $\boldsymbol{\ell}$ and to the rows of the matrix $\mathbf{M}$ (denoted by $\boldsymbol{M}_0, \ldots, \boldsymbol{M}_{\sqrt{N}-1}$), the algorithm starts by reading ℓ_0, streaming through $\boldsymbol{M}_0$, and computing and writing out $\ell_0 \cdot \boldsymbol{M}_0$ to an intermediate read-write stream $\mathbf{W}$. In the next iteration, it reads ℓ_1 and streams through both the next row $\boldsymbol{M}_1$ *and* $\mathbf{W}$, and updates $\mathbf{W}$ to contain $\ell_0 \cdot \boldsymbol{M}_0 + \ell_1 \cdot \boldsymbol{M}_1$ by adding in the product $\ell_1 \cdot \boldsymbol{M}_1$.[16] This process continues until $\mathbf{W}$ contains the desired output $\ell_0 \cdot \boldsymbol{M}_0 + \ldots + \ell_{\sqrt{N}-1} \cdot \boldsymbol{M}_{\sqrt{N}-1}$. Clearly, throughout this process, the algorithm only consumes $O(\log N)$ random access space and $O(\sqrt{N})$ streaming space (for $\mathbf{W}$). We describe the full construction of Hyrax in Section C.3.

[16] To be more precise, because our model does not allow simultaneous read-write access to the same stream, the algorithm would need to use two intermediate read-write streams $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$, and alternate between them in each iteration to obtain the desired sum: once the algorithm has written out $\ell_0 \cdot \boldsymbol{M}_0$ onto $\boldsymbol{W}_1$, it must stream through both the running sum in $\boldsymbol{W}_1$ as well as $\boldsymbol{M}_1$ and compute and write out the new running sum $\ell_0 \cdot \boldsymbol{M}_0 + \ell_1 \cdot \boldsymbol{M}_1$ onto $\boldsymbol{W}_2$. It would then read this new running sum from $\boldsymbol{W}_2$ as well as ℓ_2 and $\boldsymbol{M}_2$ to write out $\ell_0 \cdot \boldsymbol{M}_0 + \ell_1 \cdot \boldsymbol{M}_1 + \ell_2 \cdot \boldsymbol{M}_2$ onto $\boldsymbol{W}_1$, and so on.

3 Related work

3.1 Read-only streaming SNARKs

A recent line of work attempts to tackle the prover space bottleneck by constructing 'streaming' SNARKs whose prover requires only a small amount of random-access space, and can only access the circuit wire values in a *read-only* streaming manner. This model differs from ours in that the prover does not have access to intermediate RW streams. Below, we discuss these works that construct such streaming SNARKs, and remark that they sacrifice on either prover or verifier time.

Comparison of frameworks. RW streaming enables more efficient composition of streaming algorithms (see Section 4.1) as the output of subroutines can be stored in external memory and reused later. This enables better concrete efficiency for RW streaming SNARKs compared to read-only streaming SNARKs: the SNARK prover $\mathcal{P}$ must feed the polynomials output by the PIOP prover to both $\mathsf{PC.Commit}$ and $\mathsf{PC.Open}$. In the RW setting, $\mathcal{P}$ can write these polynomials to external memory and then feed the input to both PC algorithms; in the read-only setting $\mathcal{P}$ must recompute them.

Block et al. [BHRRS20; BHRRS21] construct read-only streaming SNARKs by designing a streaming PIOP and combining it with streaming PC schemes from inner-product arguments [BHRRS20] or groups of unknown order [BHRRS21]. Both works achieve logarithmic random-access space complexity for the prover, but incur quasi-linear time complexity. They do not implement their SNARK, and so we cannot provide concrete efficiency comparisons. Our RW streaming inner-product argument is inspired by the read-only streaming one in [BHRRS20], but, unlike the latter, is able to achieve cryptographic linear time complexity.

Gemini [BCHO22] generalizes the notion of streaming SNARKs and introduces *elastic* SNARKs that have prover algorithms optimized for two different memory regimes. The first regime is a memory-intensive one where the focus is on minimizing prover time, while the second regime is the read-only streaming setting where the focus is on minimizing prover memory. The latter regime is the relevant point of comparison for Scribe. We provide a quantitative comparison with Gemini's streaming prover in Section 8, and so focus here on a qualitative comparison. On an instance of size N, Gemini's streaming prover achieves $O(N \log^2 N)$ prover time with logarithmic prover memory, and does so by applying their analogue of the 'PIOP + PC $\rightarrow$ SNARK' transformation [CHMMVW20; BFS20] to streaming PIOPs and streaming PC schemes that they construct. Attempting to use read-write streams to improve the efficiency of their SNARK does not seem to work, as their PIOP requires multiplying a random vector by a sparse matrix, and this seems to inherently require quasi-linear time in a low-memory setting.

Epistle [ZCLKZ24] designs a new SNARK that improves upon Gemini: its streaming prover requires only $O(N \log N)$ time without increasing prover memory. Like Scribe, Epistle's starting point is HyperPlonk [CBBZ23], but it switches out HyperPlonk's prodcheck PIOP in favor of a novel construction that is more amenable to read-only streaming. Like HyperPlonk's prodcheck, their new prodcheck PIOP also reduces to a sumcheck, but the key difference is that this reduction requires only $O(N)$ time even when restricted to $O(\log N)$ space. The key time complexity bottleneck of Epistle's PIOP is the sumcheck protocol, since it requires $O(N \log N)$ time in the read-only streaming model. Epistle's code is not publicly available, but we were able to obtain a private copy, and compare against it in Section 8.

Baweja et al. [Baw+25] introduce a read-only streaming sumcheck prover for multilinear polynomials which requires $O(N^{1/k})$ space and $O(kN)$ time for a tunable parameter k. Setting $k = \log N / \log \log N$ gives a $O(\log N)$ space and $O(N \log N / \log \log N)$ time sumcheck prover. They show this is optimal via an unconditional matching lower bound.

Sparrow [PP24] is a read-only streaming SNARK whose prover requires $O(\sqrt{N})$ random-access space and $O(N \log \log N)$ time. Sparrow only supports data-parallel circuits. On a technical level, Sparrow introduces a

new sumcheck protocol for products of multilinear polynomials (different from the standard one), and designs a read-only streaming PIOP for it.

Hobbit [PP25] designs a sumcheck protocol for products of multilinear polynomials which requires $O(N)$ time and $O(\sqrt{N})$ random-access space, albeit at the cost of an increased proof size, and round and verifier complexity of $O(\sqrt{N})$.

3.2 Complexity-preserving SNARKs

Complexity-preserving SNARKs [BC12] impose at most a poly-logarithmic multiplicative overhead in prover time and space over the corresponding costs for the computation being proven. We recap some relevant recent works in this space.

Low-memory SNARKs via IVC. Incrementally-verifiable computation (IVC) is the most popular means of constructing a complexity-preserving SNARK [BCCT12]. Unfortunately, IVC-based approaches generally only support uniform computations and require non-black-box use of cryptography. The latter leads to further issues such as concrete efficiency overheads in some cases, difficulties in achieving provable security, and a need for heuristics due to the reliance on random oracles, all stemming Nevertheless, IVC-based complexity-preserving SNARKs achieve good concrete efficiency.

Mangrove [NDCTB24] builds a complexity-preserving SNARK by reducing circuit-satisfiability to a uniform computation, and applying efficient accumulation/folding-based IVC schemes [BCLMS21; KST22] to this uniform computation. Mangrove's prover can be seen as a read-only streaming prover that makes two passes over the witness, and requires $O(\log N)$ random-access space. Mangrove does not provide an implementation to compare against, and moreover suffers from many of the aforementioned limitations of IVC-based SNARKs.

Ligetron [WHV24] Ligetron builds an argument of knowledge with prover time $O(N \log N)$ and $O(\sqrt{N})$ random-access space, improving upon a prior theoretical work of Bangalore et al. [BBHV22]. Unlike Scribe, Ligetron does not have a succinct verifier. Ligetron achieves good concrete efficiency.

4 Read-write streaming algorithms

We define read-write (RW) streams [GKS05; BJR07; PY14] in detail in Definition 4.1, and describe the behavior of RW streaming algorithms in Definition 4.2. Subsequently, we describe common RW streaming algorithms in Section 4.2 that will be useful subroutines when designing RW streaming PIOPs and PC schemes in later sections.

Definition 4.1. *A* read-write stream $\mathbf{S}$ *is a tuple* $(\boldsymbol{a}, p, m, \ell)$ *where* $\boldsymbol{a}$ *is an underlying ordered set,* p *is a pointer to the ordered set, and* $m \in \{\mathsf{r}, \mathsf{w}\}$ *specifies whether* $\mathbf{S}$ *is in read mode (i.e.* $m = \mathsf{r}$*) or write mode (i.e.* $m = \mathsf{w}$*), and* ℓ *is the length of the underlying ordered set. Read-write streams support the following operations:*

- $\mathbf{S}.\mathsf{read}()$*: if* $\mathbf{S}.m = \mathsf{r}$*, return the element of* $\mathbf{S}.\boldsymbol{a}$ *at the current position of* $\mathbf{S}.p$*, and move* $\mathbf{S}.p$ *to the next element of* $\mathbf{S}.\boldsymbol{a}$*.*
- $\mathbf{S}.\mathsf{write}(w)$*: if* $\mathbf{S}.m = \mathsf{w}$*, write* w *at the current position of* $\mathbf{S}.p$*, and move* $\mathbf{S}.p$ *to the next element of* $\mathbf{S}.\boldsymbol{a}$*.*
- $\mathbf{S}.\mathsf{init}(N)$*: initialize* $\mathbf{S} = (\boldsymbol{a}, p, \mathsf{w}, N)$ *in write mode by allocating a contiguous space for an array of* N *elements* $\boldsymbol{a}$*, and initialize* $\mathbf{S}.p$ *at the beginning of the allocated space. This space is not necessarily initialized with default values.*
- $\mathbf{S}.\mathsf{restart}()$ *resets* $\mathbf{S}.p$ *to the beginning of* $\mathbf{S}.\boldsymbol{a}$*.*
- $\mathbf{S}.\mathsf{swapmode}()$ *resets* $\mathbf{S}.p$ *to the beginning of* $\mathbf{S}.\boldsymbol{a}$ *and toggles* $\mathbf{S}.m$ *between* r *and* w*.*
- $\mathbf{S}.\mathsf{len}()$ *returns the length of the underlying set* $\mathbf{S}.\ell$*.*

Elements of underlying set. In the RW streaming algorithms described in subsequent sections, the underlying ordered set $\boldsymbol{a}$ of a RW stream $\mathbf{S}$ always contains elements from a field $\mathbb{F}$ or group $\mathbb{G}$, or constant-sized tuples thereof.

RW stream notation. We will typically denote a RW stream that only uses read mode as $\mathbf{R}$, a RW stream that only uses the write mode as $\mathbf{W}$, and a RW stream that uses both modes as $\mathbf{S}$. Additionally, we use $\mathbf{R}(\boldsymbol{a})$ to denote a RW stream in read mode where the underlying ordered set is $\boldsymbol{a}$ and the underlying pointer $\mathbf{S}.p$ is initialized to the beginning of $\boldsymbol{a}$.

Definition 4.2. *A read-write (RW) streaming algorithm* $\mathcal{A}(\mathbf{I}) \mapsto \mathbf{O}$ *is an algorithm that*

- *takes as input RW stream* $\mathbf{I}$ *of length* N *and a RW stream* $\mathbf{O}$ *onto which it writes its output.*
- *has at most* $m(N)$ *amount of random-access space that it can read and write to.*
- *can allocate intermediate streams using* init *and perform read/write operations on them in a streaming manner using* $\mathsf{read}()$*,* $\mathsf{restart}()$*, and* $\mathsf{write}(\cdot)$*.*

Henceforth, when we say streaming*, we mean read-write streaming unless otherwise specified.*

The *random-access space complexity* of $\mathcal{A}$ is the amount of random-access space $m(N)$ allocated by $\mathcal{A}$. The *streaming space complexity* of $\mathcal{A}$ is $\sum_{i=1}^{k} \ell_i$ where ℓ_i is the length of the i-th intermediate stream allocated by $\mathcal{A}$. Both these complexities specifically count the amount of space *allocated* by $\mathcal{A}$, and not the total amount of space *used*. The distinction between the two is important because the former does not count the space required to store the input or output of $\mathcal{A}$ to avoid over-counting the space used when *composing* algorithms.

Restrictions of our model. Definition 4.2 places some restrictions on RW streaming algorithms: namely, it assumes that input streams are immutable, and that algorithms have a single input and output stream. These restrictions are without loss of generality, as detailed in Appendix F

The external memory model. The external memory (EM) model [AV88], like the read-write streaming model, captures algorithms whose inputs (and intermediate state) are too large to fit into a computer's main

memory at once, and are hence stored in external memory. Unlike the RW streaming model, the EM model allows algorithms *random-access* to the external memory. However, these accesses must occurs in large batches, and the I/O cost of the algorithm is measured in terms of the number of such batches. By batching I/O, an EM algorithm amortizes I/O cost over many elements.

We show in Appendix E that, even though read-write streaming algorithms perform I/O one element at a time, their streaming nature enables straightforward batching, and hence also good EM I/O complexity.

4.1 Composition of RW streaming algorithms

Let $\mathcal{A}(\mathbf{I}_1) \mapsto \mathbf{O}_1$ and $\mathcal{B}(\mathbf{I}_2) \mapsto \mathbf{O}_2$ be two RW streaming algorithms. We say that $\mathcal{C}(\mathbf{I}_1) \mapsto \mathbf{O}_2$ is the *composition* of $\mathcal{A}$ and $\mathcal{B}$ if it invokes $\mathcal{A}$ to obtain the input for $\mathcal{B}$, and then invokes $\mathcal{B}$ on the latter to obtain the final output. That is, it: (i) allocates space for $\mathbf{O}_1$; (ii) calls $\mathcal{A}(\mathbf{I}_1) \mapsto \mathbf{O}_1$; (iii) once $\mathcal{A}$ terminates, calls $\mathbf{O}_1$.swapmode(); and (iv) invokes $\mathcal{B}(\mathbf{O}_1) \mapsto \mathbf{O}_2$.

This notion of composition can be used to implement RW streaming algorithms that call other RW streaming algorithms as subroutines. We say that $\mathcal{A}$ calls $\mathcal{B}$ as a subroutine if $\mathcal{A}$ can be split into two parts, $\mathcal{A}_1$ and $\mathcal{A}_2$, so that $\mathcal{A}$ is the composition of $\mathcal{A}_1$, $\mathcal{B}$, and $\mathcal{A}_2$. This naturally extends to multiple subroutine calls. The cost of subroutine calls is detailed below:

Lemma 4.3 (complexity of subroutine calls)**.** *Let $\mathcal{A}, \mathcal{B}$ be read-write streaming algorithms. For each algorithm $\mathcal{X} \in \{\mathcal{A}, \mathcal{B}\}$ let its time complexity be $t_{\mathcal{X}}$, its random-access space complexity be $m_{\mathcal{X}}$, and its streaming space complexity be $m_{\mathcal{X}}$. Let L be the number of times that $\mathcal{A}$ calls $\mathcal{B}$ as a subroutine. Then $\mathcal{A}$ composed with $\mathcal{B}$ has*

- *time complexity $t_{\mathcal{A}} + L \cdot t_{\mathcal{B}}$,*
- *random-access space complexity $m_{\mathcal{A}} + m_{\mathcal{B}}$, and*
- *streaming space complexity $s_{\mathcal{A}} + s_{\mathcal{B}}$.*

Proof. Let the i-th unique call to $\mathcal{B}$ be $\mathcal{B}(\mathbf{R}_i) \mapsto \mathbf{W}_i$. $\mathcal{A}$ prepares each $\mathbf{R}_i$ and allocates space for each $\mathbf{W}_i$. The time and space required for these operations is already included in $t_{\mathcal{A}}$, $m_{\mathcal{A}}$, and $s_{\mathcal{A}}$. Each call to $\mathcal{B}$ takes $t_{\mathcal{B}}$ time, which implies a total additional time of $L \cdot t_{\mathcal{B}}$. However, each call to $\mathcal{B}$ also requires $m_{\mathcal{B}}$ random-access space and $s_{\mathcal{B}}$ streaming space, but this space can be reclaimed after each call to $\mathcal{B}$. Therefore the additional random-access space and streaming space required are only $m_{\mathcal{B}}$ and $s_{\mathcal{B}}$ respectively. □

Remark 4.4 (Skipping allocation of intermediate streams)**.** Say RW streaming algorithm $\mathcal{A}(\mathbf{I}) \mapsto \mathbf{O}$ invokes $\mathcal{B}(\mathbf{I}) \mapsto \mathbf{S}$, and then invokes $\mathcal{C}(\mathbf{S}) \mapsto \mathbf{O}$. As stated, our model requires that $\mathcal{A}$ allocate an intermediate stream $\mathbf{S}$ that is provided as input to both $\mathcal{B}$ and $\mathcal{C}$. However, if $\mathcal{C}$ only makes a single pass over $\mathbf{S}$, then we can implement $\mathcal{A}$ without allocating space for $\mathbf{S}$ at all by directly "piping" the output of $\mathcal{B}$ to $\mathcal{C}$, similar to the composition of read-only streaming algorithms. We denote that such an optimization is taking place via the arrow '$\rightsquigarrow$'. With this notation, $\mathcal{A}$ can be written as $\mathcal{A} := \mathbf{O} \leftarrow \mathcal{C} \rightsquigarrow \mathcal{B} \rightsquigarrow \mathbf{I}$.

4.2 Common RW streaming subroutines

We define below some helper functions that we use in our streaming algorithms in the rest of the paper.

- Zip takes as input k streams $\mathbf{R}_1, \ldots, \mathbf{R}_k$ of length N containing k ordered sets, and outputs a stream $\mathbf{W}$ whose i-th element is a tuple of k elements, where the j-th element of the tuple is the i-th element of $\mathbf{R}_j$.
- Map takes as input a stream $\mathbf{R}$ of length N and a function f, and outputs a stream $\mathbf{W}$ which contains the result of applying f to each element of $\mathbf{R}$.

- Reduce takes as input a stream $\mathbf{R}$ of length N, an accumulator s, and a function f, and produces a single element that is the result of reducing the elements of $\mathbf{R}$ using f.

$\mathsf{Zip}(\mathbf{R}_1, \ldots, \mathbf{R}_k) \mapsto \mathbf{W}$:
1. For i in $[1, \ldots, \mathbf{R}_1.\mathsf{len}()]$:
2. $\quad \mathbf{W}.\mathsf{write}((\mathbf{R}_1.\mathsf{read}(),$
3. $\quad\quad \ldots, \mathbf{R}_k.\mathsf{read}())).$

$\mathsf{Map}(\mathbf{R}, f) \mapsto \mathbf{W}$:
1. For i in $[1, \ldots, \mathbf{R}.\mathsf{len}()]$:
2. $\quad \mathbf{W}.\mathsf{write}(f(\mathbf{R}.\mathsf{read}())).$

$\mathsf{Reduce}(\mathbf{R}, f, e) \mapsto s$:
1. $s \leftarrow e$.
2. For i in $[1, \ldots, \mathbf{R}.\mathsf{len}()]$:
3. $\quad s \leftarrow f(s, \mathbf{R}.\mathsf{read}())$.

Splitting read-write streams. Given a RW stream $\mathbf{R}$ of length $N = 2^n$, the indices of the underlying ordered set can be represented as n-bit binary strings. That is, the i-th element of the ordered set is represented by $\mathsf{bin}_n(i)$. Given an additional parameter k, we can define two methods for splitting $\mathbf{R}$ into 2^k streams, each with $N/2^k$ elements. SplitMSB splits $\mathbf{R}$ on the k most significant bits of the indices, while SplitLSB splits $\mathbf{R}$ on the k least significant bits of the indices.

$\mathsf{SplitMSB}(\mathbf{R}, k) \mapsto (\mathbf{W}_1, \ldots, \mathbf{W}_{2^k})$:
1. For j in $[1, \ldots, 2^k]$:
2. $\quad$ For i in $[1, \ldots, \mathbf{R}.\mathsf{len}()/2^k]$: $\mathbf{W}_j.\mathsf{write}(\mathbf{R}.\mathsf{read}())$.

$\mathsf{SplitLSB}(\mathbf{R}, k) \mapsto (\mathbf{W}_1, \ldots, \mathbf{W}_{2^k})$:
1. For i in $[1, \ldots, \mathbf{R}.\mathsf{len}()/2^k]$:
2. $\quad$ For j in $[1, \ldots, 2^k]$: $\mathbf{W}_j.\mathsf{write}(\mathbf{R}.\mathsf{read}())$.

We additionally define $\mathsf{SplitLR}(\mathbf{R}) \mapsto (\mathbf{W}_L, \mathbf{W}_R)$ and $\mathsf{SplitEO}(\mathbf{R}) \mapsto (\mathbf{W}_E, \mathbf{W}_O)$ that are specific cases of SplitMSB and SplitLSB respectively, where the output consists of exactly two streams. That is, SplitLR splits $\mathbf{R}$ into a stream of the first half of the elements of $\mathbf{R}$ and a stream of the second half of the elements, while SplitEO splits $\mathbf{R}$ into a stream of the elements with even indices and a stream of the elements with odd indices. We omit the pseudocode for these.

Joining read-write streams. We also define the "inverse" algorithms JoinLR and JoinEO that take as input two RW streams and respectively concatenate them or interleave them; we omit the pseudocode for these.

RWSum and RWFoldInPlace. RWSum takes as input a parameter d, and k RW streams which represent the evaluations of k many n-variate multilinear polynomials $p_1(\boldsymbol{X}), \ldots, p_k(\boldsymbol{X})$ on $\{0,1\}^n$. Consider the univariate k-degree polynomial $f(X_n) = \sum_{\boldsymbol{b} \in \{0,1\}^{n-1}} \prod_{i=1}^{k} p_i(\boldsymbol{b}, X_n)$. RWSum outputs $d+1$ evaluations $f(0), f(1), \ldots, f(d)$ of f.

RWFoldInPlace also takes as input k RW streams which represent the evaluations of k many n-variate multilinear polynomials $p_1(\boldsymbol{X}), \ldots, p_k(\boldsymbol{X})$ on $\{0,1\}^n$. The method folds each polynomial p_i into a $(n-1)$-variate multilinear polynomial q_i into a new stream of half the length based on the folding coefficients α and β. That is, $q_i(\boldsymbol{X}) = p_i(\boldsymbol{X}, 0) \cdot \alpha + p_i(\boldsymbol{X}, 1) \cdot \beta$. Note that the space needed to store input streams of RWFoldInPlace is freed at the end of the subroutine since the folded streams are moved into the input RW streams. This subroutine is used repeatedly in sumcheck-like protocols (such as in the polynomial commitment schemes described in Section C.3), where the output streams are the input streams for the next call to RWFoldInPlace, and there is no need to store the intermediate folded streams.

$\mathsf{RWSum}(d, \mathbf{R}_1, \ldots, \mathbf{R}_k) \mapsto S$:
1. Set $S \leftarrow [0]_{s=0}^{d}$.
2. For i in $[1, \ldots, \mathbf{R}.\mathsf{len}()/2]$:
3. $\quad$ For j in $[1, \ldots, k]$:
4. $\quad\quad a_{L,j} \leftarrow \mathbf{R}_j.\mathsf{read}(); a_{R,j} \leftarrow \mathbf{R}_i.\mathsf{read}()$.
5. $\quad$ For s in $[0, \ldots, d]$:
6. $\quad\quad S[s] \leftarrow S[s] + \prod_{j=1}^{k} ((1-s) \cdot a_{L,j} + s \cdot a_{R,j})$.

$\mathsf{RWFoldInPlace}(\alpha, \beta, \mathbf{R}_1, \ldots, \mathbf{R}_k)$:
1. Define the folding function: $f(a, b) := a \cdot \alpha + b \cdot \beta$.
2. For i in $[1, \ldots, k]$:
3. $\quad$ Set $\mathbf{W}_i \leftarrow \mathsf{Map}(f) \rightsquigarrow \mathsf{Zip} \rightsquigarrow \mathsf{SplitEO} \rightsquigarrow \mathbf{R}_i$.
4. $\quad$ Set $\mathbf{R}_i \leftarrow \mathbf{W}_i.\mathsf{swapmode}()$.

Inner products and linear combinations of streams. Given RW streams $\mathbf{R}_1$ and $\mathbf{R}_2$, we describe RW streaming algorithms to compute the inner product of the underlying vectors, and to compute a linear combination with respect to coefficients α and β:

$\mathsf{InnerProd}(\mathbf{R}_1, \mathbf{R}_2) \mapsto s$:
1. $s \leftsquigarrow \mathsf{Reduce}(0, +) \leftsquigarrow \mathsf{Map}(\times) \leftsquigarrow \mathsf{Zip} \leftsquigarrow (\mathbf{R}_1, \mathbf{R}_2)$.

$\mathsf{LinComb}(\alpha, \beta, \mathbf{R}_1, \mathbf{R}_2) \mapsto \mathbf{W}$:
1. Define the function $f(a, b) := \alpha \cdot a + \beta \cdot b$.
2. Output $\mathbf{W} \leftsquigarrow \mathsf{Map}(f) \leftsquigarrow \mathsf{Zip} \leftsquigarrow (\mathbf{R}_1, \mathbf{R}_2)$.

5 Read-write streaming PIOP for HyperPlonk

In this section we construct a read-write streaming PIOP for the HyperPlonk relation. We will follow the following roadmap to achieve this goal: (a) formally define indexed relations and PIOPs in Section 5.1, (b) develop read-write streaming PIOPs for different instances of the sumcheck relation in Section 5.2, (c) develop read-write streaming PIOPs for the zerocheck relation (Section 5.3.1) and the permcheck realtion (Section 5.3.4), which in turn leads to the construction of a read-write streaming PIOP for the HyperPlonk relation in Section 5.3.5.

5.1 Preliminaries

5.1.1 Indexed relations

An *indexed relation* $\mathscr{R}$ is a set of tuples $(\mathsf{i}, \mathsf{x}, \mathsf{w})$ where i is the index, x is the instance, and w is the witness. We use $\mathscr{R}_{\mathsf{i}}$ to denote the NP relation $\{(\mathsf{x}, \mathsf{w}) : (\mathsf{i}, \mathsf{x}, \mathsf{w}) \in \mathscr{R}\}$ and $\mathscr{L}_{\mathsf{i}}$ to denote the NP language corresponding to it. An indexed *oracle* relation is an indexed relation where the index i and the instance x contain 'implicit' inputs that are specified as oracles, i.e., the membership-checking algorithm for such a relation has only query access to these oracles. We adopt notation from Chen et al. [CBBZ23] and use $[\![z]\!]$ to denote when the input z is provided as an oracle.

5.1.2 Polynomial IOPs

A **Polynomial Interactive Oracle Proof** (PIOP) over a field family $\mathcal{F}$ for an indexed relation $\mathscr{R}$ is a tuple $\mathsf{PIOP} = (\mathsf{k}, \mathsf{s}, \mathsf{I}, \mathsf{P}, \mathsf{V})$ where $\mathsf{k}, \mathsf{s} \colon \{0,1\}^* \to \mathbb{N}$ are polynomial-time functions and $\mathsf{I}, \mathsf{P}, \mathsf{V}$ are three algorithms known as the *indexer*, *prover*, and *verifier*. The parameter k specifies the number of rounds of interaction between the prover and the verifier, and s specifies the number of polynomials in each round.

In the offline phase, before the instance and witness are specified, the indexer I receives as input a field $\mathbb{F} \in \mathcal{F}$ and an index i for $\mathscr{R}$, and outputs $\mathsf{s}(0)$ polynomials $p_{0,1}, \ldots, p_{0,\mathsf{s}(0)}$.

In the online phase, given an instance x and witness w such that $(\mathsf{i}, \mathsf{x}, \mathsf{w}) \in \mathscr{R}$, the prover P receives $(\mathbb{F}, \mathsf{i}, \mathsf{x}, \mathsf{w})$ and the verifier V receives $(\mathbb{F}, \mathsf{x})$ and oracle access to the polynomials output by $\mathsf{I}(\mathbb{F}, \mathsf{i})$. The prover P and the verifier V interact over $\mathsf{k} = \mathsf{k}(|\mathsf{i}|)$ rounds.

For $i \in [\mathsf{k}]$, in the i-th round of interaction, the verifier V sends a message $\mu_i \in \mathbb{F}^*$ to the prover P; then the prover P replies with $\mathsf{s}(i)$ oracle polynomials $p_{i,1}, \ldots, p_{i,\mathsf{s}(i)}$. The verifier may query any of the polynomials it has received any number of times. A query consists of a location $z \in \mathbb{F}$ for an oracle $p_{i,j}$, and its corresponding answer is $p_{i,j}(z) \in \mathbb{F}$. After the interaction, the verifier accepts or rejects. Every PIOP must satisfy the following properties.

Completeness. For every field $\mathbb{F} \in \mathcal{F}$ and index-instance-witness tuple $(\mathsf{i}, \mathsf{x}, \mathsf{w}) \in \mathscr{R}$, the probability that $\mathsf{P}(\mathbb{F}, \mathsf{i}, \mathsf{x}, \mathsf{w})$ convinces $\mathsf{V}^{\mathsf{I}(\mathbb{F},\mathsf{i})}(\mathbb{F}, \mathsf{x})$ to accept in the interactive oracle protocol is 1.

Knowledge soundness. We say that PIOP has knowledge error ϵ if there exists a probabilistic polynomial-time extractor E such that for every field $\mathbb{F} \in \mathcal{F}$, index i, instance x, and malicious prover $\tilde{\mathsf{P}}$,

$$\Pr\left[\begin{array}{c} (\mathsf{i}, \mathsf{x}, \mathsf{w}) \notin \mathscr{R} \\ \wedge \\ \langle \tilde{\mathsf{P}}, \mathsf{V}^{\mathsf{I}(\mathbb{F},\mathsf{i})}(\mathbb{F}, \mathsf{x}) \rangle = 1 \end{array} \middle| \; \mathsf{w} \leftarrow \mathsf{E}^{\tilde{\mathsf{P}}}(\mathbb{F}, \mathsf{i}, \mathsf{x}) \right] = \epsilon$$

The *query complexity* q is the total number of queries made by the verifier to the indexer and prover polynomials.

Additional notation and assumptions. In this paper, we only consider public-coin PIOPs where the verifier makes non-adaptive queries. We consider n-variate polynomials over a finite field $\mathbb{F}$ and assume that $N = 2^n$, and that N is much smaller than $|\mathbb{F}|$. RW streaming provers have access to an input polynomial p as a RW stream of evaluations on $\{0,1\}^n$ in a lexicographic order $[p(\mathsf{bin}(0)), p(\mathsf{bin}(1)), \ldots, p(\mathsf{bin}(N-1))]$, and we denote this stream as $\mathbf{R}(p)$. We omit the verifier's code for brevity, since they are unmodified from the non-streaming versions.

5.2 RW streaming prover for sumcheck

Extending the techniques discussed for multilinear polynomials in Section 2.5, we build a RW streaming sumcheck prover for a class of multivariate polynomials that we call *sumprod* polynomials[17] in Section 5.2.1. In Section 5.2.2, we then present the Lagrange sumcheck: a special case of the sumprod sumcheck where one of the constituent polynomials is the Lagrange polynomial. Then, in Section 5.2.3, we build a RW streaming prover for *batch* sumcheck, which allows proving sumcheck claims about multiple sumprod polynomials simultaneously. These are key building blocks for the RW streaming algorithms that follow in Section 5.3.

5.2.1 Sumcheck for sumprod polynomials

We now describe our read-write streaming prover for the sumcheck relation for *sumprod* polynomials, which are multivariate polynomials that can be expressed as sums of products of multilinear polynomials. We define this class next, and show how to generalize the sumcheck prover for multilinear polynomials (Section 2.5.1) to this class.

Definition 5.1. *The relation $\mathscr{R}_{\mathsf{SSC}}$ contains tuples of the form*

$$(\mathsf{i}_{\mathsf{SSC}}, \mathsf{x}_{\mathsf{SSC}}, \mathsf{w}_{\mathsf{SSC}}) = ((\mathbb{F}, n, d, h), ([\![p_1]\!], \ldots, [\![p_d]\!], \sigma), (p_1, \ldots, p_d))$$

where $\sigma \in \mathbb{F}$ is the target sum, each p_i is an n-variate multilinear polynomial, and h is a multilinear polynomial with ℓ monomials such that $\sum_{\boldsymbol{x} \in \{0,1\}^n} h(p_1(\boldsymbol{x}), \ldots, p_d(\boldsymbol{x})) = \sigma$.

We obtain a RW streaming prover for this relation by modifying the algorithm in Section 2.5.1 as follows. We first consider a single product of d multilinear polynomials. Recall that in the multilinear case, the round polynomials a_i sent by P in each round are linear, and therefore 2 evaluations are sufficient to specify them. Therefore, P computes 2 evaluations of the a_i's and sends them to V. For a product of d multilinear polynomials, each polynomial a_i is a univariate polynomial of degree d, and P specifies this via $d+1$ evaluations, which it sends to V.

To generalize to the sumprod polynomial $h(p_1, \ldots, p_d)$, P still computes $d+1$ evaluations of each round polynomial, but now it does so by iterating through all monomials in h, and computing and summing evaluations for each of these. That is, for each monomial, P computes $d+1$ evaluations of the respective round polynomial (regardless of the degree of the monomial), and adds them to a running sum. At the end of the round, P is left with $d+1$ running sums corresponding to $d+1$ round polynomial evaluations, which it sends to V.

We prove the following lemma which helps us show that the prover sends the correct values to the verifier in Step 8 for any iteration of the loop in Step 2.

[17] Recall that a (d, ℓ)-sumprod polynomial is a polynomial of the form $h(p_1(\boldsymbol{x}), \ldots, p_d(\boldsymbol{x})) = \sigma$, where h is a multilinear polynomial with ℓ monomials, and $p_1, \ldots, p_d$ are multilinear polynomials.

Lemma 5.2. *Let q be an input polynomial of Algorithm 1 such that $\mathbf{S}$ contains its evaluations over $\{0,1\}^n$. Also, let q_i be the i-variate polynomial such that $\mathbf{S}$ contains its evaluations over $\{0,1\}^i$ at the beginning of iteration i of Step 2. Then $q_i(\mathsf{bin}(j)) = q(\mathsf{bin}(j), r_{i+1}, \ldots, r_n)$ for all $j \in \{0, \ldots, N_i - 1\}$.*

Proof. We proceed by induction and consider the first iteration of Step 2 when $i = n$. At the beginning of the iteration, $q_n(\mathsf{bin}(j)) = q(\mathsf{bin}(j))$ for all $j \in \{0, \ldots, N-1\}$ as desired.

Now consider $i < n$. By the induction hypothesis, we have $q_{i+1}(\mathsf{bin}(j)) = q(\mathsf{bin}(j), r_{i+2}, \ldots, r_n)$ for all $j \in \{0, \ldots, N_{i+1} - 1\}$ at the beginning of iteration $i+1$. At the end of the iteration $i+1$ (and before the start of iteration i), Step 11 creates a stream of $N_i/2$ elements such that

$$\begin{aligned} q_i(\mathsf{bin}(j)) &= (1 - r_{i+1}) \cdot q_{i+1}(\mathsf{bin}(2j), r_{i+2}, \ldots, r_n) + r_{i+1} \cdot q_{i+1}(\mathsf{bin}(2j+1), r_{i+2}, \ldots, r_n) \\ &= (1 - r_{i+1}) \cdot q(\mathsf{bin}(2j), r_{i+2}, \ldots, r_n) + r_{i+1} \cdot q(\mathsf{bin}(2j+1), r_{i+2}, \ldots, r_n) \\ &= (1 - r_{i+1}) \cdot q(\mathsf{bin}(j), 0, r_{i+2}, \ldots, r_n) + r_{i+1} \cdot q(\mathsf{bin}(j), 1, r_{i+2}, \ldots, r_n) \\ &= q(\mathsf{bin}(j), r_{i+1}, \ldots, r_n) \end{aligned}$$

for all $j \in \{0, \ldots, N_{i-1} - 1\}$ as desired. □

Lemma 5.3. *PIOP 1 and Algorithm 1 together comprise a read-write streaming PIOP for $\mathscr{R}_{\mathsf{SSC}}$ for n-variate (d, ℓ)-sumprod polynomials with the following efficiency properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(d\ell N)$	$O(d + \ell)$	$O(dN)$	$O(d)$	$O(d \log N/\lvert\mathbb{F}\rvert)$	$O(d \log N)$	$O(d)$

Proof. We will prove the statement for the product of 2 polynomials p and q that received in two input streams $\mathbf{R}_p$ and $\mathbf{R}_q$. The proof extends to (d, ℓ)-sumprod polynomials straightforwardly. Completeness and soundness proofs follow from [Tha22] if we can show that the messages sent by P in Step 8 are correct. That is, we need to show that $S[\alpha] = a_i(\alpha)$ for any $\alpha \in \mathbb{F}$.

If p_i is the i-variate polynomial that $\mathbf{R}_p$ contains at the beginning of iteration i of Step 2 in Algorithm 1, then $p_i(\mathsf{bin}(j)) = p(\mathsf{bin}(j), r_{i+1}, \ldots, r_n)$ for all $j \in \{0, \ldots, N_i - 1\}$ due to Lemma 5.2 (similarly $q_i(\mathsf{bin}(j)) = q(\mathsf{bin}(j), r_{i+1}, \ldots, r_n)$). We have

$$\begin{aligned} a_i(\alpha) &= \sum_{\boldsymbol{b} \in \{0,1\}^{i-1}} p(\boldsymbol{b}, \alpha, r_{i+1}, \ldots, r_n) \cdot q(\boldsymbol{b}, \alpha, r_{i+1}, \ldots, r_n) \\ &= \sum_{j=0}^{N_i/2-1} ((1-\alpha)p_i(\mathsf{bin}(2j)) + \alpha p_i(\mathsf{bin}(2j+1))) \cdot ((1-\alpha)q_i(\mathsf{bin}(2j)) + \alpha q_i(\mathsf{bin}(2j+1))) = S[\alpha] \end{aligned}$$

as desired.

P requires $O(d\ell N_i)$ time in the ith round of the protocol, and therefore requires $O(d\ell N)$ time in total. Moreover, P only requires $O(d)$ random-access space to store the round polynomial evaluations, and $O(\ell)$ random-access space to store h. Finally, P requires $O(dN)$ streaming space across d distinct streams to store the folded multilinear polynomials. V makes one query for each multilinear polynomial and therefore makes d queries in total, and the communication is $O(d \log N)$ since P sends $d+1$ evaluations of $\log N$ round polynomials. □

5.2.2 Lagrange sumcheck for sumprod polynomials

Definition 5.4. *The relation $\mathscr{R}_{\mathsf{LSC}}$ contains tuples of the form*

$$(\mathsf{i}_{\mathsf{LSC}}, \mathsf{x}_{\mathsf{LSC}}, \mathsf{w}_{\mathsf{LSC}}) = ((\mathbb{F}, n, d, h), ([\![p_1]\!], \ldots, [\![p_d]\!], \boldsymbol{t}, \sigma), (p_1, \ldots, p_d))$$

where $\sigma \in \mathbb{F}$ is the target sum, each p_i is an n-variate multilinear polynomial, $\boldsymbol{t} \in \mathbb{F}^n$, and h is a multilinear polynomial with ℓ monomials such that $\sum_{\boldsymbol{x} \in \{0,1\}^n} h(p_1(\boldsymbol{x}), \ldots, p_d(\boldsymbol{x})) \cdot \mathrm{eq}(\boldsymbol{t}, \boldsymbol{x}) = \sigma$.

Since this relation is a special case of the sumprod sumcheck relation Definition 5.1, we use the same PIOP as in PIOP 1, and describe an efficient RW streaming prover specifically for this relation. We define a helper method RWSumEq, which is similar to the RWSum method defined in Section 4.2, but additionally takes as input a vector $\boldsymbol{t} \in \mathbb{F}^n$, which describes the Lagrange polynomial.

Read-write Streaming Algorithm 2: Lagrange sumcheck for sumprod polynomials

$\mathsf{P}(\mathsf{i}_{\mathsf{LSC}}, \mathsf{x}_{\mathsf{LSC}}, (\mathbf{R}_1(p_1), \ldots, \mathbf{R}_d(p_d)))$:

1. Initialize folding coefficient for $\mathrm{eq}(\boldsymbol{t}, \boldsymbol{X})$: $\gamma \leftarrow 1$.
2. For each i in $[n, \ldots, 1]$:
3. Initialize running sums: $S \leftarrow [0]_{j=0}^{d+1}$.
4. For each monomial $(c \cdot X_{j_1} \cdot X_{j_2} \cdot \ldots \cdot X_{j_k})$ in h:
5. Get $d+2$ evaluations for the monomial's round polynomial: set $E \leftarrow \mathsf{RWSumEq}(d+1, i, \boldsymbol{t}, \mathbf{R}_{j_1}, \ldots, \mathbf{R}_{j_k})$.
6. Add each evaluation to the corresponding running sum: for s in $[0, \ldots, d]$: set $S[s] \leftarrow S[s] + c \cdot E[s]$.
7. Restart the streams that were used for this monomial: $\mathbf{R}_{j_1}.\mathsf{restart}(), \ldots, \mathbf{R}_{j_k}.\mathsf{restart}()$.
8. Send $[a_i(s) := \gamma \cdot S[s]]_{s=0}^{d+1}$ to V.
9. If $i \neq 1$:
10. Receive verifier challenge $r_i \xleftarrow{\$} \mathbb{F}$ from V.
11. Fold each stream individually: $\mathsf{RWFoldInPlace}(1 - r_i, r_i, \mathbf{R}_1, \ldots, \mathbf{R}_d)$.
12. Obtain folded Lagrange polynomial by updating folding coefficient: $\gamma \leftarrow \gamma \cdot (t_i \cdot r_i + (1 - t_i) \cdot (1 - r_i))$.

$\mathsf{RWSumEq}(d, i, \boldsymbol{t}, \mathbf{R}_1, \ldots, \mathbf{R}_k) \mapsto S$:

1. $\mathsf{Eq.Init}(t_1, t_2, \ldots, t_i)$.
2. Set $S \leftarrow [0]_{s=0}^{d}$.
3. For i in $[1, \ldots, \mathbf{R}.\mathsf{len}()/2]$:
4. For j in $[1, \ldots, k]$:
5. $a_{L,j} \leftarrow \mathbf{R}_j.\mathsf{read}(); a_{R,j} \leftarrow \mathbf{R}_i.\mathsf{read}()$.
6. $a_{L,k+1} \leftarrow \mathsf{Eq.Next}; a_{R,k+1} \leftarrow \mathsf{Eq.Next}$.
7. For s in $[0, \ldots, d]$:
8. $S[s] \leftarrow S[s] + \prod_{j=1}^{k+1} ((1 - s) \cdot a_{L,j} + s \cdot a_{R,j})$

Remark 5.5 (Lagrange sumcheck for multilinear polynomials)**.** A practical optimization of Algorithm 2 is to use it for the special case of multilinear polynomials, i.e., when $d = 1$ and $h(X) = X$. In this case, [NTZ25] show that one can construct a read-only streaming sumcheck prover that requires only $O(N)$ time and $O(\sqrt{N})$ space. This algorithm can be adapted to the RW streaming setting by using write-streams to store the intermediate tables of size $O(\sqrt{N})$ to obtain a RW streaming sumcheck prover that requires $O(N)$ time, $O(\sqrt{N})$ streaming space, and $O(\log N)$ random-access space.

Lemma 5.6. *PIOP 1 and Algorithm 2 together comprise a read-write streaming PIOP for $\mathscr{R}_{\mathsf{LSC}}$ for n-variate (d, ℓ)-sumprod polynomials with the following properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(d\ell N)$	$O(d + \ell + \log N)$	$O(dN)$	$O(d)$	$O(d \log N/\lvert\mathbb{F}\rvert)$	$O(d \log N)$	$O(d)$

Proof. All properties follow from Lemma 5.3 except for the random-access space complexity, which increases to $O(d + \ell + \log N)$ due to the additional space required to store $\boldsymbol{t}$. The time complexity does not increase because each call to RWSumEq in iteration i of Step 2 still requires $O(d \cdot 2^i)$ time for each monomial in h, since the Lagrange polynomial evaluations can be streamed in $O(2^i)$ time.

We additionally need to prove that the messages sent by P in each iteration of Step 2 are correct. That is, we need to show that for all $\alpha \in \{0, \ldots, d+1\}$,

$$\gamma_i \cdot S[\alpha] = \sum_{\boldsymbol{x} \in \{0,1\}^{i-1}} h(p_1(\boldsymbol{x}, \alpha, r_{i+1}, \ldots, r_n), \ldots, p_d(\boldsymbol{x}, \alpha, r_{i+1}, \ldots, r_n)) \cdot \mathrm{eq}((\boldsymbol{x}, \alpha, r_{i+1}, \ldots, r_n), \boldsymbol{t}) \ ,$$

where γ_i is the folding coefficient at the beginning of the iteration i of Step 2. We prove the above statement for a single multilinear polynomial q for brevity, the proof extends to (d, ℓ)-sumprod polynomials straightforwardly, using the same arguments as in Lemma 5.3. Let q_i be the polynomial stored in the input stream $\mathbf{R}$ in iteration i of Step 2. We have

$$\begin{aligned}
&\sum_{\boldsymbol{x} \in \{0,1\}^{i-1}} q(\boldsymbol{x}, \alpha, r_{i+1}, \ldots, r_n) \cdot \mathrm{eq}(\boldsymbol{t}, (\boldsymbol{x}, \alpha, r_{i+1}, \ldots, r_n)) \\
&= \sum_{\boldsymbol{x} \in \{0,1\}^{i-1}} q_i(\boldsymbol{x}, \alpha) \cdot \mathrm{eq}(\boldsymbol{t}, (\boldsymbol{x}, \alpha, r_{i+1}, \ldots, r_n)) && \text{[Lemma 5.2]} \\
&= \prod_{j=i+1}^{n} ((1-r_j)(1-t_j) + r_j t_j) \cdot \sum_{\boldsymbol{x} \in \{0,1\}^{i-1}} q_i(\boldsymbol{x}, \alpha) \cdot \mathrm{eq}((t_1, \ldots, t_i), (\boldsymbol{x}, \alpha)) \\
&= \gamma_i \cdot \sum_{\boldsymbol{x} \in \{0,1\}^{i-1}} q_i(\boldsymbol{x}, \alpha) \cdot \mathrm{eq}((t_1, \ldots, t_i), (\boldsymbol{x}, \alpha)) = \gamma_i \cdot S[\alpha]
\end{aligned}$$

as desired. □

5.2.3 Batch sumcheck for sumprod polynomials

We finally present the batch RW streaming sumcheck for multiple (d, ℓ)-sumprod polynomials. Let $p_1, \ldots, p_\nu$ be n-variate (d, ℓ)-sumprod polynomials, and let $\sigma_1, \ldots, \sigma_\nu$ be their claimed sums on $\{0,1\}^n$ respectively. We can check all these claims simultaneously via the standard random linear combination technique.

In more detail, P obtains a random challenge α from V, and runs a RW streaming sumcheck PIOP for the (d, ℓ)-sumprod polynomial $p(\boldsymbol{X}) = \sum_{i=1}^{\nu} \alpha^{i-1} p_i(\boldsymbol{X})$, and target sum $\sigma = \sum_{i=1}^{\nu} \alpha^{i-1} \sigma_i$. This transformation incurs a negligible soundness error.

We now define the batch sumcheck relation. Let $p_{(i,j)}$ be the jth constituent multilinear polynomial of p_i. That is, $p_i = h_i(p_{(i,1)}, \ldots, p_{(i,d)})$. V has oracle access to each $p_{(i,j)}$ individually, and P has streaming access to all $d\nu$ of them individually.

$$\begin{aligned}
\mathscr{R}_{\mathsf{BSC}} &= (\mathsf{i}_{\mathsf{BSC}}, \mathsf{x}_{\mathsf{BSC}}, \mathsf{w}_{\mathsf{BSC}}) \\
&= \left((\mathbb{F}, n, d, \nu, h_1, \ldots, h_\nu) \, , \, (\llbracket p_{(1,1)} \rrbracket, \ldots, \llbracket p_{(\nu,d)} \rrbracket, \sigma_1, \ldots, \sigma_\nu) \, , \, (p_{(1,1)}, \ldots, p_{(\nu,d)}) \right)
\end{aligned}$$

We present a RW streaming prover for $\mathscr{R}_{\mathsf{BSC}}$ whose efficiency is determined by the efficiency of the underlying sumcheck protocol for sumprod polynomials.

Read-write Streaming Algorithm 3: BATCH SUMCHECK for multiple (d, ℓ)-sumprod polynomials

$\mathsf{P}(\mathsf{i}_{\mathsf{BSC}}, \mathsf{x}_{\mathsf{BSC}}, (\mathbf{R}_{(1,1)}(p_{(1,1)}), \ldots, \mathbf{R}_{(\nu,d)}(p_{(\nu,d)})))$:

1. P receives $\alpha \xleftarrow{\$} \mathbb{F}$ from V.
2. Initialize batch target sum $\sigma \leftarrow \sum_{i=1}^{\nu} \alpha^{i-1}\sigma_i$.
3. Define batch polynomial $h(X_{1,1}, \ldots, X_{1,d}, \ldots, X_{\nu,1}, \ldots, X_{\nu,d}) := \sum_{i=1}^{\nu} \alpha^{i-1} h_i(X_{i,1}, \ldots, X_{i,d})$
4. Define $\mathsf{i} := (\mathbb{F}, n, d\nu, h)$ and $\mathsf{x} := ([\![p_{(1,1)}]\!], \ldots, [\![p_{(\nu,d)}]\!], \sigma)$ and $\mathsf{w} := (\mathbf{R}_{(1,1)}, \ldots, \mathbf{R}_{(\nu,d)})$
5. P and V invoke the RW streaming sumcheck PIOP for a $(d\nu, \ell\nu)$-sumprod polynomial for $(\mathsf{i}, \mathsf{x}, \mathsf{w})$ defined above.

5.3 Read-write streaming prover for HyperPlonk's PIOP

We present RW streaming PIOPs for several relations including zerocheck (Section 5.3.1), prodcheck (Section 5.3.2), multiset-equality-check (Section 5.3.3), and permcheck (Section 5.3.4).

Then, in Section 5.3.5 we show how to use these PIOPs to create a RW streaming PIOP for the HyperPlonk relation [CBBZ23], which in turn directly gives us a RW streaming PIOP for circuit satisfiability as described in Section 2.2. For each PIOP, we first recall the standard non-streaming implementation of the prover, and then present a RW streaming version for the same.

5.3.1 Zerocheck PIOP

Given an n-variate polynomial p, the zerocheck PIOP checks if p is zero at all points of an n-dimensional boolean hypercube $\{0,1\}^n$. This is formalized via the following relation for (d, ℓ)- sumprod polynomials:

Definition 5.7. *The zerocheck relation $\mathscr{R}_{\mathsf{ZC}}$ for (d, ℓ)-sumprod polynomials is an indexed relation consisting of the following tuples:*

$$(\mathsf{i}_{\mathsf{ZC}}, \mathsf{x}_{\mathsf{ZC}}, \mathsf{w}_{\mathsf{ZC}}) = ((\mathbb{F}, n, d, h), ([\![p_1]\!], \ldots, [\![p_d]\!]), (p_1, \ldots, p_d))$$

where each p_i is an n-variate multilinear polynomial, and h is a multilinear polynomial with ℓ terms such that $h(p_1(\boldsymbol{x}), \ldots, p_d(\boldsymbol{x})) = 0$ for all $\boldsymbol{x} \in \{0,1\}^n$.

The PIOP below illustrates a standard way of proving this relation.

PIOP 3: ZEROCHECK

$\langle \mathsf{P}(\mathsf{i}_{\mathsf{ZC}}, \mathsf{x}_{\mathsf{ZC}}, \mathsf{w}_{\mathsf{ZC}}), \mathsf{V}(\mathsf{i}_{\mathsf{ZC}}, \mathsf{x}_{\mathsf{ZC}}) \rangle$:

1. P receives $\boldsymbol{r} \xleftarrow{\$} \mathbb{F}^n$ from V.
2. P computes the multilinear polynomial $q(\boldsymbol{X}) := \mathrm{eq}(\boldsymbol{X}, \boldsymbol{r})$.
3. P and V invoke the sumcheck PIOP for the claim "$\sum_{\boldsymbol{x} \in \{0,1\}^n} h(p_1(\boldsymbol{x}), \ldots, p_d(\boldsymbol{x})) \cdot q(\boldsymbol{x}) = 0$".

Prior work (for example, [CBBZ23]) provides completeness and soundness proof for this PIOP. We show how to construct a streaming prover for (d, ℓ)-sumprod polynomials using Algorithm 1. Let h be the multilinear polynomial such that $p(\boldsymbol{X}) = h(p_1(\boldsymbol{X}), \ldots, p_d(\boldsymbol{X}))$. P and V have streaming and oracle access to p_i respectively.

Read-write Streaming Algorithm 4: ZEROCHECK for (d, ℓ)-sumprod polynomials

$\mathsf{P}(\mathsf{i}_{\mathsf{ZC}}, ([\![p_1]\!], \ldots, [\![p_d]\!]), \mathsf{w}_{\mathsf{ZC}})$:

1. Receive $\boldsymbol{r} \xleftarrow{\$} \mathbb{F}^n$ from V.
2. Define Lagrange sumcheck instance: $\mathsf{x} := ([\![p_1]\!], \ldots, [\![p_d]\!], [\![q]\!], \boldsymbol{r}, 0)$
3. Invoke the RW streaming Lagrange sumcheck PIOP for sumprod polynomials for $(\mathsf{i}_{\mathsf{ZC}}, \mathsf{x}, \mathsf{w}_{\mathsf{ZC}})$.

Lemma 5.8. *PIOP 3 and Algorithm 4 together comprise a read-write streaming PIOP for $\mathscr{R}_{\mathsf{ZC}}$ for n-variate (d, ℓ)-sumprod polynomials with the following properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(d\ell N)$	$O(d + \ell + \log N)$	$O(dN)$	$O(d)$	$O(d \log N/\lvert\mathbb{F}\rvert)$	$O(d \log N)$	$O(d)$

Proof. The PIOP properties follow from the analysis of Chen et al. [CBBZ23] because the prover in Algorithm 4 sends the same values to V as in PIOP 3, which follows from Lemma 5.3.

An additional $\log N$ random-access space is required to store $\boldsymbol{r}$ while computing $\mathrm{eq}(\boldsymbol{X}, \boldsymbol{r})$. The total prover time required to compute $\mathrm{eq}(\boldsymbol{X}, \boldsymbol{r})$ is $O(N)$ due to the amortized efficiency of $\mathsf{Eq.Next}$ as discussed in Section 2.6.1. Other efficiency parameters follow from Lemma 5.3. □

Remark 5.9. Algorithm 4 can be viewed as an interactive reduction from zerocheck to sumcheck. This protocol will be used alongside other PIOPs that will also reduce to sumcheck, therefore it will be helpful to perform a single batch sumcheck for all the sumcheck claims. To do so, we extract a method ZToS that performs Step 1 to Step 2 of Algorithm 4, and emits the resulting sumcheck claim.

5.3.2 Prodcheck PIOP

Given n-variate polynomials p and q and a target product σ, the prodcheck PIOP tests if the product of $p(\boldsymbol{x})/q(\boldsymbol{x})$ is σ over all $\boldsymbol{x} \in \{0,1\}^n$. This is formalized via the following relation:

Definition 5.10 (Prodcheck relation)**.** *The prodcheck relation $\mathscr{R}_{\mathsf{PDC}}$ is an indexed relation consisting of tuples $(\mathsf{i}_{\mathsf{PDC}}, \mathsf{x}_{\mathsf{PDC}}, \mathsf{w}_{\mathsf{PDC}}) = ((\mathbb{F}, n), (\llbracket p \rrbracket, \llbracket q \rrbracket, \sigma), (p, q))$, where p, q are n-variate multilinear polynomials such that $\prod_{\boldsymbol{x} \in \{0,1\}^n} p(\boldsymbol{x})/q(\boldsymbol{x}) = \sigma$.*

The following PIOP (due to Setty and Lee [SL20]) illustrates a standard way of proving this relation.

PIOP 4: Prodcheck

$\langle \mathsf{P}(\mathsf{i}_{\mathsf{PDC}}, \mathsf{x}_{\mathsf{PDC}}, \mathsf{w}_{\mathsf{PDC}}), \mathsf{V}(\mathsf{i}_{\mathsf{PDC}}, \mathsf{x}_{\mathsf{PDC}}) \rangle$:

1. P sends $(n+1)$-variate polynomial ν such that $\nu(0, \boldsymbol{X}) := p(\boldsymbol{X})/q(\boldsymbol{X})$ and $\nu(1, \boldsymbol{X}) := \nu(\boldsymbol{X}, 0) \cdot \nu(\boldsymbol{X}, 1)$.
2. Define $f(\boldsymbol{X}) := \nu(0, \boldsymbol{X}) \cdot q(\boldsymbol{X}) - p(\boldsymbol{X})$ and $g(\boldsymbol{X}) := \nu(1, \boldsymbol{X}) - \nu(\boldsymbol{X}, 0) \cdot \nu(\boldsymbol{X}, 1)$.
3. P and V invoke the zerocheck PIOP for $f(\boldsymbol{X})$ and $g(\boldsymbol{X})$.
4. V checks that $\nu(1, 1, \ldots, 1, 0) = \sigma$.

Prior work [SL20; CBBZ23] provides completeness and soundness proof for this PIOP. We now describe our read-write streaming prover for this PIOP. As discussed in Section 2.6.2, the computation of the polynomial ν induces a binary-tree structure. We define the $(n-i)$-variate polynomial $\nu_i(\boldsymbol{X}) = \nu(1, \ldots, 1, 0, \boldsymbol{X})$ where the first i variables of ν are fixed to 1, and the $(i+1)$-th variable is fixed to 0. P computes ν_i for $i = 0, \ldots, n$ in that order, since an evaluation of ν_{i+1} is the product of two evaluations of ν_i. Finally, it concatenates the $n+1$ streams to get one stream for ν.

Read-write Streaming Algorithm 5: PRODCHECK

$\mathsf{P}(\mathsf{i}_{\mathsf{PDC}}, \mathsf{x}_{\mathsf{PDC}}, (\mathbf{R}_p(p), \mathbf{R}_q(q)))$:

Initialize RW streams for ν_j as defined above:

1. Allocate space for each ν_j: for j in $[0, \ldots, n]$: $\mathbf{S}_{\nu,j}.\mathsf{init}(N/2^j)$.
2. Compute stream for polynomial $\nu_0(\boldsymbol{X}) := \nu(0, \boldsymbol{X})$: $\mathbf{S}_{\nu,0} \leftarrow \mathsf{Map}(/) \rightsquigarrow \mathsf{Zip} \rightsquigarrow (\mathbf{R}_p, \mathbf{R}_q)$.

Compute each ν_i using the binary tree structure:

3. For j in $[1, \ldots, n]$:
4. $\quad \mathbf{S}_{\nu,j-1}.\mathsf{swapmode}()$.
5. $\quad \mathbf{S}_{\nu,j} \leftarrow \mathsf{Map}(\times) \rightsquigarrow \mathsf{Zip} \rightsquigarrow \mathsf{SplitEO} \rightsquigarrow \mathbf{S}_{\nu,j-1}$.
6. $\mathbf{S}_\nu \leftarrow \mathsf{Concat}(\mathbf{S}_{\nu,0}, \mathbf{S}_{\nu,1}, \ldots, \mathbf{S}_{\nu,n}, [0])$ (append 0 at the end to make the stream length a power of 2).
7. Create $[\![\nu]\!]$ from $\mathbf{S}_\nu$ and send to V.

Split stream of ν into halves for the zerocheck instances:

8. Define $\nu_L(\boldsymbol{X}) := \nu(0, \boldsymbol{X})$ and $\nu_R(\boldsymbol{X}) := \nu(1, \boldsymbol{X})$ and $\nu_E(\boldsymbol{X}) := \nu(\boldsymbol{X}, 0)$ and $\nu_O(\boldsymbol{X}) := \nu(\boldsymbol{X}, 1)$.
9. Split ν to obtain streams for $\nu(\boldsymbol{X}, 0)$ and $\nu(\boldsymbol{X}, 1)$: $(\mathbf{R}_{\nu,E}, \mathbf{R}_{\nu,O}) \leftarrow \mathsf{SplitEO}(\mathbf{S}_\nu)$.
10. Split ν to obtain streams for $\nu(0, \boldsymbol{X})$ and $\nu(1, \boldsymbol{X})$: $(\mathbf{R}_{\nu,L}, \mathbf{R}_{\nu,R}) \leftarrow \mathsf{SplitLR}(\mathbf{S}_\nu)$.

Initialize sumcheck instances using ZToS:

11. Define $t_0(x_1, x_2, x_3) := x_1 \cdot x_2 - x_3$.
12. Obtain sumcheck claim for the zerocheck claim "$\nu_L(\boldsymbol{X}) \cdot q(\boldsymbol{X}) - p(\boldsymbol{X}) = 0$":
    $(\mathsf{i}_f = (\_, \_, \_, t_1), \mathsf{x}_f, \mathsf{w}_f) \leftarrow \mathsf{ZToS}((\mathbb{F}, n, 2, t_0), ([\![\nu_L]\!], [\![q]\!], [\![p]\!]), (\mathbf{R}_{\nu,L}, \mathbf{R}_q, \mathbf{R}_p))$.
13. Obtain sumcheck claim for the zerocheck claim "$\nu_E(\boldsymbol{X}) \cdot \nu_O(\boldsymbol{X}) - \nu_R(\boldsymbol{X}) = 0$":
    $(\mathsf{i}_g = (\_, \_, \_, t_2), \mathsf{x}_g, \mathsf{w}_g) \leftarrow \mathsf{ZToS}((\mathbb{F}, n, 2, t_0), ([\![\nu_E]\!], [\![\nu_O]\!], [\![\nu_R]\!]), (\mathbf{R}_{\nu,E}, \mathbf{R}_{\nu,O}, \mathbf{R}_{\nu,R}))$.
14. Define $\mathsf{i} := (\mathbb{F}, n, 3, 2, t_1, t_2)$ and $\mathsf{x} := (\mathsf{x}_f, \mathsf{x}_g)$ and $\mathsf{w} := (\mathsf{w}_f, \mathsf{w}_g)$.
15. Invoke the RW streaming batch sumcheck PIOP for sumprod polynomials for $(\mathsf{i}, \mathsf{x}, \mathsf{w})$ defined above, which batches the two sumcheck claims into one.

Remark 5.11. In the above algorithm V implicitly has polynomial oracle access to $\nu_L, \nu_R, \nu_E, \nu_O$ due to having polynomial oracle access to ν.

Lemma 5.12. *PIOP 4 and Algorithm 5 together comprise a read-write streaming PIOP for $\mathscr{R}_{\mathsf{PDC}}$ with the following properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(N)$	$O(\log N)$	$O(N)$	$O(1)$	$O(N/\lvert\mathbb{F}\rvert)$	$O(\log N)$	$O(1)$

Proof. The PIOP properties follow from the analysis of Chen et al. [CBBZ23] because P in Algorithm 5 produces the same values as PIOP 4. The time and space efficiency parameters follow from Lemma 5.8 where $d = O(1)$. □

5.3.3 Multiset-equality-check PIOP

We use the read-write streaming PIOP for prodcheck to build a read-write streaming PIOP for proving claims about equality of multisets encoded as multilinear polynomials.

Definition 5.13 (multiset-equality)**.** *The indexed relation $\mathscr{R}_{\mathsf{MEC}}$ consists of tuples $(\mathsf{i}_{\mathsf{MEC}}, \mathsf{x}_{\mathsf{MEC}}, \mathsf{w}_{\mathsf{MEC}}) = ((\mathbb{F}, n), ([\![p]\!], [\![q]\!]), (p, q))$ where p and q are n-variate multilinear polynomials such that the multisets $\{\!\{p(\boldsymbol{x})\}\!\}_{\boldsymbol{x} \in \{0,1\}^n}$ and $\{\!\{q(\boldsymbol{x})\}\!\}_{\boldsymbol{x} \in \{0,1\}^n}$ are equal.*

The PIOP below illustrates a way of proving this as described by Chen et al. [CBBZ23]:

PIOP 5: MULTISET-EQUALITY-CHECK

$\langle \mathsf{P}(\mathsf{i}_{\mathsf{MEC}}, \mathsf{x}_{\mathsf{MEC}}, \mathsf{w}_{\mathsf{MEC}}), \mathsf{V}(\mathsf{i}_{\mathsf{MEC}}, \mathsf{x}_{\mathsf{MEC}}) \rangle$:

1. P receives a random challenge $\alpha \in \mathbb{F}$ from V.
2. P computes polynomials $p'(\boldsymbol{X}) := \alpha + p(\boldsymbol{X})$ and $q'(\boldsymbol{X}) := \alpha + q(\boldsymbol{X})$.
3. P and V invoke the prodcheck PIOP for the claim "$\prod_{\boldsymbol{x} \in \{0,1\}^n} p'(\boldsymbol{x})/q'(\boldsymbol{x}) = 1$".

Below we demonstrate a streaming prover for this that avoids intermediate read-write streams.

Read-write Streaming Algorithm 6: MULTISET-EQUALITY-CHECK

$\mathsf{P}(\mathsf{i}_{\mathsf{MEC}}, \mathsf{x}_{\mathsf{MEC}}, (\mathbf{R}_p(p), \mathbf{R}_q(q)))$:

1. Receive a random challenge $\alpha \in \mathbb{F}$ from V.

   Initialize read-only streams for $p'(\boldsymbol{X}) := \alpha + p(\boldsymbol{X})$ and $q'(\boldsymbol{X}) := \alpha + q(\boldsymbol{X})$:
2. $\mathbf{S}_{p'} \leftsquigarrow \mathsf{Map}(x \mapsto \alpha + x) \leftsquigarrow \mathbf{R}_p$.
3. $\mathbf{S}_{q'} \leftsquigarrow \mathsf{Map}(x \mapsto \alpha + x) \leftsquigarrow \mathbf{R}_q$.

   Reduce to prodcheck:
4. Define $\mathsf{i} := (\mathbb{F}, n); \mathsf{x} := (\llbracket p' \rrbracket, \llbracket q' \rrbracket, 1); \mathsf{w} := (\mathbf{S}_{p'}, \mathbf{S}_{q'})$
5. Invoke the RW streaming prodcheck PIOP for $(\mathsf{i}, \mathsf{x}, \mathsf{w})$.

Lemma 5.14. *PIOP 5 and Algorithm 6 together comprise a read-write streaming PIOP for $\mathscr{R}_{\mathsf{MEC}}$ with the following properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(N)$	$O(\log N)$	$O(N)$	$O(1)$	$O(N/\lvert\mathbb{F}\rvert)$	$O(\log N)$	$O(1)$

Proof. PIOP properties follow from the analysis of Chen et al. [CBBZ23] because P in Algorithm 6 sends the same values to V as in PIOP 5. Moreover, Algorithm 6 has the same time and space complexity as the RW streaming prodcheck PIOP. □

5.3.4 Permcheck PIOP

We use the foregoing read-write streaming PIOP for multiset-equality to design a read-write streaming PIOP for proving that two lists are permutations.

Definition 5.15 (permcheck)**.** *The index relation $\mathscr{R}_{\mathsf{PC}}$ consists of tuples $(\mathsf{i}_{\mathsf{PC}}, \mathsf{x}_{\mathsf{PC}}, \mathsf{w}_{\mathsf{PC}}) = ((\mathbb{F}, n, \pi), (\llbracket p \rrbracket, \llbracket q \rrbracket, \llbracket \hat{\pi} \rrbracket), (p, q, \hat{\pi}))$ where p, q are multilinear polynomials and $\hat{\pi}$ is the multilinear extension of a permutation $\pi : [N] \to [N]$ such that such that $p(\boldsymbol{x}) = q(\mathsf{bin}(\pi(\mathsf{int}(\boldsymbol{x}))))$ for all $\boldsymbol{x} \in \{0,1\}^n$.*

The PIOP below illustrates a way of proving this as described by Chen et al. [CBBZ23]:

PIOP 6: PERMCHECK

$\langle \mathsf{P}(\mathsf{i}_{\mathsf{PC}}, \mathsf{x}_{\mathsf{PC}}, \mathsf{w}_{\mathsf{PC}}), \mathsf{V}(\mathsf{i}_{\mathsf{PC}}, \mathsf{x}_{\mathsf{PC}}) \rangle$:

1. P receives a random challenge $\beta \in \mathbb{F}$ from V.
2. P computes polynomials $p'(\boldsymbol{X}) := p(\boldsymbol{X}) + \hat{\pi}(\boldsymbol{X}) \cdot \beta$ and $q'(\boldsymbol{X}) := q(\boldsymbol{X}) + \mathsf{int}(\boldsymbol{X}) \cdot \beta$.
3. P and V invoke the multiset-equality PIOP for the claim "$\{\!\{p'(\boldsymbol{x}) : \boldsymbol{x} \in \{0,1\}^n\}\!\} = \{\!\{q'(\boldsymbol{x}) : \boldsymbol{x} \in \{0,1\}^n\}\!\}$".

Below we demonstrate a streaming prover for this PIOP that avoids intermediate read-write streams.

Read-write Streaming Algorithm 7: PERMCHECK

$\mathsf{P}(\mathsf{i}_{\mathsf{PC}}, \mathsf{x}_{\mathsf{PC}}, (\mathbf{R}_p(p), \mathbf{R}_q(q), \mathbf{R}_{\hat{\pi}}(\hat{\pi}))$:

1. Receive a random challenge $\beta \in \mathbb{F}$ from V.

   Initialize read-only streams for $p'(\boldsymbol{X}) := p(\boldsymbol{X}) + \hat{\pi}(\boldsymbol{X}) \cdot \beta$ and $q'(\boldsymbol{X}) := q(\boldsymbol{X}) + \mathsf{int}(\boldsymbol{X}) \cdot \beta$:
2. $\mathbf{S}_{p'} \leftsquigarrow \mathsf{Map}((x, y) \mapsto x + \beta \cdot y) \leftsquigarrow \mathsf{Zip} \leftsquigarrow (\mathbf{R}_p, \mathbf{R}_{\hat{\pi}})$.
3. $\mathbf{S}_{q'} \leftsquigarrow \mathsf{Map}((x, y) \mapsto x + \beta \cdot y) \leftsquigarrow \mathsf{Zip} \leftsquigarrow \left(\mathbf{R}_q, [i]_{i=1}^N\right)$.

   Reduce to multiset-equality-check:
4. Define $\mathsf{i} := (\mathbb{F}, n)$ and $\mathsf{x} := (\llbracket p \rrbracket, \llbracket q \rrbracket)$ and $\mathsf{w} := (\mathbf{S}_{p'}, \mathbf{S}_{q'})$
5. Invoke the RW streaming multiset-equality-check PIOP for $(\mathsf{i}, \mathsf{x}, \mathsf{w})$.

Lemma 5.16. *PIOP 6 and Algorithm 7 together comprise a read-write streaming PIOP for for $\mathscr{R}_{\mathsf{PC}}$ for n-variate multilinear polynomials with the following properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(N)$	$O(\log N)$	$O(N)$	$O(1)$	$O(N/\lvert\mathbb{F}\rvert)$	$O(\log N)$	$O(1)$

Proof. PIOP properties follow from the analysis of Chen et al. [CBBZ23] because P in Algorithm 7 sends the same values to V as in PIOP 6. Moreover, Algorithm 7 has the same time and space complexity as the RW streaming multiset-equality-check PIOP.

□

Remark 5.17. Similar to the RW streaming prover for the zerocheck PIOP, Algorithm 7 can be viewed as an interactive reduction from permcheck to sumcheck. Similar to ZToS, we extract a method PToS that performs Step 1 to Step 4 of Algorithm 7, and emits the resulting sumcheck claim.

5.3.5 HyperPlonk PIOP

We now discuss HyperPlonk's circuit representation, and then define the HyperPlonk relation $\mathscr{R}_{\mathsf{HPC}}$.

Definition 5.18. *For an arithmetic circuit $C : \mathbb{F}^k \to \mathbb{F}$, the* HyperPlonk circuit representation *is a tuple $A(C) = \left(\mathbb{F}, d, N, N_p, N_w, N_q, q, \pi, f_0, \ldots, f_{N_q-1}\right)$ where*

- *$\mathbb{F}$ is a finite field,*
- *N is the number of gates in C,*
- *N_p is the number of public inputs to C,*
- *$N_w - 1$ is the arity of C,*
- *N_q is the number of different types of gates in C,*
- *$q : \{0, \ldots, N-1\} \to \{0, \ldots, N_q - 1\}$ is a selector function such that $q(i)$ is the "type" of the i-th gate,*
- *$\pi : \{0, \ldots, N_w - 1\} \times \{0, \ldots, N-1\} \to \{0, \ldots, N_w - 1\} \times \{0, \ldots, N-1\}$ describes the wiring identity constraints, and*
- *$f_j : \mathbb{F}^{N_w - 1} \to \mathbb{F}$ is a degree-d map that describes the behavior of the j-th type of gate.*

We assume that N, N_p, N_w, N_q are powers of 2, i.e., that there exist $n, n_p, n_w, n_q \in \mathbb{N}$ such that $N = 2^n$, $N_p = 2^{n_p}$, $N_w = 2^{n_w}$, and $N_q = 2^{n_q}$.

Definition 5.19. *The HyperPlonk relation $\mathscr{R}_{\mathsf{HPC}}$ is an indexed relation consisting of the following tuples:*

$$(\mathsf{i}_{\mathsf{HPC}}, \mathsf{x}_{\mathsf{HPC}}, \mathsf{w}_{\mathsf{HPC}}) = (A, (\llbracket \boldsymbol{w} \rrbracket, \boldsymbol{p}), (\boldsymbol{w}))$$

where $A = \left(\mathbb{F}, d, N, N_p, N_w, N_q, q, \pi, f_0, \ldots, f_{N_q-1}\right)$ *is the arithmetic representation of* C *according to Definition 5.18,* $\boldsymbol{p} \in \mathbb{F}^{N_p}$ *is the public input vector, and* $\boldsymbol{w} \in \mathbb{F}^{N_w \times N}$ *is the witness vector where* $w_{i,j}$ *is the* i*-th input of gate* j *if* $i < N_w - 1$ *and the output of gate* j *if* $i = N_w - 1$.
These satisfy the following constraints:

- *wiring identity:* $w_{i,j} = w_{\pi(i,j)}$ *for all* $i \in \{0, \ldots, N_w - 1\}, j \in \{0, \ldots, N - 1\}$.
- *gate identity:* $f_{q(i)}\left(w_{0,i}, \ldots, w_{N_w-2,i}\right) = w_{N_w-1,i}$ *for all* $i \in \{0, \ldots, N - 1\}$.
- *public input consistency: check if the initial part of* $\boldsymbol{w}$ *is consistent with the public input. That is, for all* $j \in [0, \ldots, N_p - 1]$,

$$w_{i,j} = \begin{cases} p_j : & i = N_w - 1 \\ 0 : & i < N_w - 1 \end{cases}.$$

Remark 5.20. In Definition 5.19, N_w, N_q, d are all small constants and only depend on the arity of the circuit, number of different types of gates in the circuit, and maximum degree of a gate. For example, for simple binary circuits with only addition and multiplication gates, $N_w = 3, N_q = 2, d = 1$. Therefore $f_0, \ldots, f_{N_q-1}$ have constant-sized descriptions that can be stored in memory by both P and V.

Multilinear polynomial representation. Since $\mathscr{R}_{\mathsf{HPC}}$ is defined in the context of *vectors* instead of polynomials, we need a few more definitions to describe the PIOP for $\mathscr{R}_{\mathsf{HPC}}$ so that we can reduce it to the aforementioned zerocheck and permcheck PIOPs:

- $\tilde{p} : \{0,1\}^{n_p} \to \mathbb{F}$ where $\tilde{p}(\mathsf{bin}_{n_p}(i)) = p_i$.
- $\tilde{w} : \{0,1\}^{n_w+n} \to \mathbb{F}$ where $\tilde{w}(\mathsf{bin}_{n_w}(i), \mathsf{bin}_n(j)) = w_{i,j}$.
- $\tilde{q} : \{0,1\}^{n_q+n} \to \mathbb{F}$ where

$$\tilde{q}(\mathsf{bin}_{n_q}(i), \mathsf{bin}_n(j)) = \begin{cases} 1 : & q(j) = i \\ 0 : & \text{otherwise} \end{cases}.$$

- $\tilde{\pi} : \{0,1\}^{n_w+n} \to \mathbb{F}$ where $\tilde{\pi}(\mathsf{bin}_{n_w}(i), \mathsf{bin}_n(j)) = a + N_w \cdot b.$ such that $(a, b) = \pi(i, j)$.
- Define the "combining gate function" as follows:

$$\tilde{f}(\boldsymbol{X}, Y_0, Y_1, \ldots, Y_{N_w-1}) = f_{t(\boldsymbol{X})}(Y_0, Y_1, \ldots, Y_{N_w-2}) - Y_{N_w-1}$$

  where $\boldsymbol{X} \in \{0,1\}^{N_q}$ and $t(\boldsymbol{X})$ is the smallest index such that $X_t = 1$.

We additionally define $\hat{p}, \hat{w}, \hat{q}, \hat{\pi}$ as the multilinear extensions of $\tilde{p}, \tilde{w}, \tilde{q}, \tilde{\pi}$ respectively. We also define $\hat{f} : \mathbb{F}^n \to \mathbb{F}$ as follows:

$$\hat{f}(\boldsymbol{X}) = \tilde{f}\left(\hat{q}(\mathsf{bin}_{n_q}(0), \boldsymbol{X}), \ldots, \hat{q}(\mathsf{bin}_{n_q}(N_q - 1), \boldsymbol{X}), \hat{w}(\mathsf{bin}_{n_w}(0), \boldsymbol{X}), \ldots, \hat{w}(\mathsf{bin}_{n_w}(N_w - 1), \boldsymbol{X})\right)$$

By definition of $\hat{f}$, only one of the first N_q inputs to $\tilde{f}$ is equal to 1, and the rest are 0; if the $t(\boldsymbol{X})$-th input is 1, then clearly the $\boldsymbol{X}$-th gate of the circuit has type $t(\boldsymbol{X})$. Therefore, if the gate constraint is satisfied, then we must have $\hat{f}(\boldsymbol{X}) = f_{t(\boldsymbol{X})}(\hat{w}(\mathsf{bin}(0), \boldsymbol{X}), \ldots, \hat{w}(\mathsf{bin}(N_w - 2), \boldsymbol{X})) - \hat{w}(\mathsf{bin}(N_w - 1), \boldsymbol{X}) = 0$ for all $\boldsymbol{X} \in \{0,1\}^n$.

Indexer and Preprocessing. Both P and V can compute $\hat{p}$ independently in the preprocessing phase since they have access to $\boldsymbol{p}$. Moreover, given the circuit C, the PIOP Indexer can initialize $\hat{q}$ and $\hat{\pi}$ and send them to P, and similarly send $[\![\hat{q}]\!]$ and $[\![\hat{\pi}]\!]$ to V.

HyperPlonk PIOP. Chen et al. [CBBZ23] propose the following PIOP for Definition 5.19:

PIOP 7: HyperPlonk

Indexer: Initialize $\hat{q}$ and $\hat{\pi}$ and send them to P. Also send $[\![\hat{q}]\!]$ and $[\![\hat{\pi}]\!]$ to V.

Protocol: $\langle \mathsf{P}(\mathsf{i}_{\mathsf{HPC}}, \mathsf{x}_{\mathsf{HPC}}, \mathsf{w}_{\mathsf{HPC}}), \mathsf{V}(\mathsf{i}_{\mathsf{HPC}}, \mathsf{x}_{\mathsf{HPC}}, [\![\hat{q}]\!], [\![\hat{\pi}]\!]) \rangle$:

1. P initializes $[\![\hat{w}]\!]$ and sends it to V.
2. Gate identity check: P and V invoke the zerocheck PIOP for the claim "$\hat{f}(\boldsymbol{x}) = 0$ for all $\boldsymbol{x} \in \{0,1\}^n$".
3. Wiring identity check: P and V invoke the permcheck PIOP for the claim "$\hat{w}(\boldsymbol{x}) = \hat{w}(\pi(\boldsymbol{x}))$ for all $\boldsymbol{x} \in \{0,1\}^{n+n_w}$".
4. Public input consistency check: V samples $\boldsymbol{r} \xleftarrow{\$} \mathbb{F}^{n_p}$ and checks $\hat{p}(\boldsymbol{r}) = \hat{w}(0^{n+n_w-n_p}, \boldsymbol{r})$ by making an oracle query to $\hat{p}$ and $\hat{w}$.

We construct a read-write streaming prover for the above PIOP:

Read-write Streaming Algorithm 8: HyperPlonk Relation

$\mathsf{P}(\mathsf{i}_{\mathsf{HPC}}, \mathsf{x}_{\mathsf{HPC}}, \mathbf{R}_q(\hat{q}), \mathbf{R}_w(\hat{w}), \mathbf{R}_\pi(\hat{\pi}))$:

1. Create $[\![\hat{w}]\!]$ and send it to V.

Split selector polynomial into N_q streams: a stream for each type of gate:

2. For i in $[0, \ldots, N_q - 1]$: $\mathbf{S}_{q,i}.\mathsf{init}(N)$.
3. $(\mathbf{S}_{q,0}, \ldots, \mathbf{S}_{q,N_q-1}) \leftarrow \mathsf{SplitLSB}(\hat{q}, n_q)$.

Split witness polynomial into N_w streams so that the i-th stream stores the i-th input of all gates and the $(N_w - 1)$-th stream stores the output of all gates:

4. For i in $[0, \ldots, N_w - 1]$: $\mathbf{S}_{w,i}.\mathsf{init}(N)$.
5. $(\mathbf{S}_{w,0}, \ldots, \mathbf{S}_{w,N_w-1}) \leftarrow \mathsf{SplitLSB}(\hat{w}, n_w)$.

Using PToS, initialize sumcheck instance of the permcheck for the wiring identity:

6. $(\mathsf{i}_1 = (\_, \_, k_1, h_1), \mathsf{x}_1, \mathsf{w}_1) \leftarrow \mathsf{PToS}((\mathbb{F}, n + n_w, \pi), ([\![\hat{w}]\!], [\![\hat{w}]\!], [\![\hat{\pi}]\!]), (\mathbf{R}_w, \mathbf{R}_w))$.

Using ZToS initialize sumcheck instance of the zerocheck for the gate identity:

7. Using the combining gate function $\tilde{f}$, define $\mathsf{i}_{\mathsf{ZC}} := (\mathbb{F}, n, N_w + N_q, \tilde{f})$
8. Define $\mathsf{x}_{\mathsf{ZC}} := ([\![q_0]\!], \ldots, [\![q_{N_q-1}]\!], [\![w_0]\!], \ldots, [\![w_{N_w-1}]\!])$.
9. Define $\mathsf{w}_{\mathsf{ZC}} := (\mathbf{S}_{q,0}, \ldots, \mathbf{S}_{q,N_q-1}, \mathbf{S}_{w,0}, \ldots, \mathbf{S}_{w,N_w-1})$.
10. $(\mathsf{i}_2 = (\_, \_, k_2, h_2), \mathsf{x}_2, \mathsf{w}_2) \leftarrow \mathsf{ZToS}(\mathsf{i}_{\mathsf{ZC}}, \mathsf{x}_{\mathsf{ZC}}, \mathsf{w}_{\mathsf{ZC}})$.

Batch the two sumcheck instances together:

11. Define $\mathsf{i} := (\mathbb{F}, n, k_1 + k_2, 2, h_1, h_2)$ and $\mathsf{x} := (\mathsf{x}_1, \mathsf{x}_2)$ and $\mathsf{w} = (\mathsf{w}_1, \mathsf{w}_2)$
12. P and V invoke a RW streaming batch sumcheck PIOP for sumprod polynomials for $(\mathsf{i}, \mathsf{x}, \mathsf{w})$ as defined above.

Lemma 5.21. *PIOP 7 and Algorithm 8 together comprise a read-write streaming PIOP for $\mathscr{R}_{\mathsf{HPC}}$ with the following properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(k \cdot \ell \cdot N)$	$O(k + \ell + \log N)$	$O(kN)$	$O(k)$	$O(kN/\lvert\mathbb{F}\rvert)$	$O(k \log N)$	$O(k)$

where $k = d \cdot (N_q + N_w)$ and ℓ is the number of monomials required to represent the combining gate function $\tilde{f}$.

Proof. The efficiency parameters of Algorithm 8 follow from the efficiency parameters of the permcheck and zerocheck PIOPs (Lemma 5.16 and Lemma 5.8).

Algorithm 8 assumes without loss of generality that $\tilde{f}$ is a multilinear polynomial ($d = 1$), which enables the invocation of the RW streaming zerocheck PIOP on it. This assumption is equivalent to the assumption

that all types of gates are multilinear polynomials. If a gate has degree $d > 1$ with arity $N_w - 1$, then it can be represented as a multilinear polynomial in $d \cdot (N_w - 1)$ variables whose inputs can be obtained by creating d copies of each input stream defined in Step 2 and Step 4.

Given the aforementioned generalization to higher degree gates, there are $d \cdot (N_q + N_w)$ number of input RW streams for the zerocheck PIOP call, each of length N. Therefore, the time, random-access space, and streaming space required to invoke the RW streaming zerocheck PIOP are $O(k \cdot \ell \cdot N), O(k + \ell + \log N), O(k \cdot N)$ respectively, where k, ℓ are defined in the lemma statement.

Moreover, $\hat{w}$ has size $d \cdot N_w \cdot N$. Therefore, the time, random-access space, and streaming space required to invoke the RW streaming permcheck PIOP are $O(d \cdot N_w \cdot N), O(d \cdot N_w + \log N), O(d \cdot N_w \cdot N)$ respectively.

The remaining PIOP properties follow from analysis of Chen et al. [CBBZ23]. □

6 Streaming polynomial commitment schemes

In this section we recall the definition of polynomial commitment (PC) schemes and present our read-write streaming algorithms for the PC scheme of Papamanthou, Shi, and Tamassia [PST13]. We defer to Appendices B and C our RW streaming algorithms for other PC schemes based on inner product arguments.

6.1 Multilinear polynomial commitment schemes

A multilinear polynomial commitment scheme is a tuple of algorithms $\mathsf{PC} = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Check})$ with the following syntax.

- $\mathsf{PC.Setup}(1^\lambda, n) \to (\mathsf{ck}, \mathsf{vk})$. On input a security parameter λ (in unary), and a maximum number of variables $n \in \mathbb{N}$, PC.Setup samples committer and verifier keys ck and vk.
- $\mathsf{PC.Commit}(\mathsf{ck}, p) \to C$. On input ck and a multilinear polynomial p over the field $\mathbb{F}$, PC.Commit outputs a commitment C to p.
- $\mathsf{PC.Open}(\mathsf{ck}, (C, \boldsymbol{z}, y), p) \to \pi$. On input ck, a polynomial p, a commitment to it C, an evaluation point $\boldsymbol{z}$, and a claimed evaluation y, PC.Open outputs an evaluation proof π asserting that y is indeed the evaluation of p at $\boldsymbol{z}$.
- $\mathsf{PC.Check}(\mathsf{vk}, (C, \boldsymbol{z}, y), \pi) \to b \in \{0, 1\}$. On input vk, a commitment C, an evaluation point $\boldsymbol{z}$, a claimed evaluation y, and an evaluation proof π, PC.Check outputs 1 if π attests that the polynomial p committed in C evaluates to y at $\boldsymbol{z}$.

A polynomial commitment scheme PC must satisfy standard completeness and extractability properties, but we do not recall these definitions here, as we only construct read-write streaming versions for the Commit and Open algorithms of existing schemes. We thus cannot affect extractability, and we show that completeness is preserved by demonstrating that our algorithms produce the same output as their non-streaming counterparts.

6.2 The PST13 PC scheme

We now recall the polynomial commitment scheme of Papamanthou, Shi, and Tamassia [PST13] as presented in [XZZPS19].

$\mathsf{PST.Setup}(1^\lambda, n) \to (\mathsf{ck}, \mathsf{vk})$:
1. Obtain $\langle \mathsf{group} \rangle = (\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, G, H) \leftarrow \mathsf{SampleGrp}(1^\lambda)$.
2. Sample random $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_n) \xleftarrow{\$} \mathbb{F}^n$.
3. For each j in $[0, 1, \ldots, n]$:
4. Set $\mathsf{ck}_j := [\mathrm{eq}_j(\boldsymbol{\alpha}_{[n-j+1:n]}, \boldsymbol{i}) \cdot G]_{\boldsymbol{i} \in \{0,1\}^j}$.
5. Set $\mathsf{ck} := ([\mathsf{ck}_j]_{j=0}^n, \langle \mathsf{group} \rangle)$.
6. Set $\mathsf{vk} := ([\alpha_i \cdot H]_{i \in [n]}, \langle \mathsf{group} \rangle)$.
7. Output $(\mathsf{ck}, \mathsf{vk})$.

$\mathsf{PST.Commit}(\mathsf{ck}, \boldsymbol{p}) \to C_p$:
1. Parse ck as $([\mathsf{ck}_j]_{j=0}^n, \langle \mathsf{group} \rangle)$.
2. Parse ck_n as $[\mathrm{eq}_n(\boldsymbol{\alpha}, \boldsymbol{i}) \cdot G]_{\boldsymbol{i} \in \{0,1\}^n}$.
3. Output $C_p := \sum_{\boldsymbol{i} \in \{0,1\}^n} p_i \cdot \mathrm{eq}_n(\boldsymbol{\alpha}, \boldsymbol{i}) \cdot G = \langle \boldsymbol{p}, \mathsf{ck}_n \rangle$.

$\mathsf{PST.Open}(\mathsf{ck}, (C_p, \boldsymbol{z}, y), \boldsymbol{p}) \to \pi$:
Parse: $\mathsf{ck} = ([\mathsf{ck}_j]_{j=0}^n, \langle \mathsf{group} \rangle)$.
1. For each j in $[1, \ldots, n]$:
2. Compute $\boldsymbol{q}_j$ corresponding to $q_j(\boldsymbol{X})$ such that $p(\boldsymbol{X}) - y = \sum_{i=1}^n q_i(\boldsymbol{X}) \cdot (X_i - z_i)$.
3. Compute $\pi_j := q_j(\boldsymbol{\alpha}) \cdot G = \langle \boldsymbol{q}_j, \mathsf{ck}_{n-j} \rangle$.
4. Output evaluation proof $\pi := (\pi_1, \ldots, \pi_n)$.

$\mathsf{PST.Check}(\mathsf{vk}, (C_p, \boldsymbol{z}, y), \pi) \to \{0, 1\}$:
Parse: $\mathsf{vk} = ([\alpha_i \cdot H]_{i \in [n]}, \langle \mathsf{group} \rangle)$ and $\pi = (\pi_1, \ldots, \pi_n)$.
1. Accept if $e(C_p - y \cdot G, H) = \sum_{i=1}^n e(\pi_i, (\alpha_i - z_i) \cdot H)$.

6.2.1 Read-write streaming algorithms for PST13

Our read-write streaming algorithms for PST.Commit and PST.Open are described below:

Read-write Streaming Algorithm 9: PST.Commit

$\mathsf{PST.Commit}(\mathsf{ck}, \mathbf{R}_p(p))$:
Parse: $\mathsf{ck} = ([\mathbf{R}_{\mathsf{ck}_j}(\mathsf{ck}_j)]_{j=0}^n, \langle \mathsf{group} \rangle)$.
1. Output $C_p \leftarrow \mathsf{InnerProd}(\mathbf{R}_p, \mathbf{R}_{\mathsf{ck}_n})$.

Read-write Streaming Algorithm 10: PST.Open

$\mathsf{PST.Open}(\mathsf{ck}, (C_p, \boldsymbol{z}, y), \mathbf{R}_p(p))$:
Parse: $\mathsf{ck} = ([\mathbf{R}_{\mathsf{ck}_j}(\mathsf{ck}_j)]_{j=0}^n, \langle \mathsf{group} \rangle)$.
1. For each i in $[1, \ldots, n]$:
2. Obtain $(\mathbf{R}_g, \mathbf{R}_h) \leftarrow \mathsf{SplitLR}(\mathbf{R}_p)$.
3. $\mathbf{R}_h \leftarrow \mathsf{LinComb}(1, -1, \mathbf{R}_h, \mathbf{R}_g)$.
4. Compute the commitment $\pi_i \leftarrow \mathsf{InnerProd}(\mathbf{R}_h, \mathbf{R}_{\mathsf{ck}_{n-i}})$.
5. $\mathbf{R}_g \leftarrow \mathsf{LinComb}(1, z_i, \mathbf{R}_g, \mathbf{R}_h)$.
6. Set $\mathbf{R}_p \leftarrow \mathbf{R}_g$.
7. Output $\pi := (\pi_1, \ldots, \pi_n)$.

Lemma 6.1. *Algorithm 9 and Algorithm 10 are read-write streaming algorithms for* PST.Commit *and* PST.Open *respectively with the following efficiency when committing to n-variate multilinear polynomials (where $N = 2^n$):*
- *Algorithm 9 requires $O(N)$ running time, $O(\log N)$ random-access space and $O(1)$ streaming space.*
- *Algorithm 10 requires $O(N)$ running time, $O(\log N)$ random-access space and $O(N)$ streaming space.*

Proof. It is easy to inspect that Algorithm 9 realizes PST.Commit and requires $O(N)$ group operations and $O(\log N)$ random-access space.

We now verify that Algorithm 10 implements the same function as PST.Open by writing out what Algorithm 10 is computing in its streams. Setting $r_0(X_1, \ldots, X_n) := p(X_1, \ldots, X_n)$, PST.Open recursively defines $r_j(X_{j+1}, \ldots, X_n)$ and $q_j(X_{j+1}, \ldots, X_n)$ for each $j \in [n]$ as follows:

$$\begin{aligned} r_{j-1}(X_j, \ldots, X_n) &= g_j(X_{j+1}, \ldots, X_n) + X_j \cdot h_j(X_{j+1}, \ldots, X_n) \\ &= (g_j(X_{j+1}, \ldots, X_n) + z_j \cdot h_j(X_{j+1}, \ldots, X_n)) + (X_j - z_j) \cdot h_j(X_{j+1}, \ldots, X_n) \\ &:= r_j(X_{j+1}, \ldots, X_n) + (X_j - z_j) \cdot q_j(X_{j+1}, \ldots, X_n) \end{aligned}$$

It then commits to each $q_i(\boldsymbol{X})$ as $\pi_i := q_i(\boldsymbol{\alpha}) \cdot G = \langle \boldsymbol{q}_j, \mathsf{ck}_{n-j} \rangle$.

Algorithm 10 identically obtains evaluations of these polynomials over their respective boolean hypercubes as follows: for every point $\boldsymbol{i} \in \{0, 1\}^{n-j+1}$, if $i_1 = 0$, we have $g_j(i_2, \ldots, i_{n-j+1}) = r_{j-1}(0, i_2, \ldots, i_{n-j+1})$.

If $i_1 = 1$, then $r_{j-1}(1, i_2, \ldots, i_{n-j+1}) = g_j(i_2, \ldots, i_{n-j+1}) + h_j(i_2, \ldots, i_{n-j+1})$. Thus, the algorithm sets $h_j(i_2, \ldots, i_{n-j+1}) = r_{j-1}(1, i_2, \ldots, i_{n-j+1}) - r_{j-1}(0, i_2, \ldots, i_{n-j+1})$. It then defines $r_j(i_2, \ldots, i_{n-j+1}) := g_j(i_2, \ldots, i_{n-j+1}) + z_j \cdot h_j(i_2, \ldots, i_{n-j+1})$ and $q_j(i_2, \ldots, i_{n-j+1}) := h_j(i_2, \ldots, i_{n-j+1})$.

Algorithm 10 will now need to commit to the witness polynomials $q_1(\boldsymbol{X}), \ldots, q_n(\boldsymbol{X})$. The commitment to the polynomial $q_j(X_{j+1}, \ldots, X_n)$ can be computed as

$$q_j(\alpha_{j+1}, \ldots, \alpha_n) \cdot G = \sum_{\boldsymbol{i} \in \{0,1\}^{n-j}} q(\boldsymbol{i}) \cdot \mathrm{eq}_{n-j}(\boldsymbol{\alpha}_{[j+1,n]}, \boldsymbol{i}) \cdot G$$

This is realised by computing $\pi_j := \mathsf{InnerProd}(\mathbf{R}_{q_j}, \mathbf{R}_{\mathsf{ck}_{n-j}})$.

Efficiency analysis. We now inspect the efficiency of Algorithm 10. In the j-th round of the for loop, the algorithm computes $h_j(X_{j+1}, \ldots, X_n)$ and $h_j(X_{j+1}, \ldots, X_n)$ given $r_{j-1}(X_j, \ldots, X_n)$. It then sets $q_j(X_{j+1}, \ldots, X_n) := h_j(X_{j+1}, \ldots, X_n)$ and $r_j(X_{j+1}, \ldots, X_n) := g_j(X_{j+1}, \ldots, X_n) + z_j \cdot h_j(X_{j+1}, \ldots, X_n)$.

To do this, the RW streaming algorithm starts by setting $\boldsymbol{R} := \boldsymbol{p}$, which is a vector of length 2^n. In the j-th step of the recursion, the length of $\boldsymbol{R}$ is 2^{n-j+1}. It sets $\boldsymbol{g} := \boldsymbol{R}_L$ and $\boldsymbol{h} := \boldsymbol{R}_R - \boldsymbol{R}_L$, which takes $O(2^{n-j+1})$ time. It then proceeds to set $\boldsymbol{R} \leftarrow \boldsymbol{g} + z_j \cdot \boldsymbol{h}$, which takes $O(2^{n-j})$ time, and commits to $\boldsymbol{h}$, which requires $O(2^{n-j})$ group operations. In total, the j-th round takes $O(2^{n-j+1})$ time.

Thus the entire protocol thus takes $O(2^n) = O(N)$ time. Since all the inputs are provided as streams, the algorithm only requires $O(n) = O(\log N)$ space to keep track of its position in the stream.

□

7 Implementation

We implemented Scribe in Rust[18] atop the `arkworks` framework [con22] by adapting and extending the HyperPlonk implementation.[19] Our implementation is modular and allows for switching out almost all components, from the PC scheme to the underlying elliptic curve. We also adapt the `jellyfish` circuit construction framework[20] to output witness streams for the circuits it constructs. We now describe the infrastructure we developed to enable efficient high-level implementations of read-write streaming algorithms.

7.1 Tools for read-write streams

To implement the read-write streaming algorithms outlined in Sections 2.6 and 2.8, we developed a slew of tools that allow for efficient streaming operations on vectors stored on disk. We believe that these tools will be of independent interest for future work on read-write streaming algorithms.

Low-overhead serialization and deserialization. To ensure that data can be efficiently streamed to and from disk, we implement custom serialization and deserialization routines for common types (e.g., integers, fields, and group elements). Our routines simultaneously minimize on-disk size and the overhead of (de)serialization. For example, we serialize elliptic curve points via a non-standard compression scheme [FOSS17] that compresses two points into three field elements. While compressed size is worse than in the standard method, decompression cost is much lower, leading to overall savings.

File-backed vectors. We provide a new `FileVec` vector type whose API resembles that of Rust's standard `Vec` type, but which uses temporary files on disk as backing storage for the vector. `FileVec` automatically switches to memory-backed storage when the vector becomes small enough. We engineer `FileVec`'s API to ensure sequential accesses. To ensure that these accesses do not populate the OS's file-system cache and increase memory usage, `FileVec` opens files with the `O_DIRECT` flag on Linux and the `F_NOCACHE` flag on MacOS.

File-backed multilinear polynomials. We use `FileVec` to build a multilinear polynomial type whose evaluations are either stored on disk, or can be streamed in a read-only manner when possible (e.g., the Lagrange basis polynomial).

Batched iterators. We implement the various stream operations required by our algorithms via a new *batched iterator* interface. We specify this interface as a Rust trait, `BatchedIterator`, whose API resembles the standard `Iterator` trait, but which processes (in parallel via `rayon`[21]) a *batch* of elements on each iteration. `BatchedIterator` supports common operations such as `map`, `filter`, `enumerate`, and `zip`, and different `BatchedIterator` instances can be composed into complex pipelines that minimize disk I/O. We also implement `BatchedIterator` for `FileVec`.

[18] https://github.com/Pratyush/scribe-system
[19] https://github.com/EspressoSystems/hyperplonk
[20] https://github.com/EspressoSystems/jellyfish
[21] https://github.com/rayon-rs/rayon

8 Evaluation

We evalauted Scribe's performance in a variety of settings, with the aim of answering the following questions:

- **Q1:** Can Scribe scale to prove large circuits, and how does time and disk space usage scale with circuit size?
- **Q2:** What is the overhead over memory-intensive proving as the compute-I/O ratio varies?
- **Q3:** What is the memory cost of witness synthesis?

Baselines. We compare Scribe against the following baselines: memory-intensive HyperPlonk (HP) [CBBZ23] and low-memory Gemini [BCHO22] and Epistle [ZCLKZ24]. We chose these baselines since they operate in the same regime as Scribe: low proof size, and support for general circuits. Both Scribe and HP use PST as the PC scheme. Other low-memory SNARKs discussed in Section 3 fail to meet one or more of these criteria. All baselines use `arkworks` v0.4, while Scribe uses v0.5. Running `arkworks`'s benchmark suite on both versions found minimal performance differences; see Appendix G for details.

We also compared against state-of-the-art industrial proof systems Halo2[22] and Plonky2[23], but they could not scale to large instances, and so we omitted comparisons against them.

Parameters. We configure all proof systems to use the BLS12-381 elliptic curve. Scribe's `FileVecs` switch over to memory-backed storage when they contain $\leq 2^{17}$ elements.

Experimental setups. We ran our experiments on an AWS EC2 `i3en.3xlarge` instance with an Intel Xeon Platinum 8175 CPU with 12 cores at 3.1 GHz, 7.5 TiB of storage, and 96 GiB of memory. The on-demand price of this instance is $1.356/hr at the time of writing. We enforce memory and bandwidth limits via Linux's `cgroups` functionality. Scribe, Gemini, and Epistle are configured to use at most 750 MiB of memory, while HP is allowed to use all available memory.

To test Scribe's performance on a low-memory mobile device, we also ran some experiments on an iPhone 13 Pro Max with 6 cores, 6 GiB RAM, and 256 GiB of disk storage.

8.1 Q1: Scalability

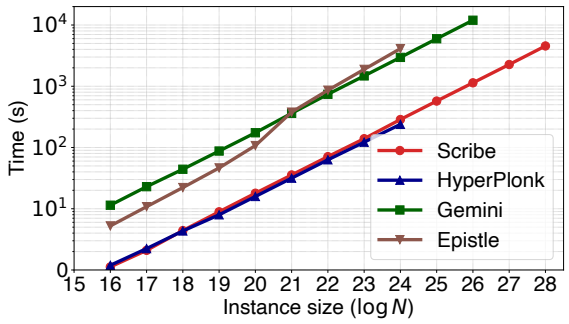

Figure 3: Proving time on AWS.

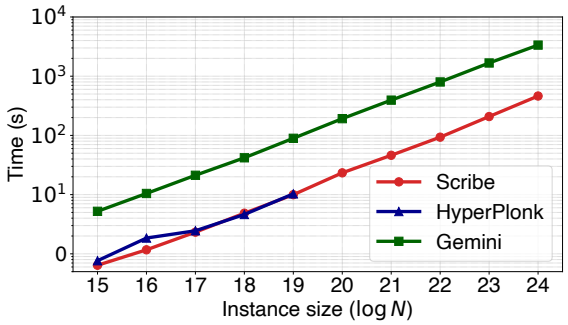

Figure 4: Proving time on iPhone.

We evaluated Scribe's resource usage in both experimental setups when scaling to large instances. We focus below on the AWS setup, as the iPhone results are qualitatively similar. The takeaway is that Scribe can prove large instances with low time and disk usage overheads.

[22] https://github.com/privacy-scaling-explorations/halo2
[23] https://github.com/0xPolygonZero/plonky2

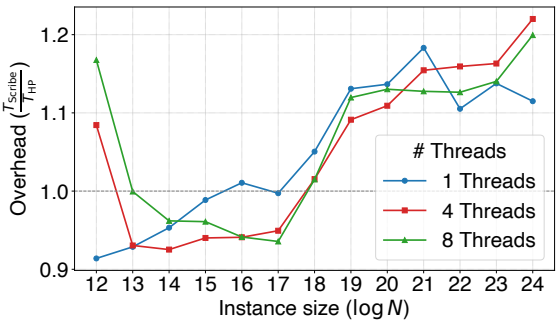

Figure 5: Overhead of Scribe as number of threads varies.

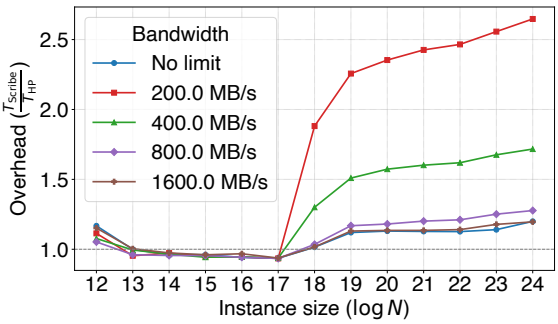

Figure 6: Overhead of Scribe when bandwidth is limited.

Q1a: running time. Fig. 3 shows that Scribe's latency scales linearly with instance size. This latency is $10.5\times$ smaller than Gemini's, $13.5\times$ smaller than Epistle's, and is just 1.1-1.2$\times$ larger than HP's latency. Unlike Epistle, Scribe does not incur an increase in proving time when moving from in-memory to streaming proving.

Q1b: disk space. Scribe's disk usage scales linearly with instance size, and grows at roughly 480 bytes per gate. At the largest instance size (2^{28}), Scribe used 145 GB of disk.

Proving time on iPhone. Fig. 4 shows that Scribe's latency on the iPhone is $7\times$ smaller than Gemini's, and is just 1.1-1.2$\times$ larger than HP's latency. Scribe can scale to 2^{24} gates on the iPhone, while HP exhausts system memory at 2^{19} gates ($32\times$ fewer). We omit a comparison with Epistle as it is slower than Gemini. We omit measurements of disk usage as these are similar to the AWS setup.

8.2 Q2: Overhead of read-write-streaming

We investigate the overhead of read-write streaming in Scribe via experiments which vary compute-to-I/O ratios.

Faster compute via more threads. The first experiment varies the number of threads used by Scribe and HP and is reported in Fig. 5. The latter shows that Scribe's overhead does not vary significantly with the number of threads, and so, for a reasonable number of threads, Scribe is not I/O bound.

Slower I/O via bandwidth throttling. Our second experiment fixes the number of threads to 8 and instead varies disk bandwidth. Fig. 6 shows that when throttling bandwidth degrades Scribe's latency, indicating an I/O bottleneck. However, even this degraded latency is $< 3\times$ that of HP.

8.3 Cost of witness synthesis

To evaluate the time and memory costs of witness synthesis (converting high-level witnesses to wire/variable assignments) in our circuit programming framework, we sample randomly-generated circuits and synthesize witnesses for these. Our circuit-sampling process takes as input a number of gates, a size for a working set S that specifies the number of variables that are in use at any given moment, and a *replacement probability* that specifies the probability that a variable in S is replaced by a new variable. It produces a circuit whose every gate is randomly sampled to be either an add or a multiply gate, and each gate's inputs are randomly sampled from S. We evaluated the space and time costs of witness synthesis for circuits with sizes ranging from 2^{15} to 2^{28} with a variety of sizes for S. The following table shows that these costs are a miniscule fraction of proof generation costs.

working set size	memory	% of proving time
2^{15}	$< 68\,\mathrm{MB}$	$< 1\%$
2^{17}	$< 72\,\mathrm{MB}$	$< 1\%$
2^{20}	$< 110\,\mathrm{MB}$	$< 1\%$

A An alternative PIOP for permcheck

Recall that the HyperPlonk PIOP invokes a permcheck PIOP (Section 5.3.4) to check the wiring constraints of the circuit. In this section we present a concrete optimization for the permcheck PIOP, which we call the *split permcheck* PIOP. In the split permcheck PIOP, the witness polynomials p and q are not available as single multilinear polynomials, but are *split* into ν multilinear polynomials each. Additionally, there are ν permutations $\pi^{(1)}, \ldots, \pi^{(\nu)}$, and the prover wants to prove to the verifier that for all $i \in [\nu]$ and $\boldsymbol{x} \in \{0,1\}^n$, $p^{(i)}(\boldsymbol{x}) = q^{(i)}(\pi^{(i)}(\boldsymbol{x}))$. We discuss how this PIOP is useful in proving the wiring constraint in Section A.4.

To build the split permcheck PIOP, we first build a PIOP for sumcheck for *rational functions* in Section A.1, use it to obtain the *split multiset-equality-check* PIOP in Section A.2 using techniques similar to [Hab22], and then use the split multiset-equality-check PIOP to build the split permcheck PIOP in Section A.3.

A.1 Sumcheck for rational functions

We begin by defining rational functions:

Definition A.1 (Rational function)**.** *An n-variate (d, ℓ)-rational function is a function f such that*

$$f(\boldsymbol{X}) = \frac{p(\boldsymbol{X})}{q(\boldsymbol{X})} = \frac{h_p\left(p^{(1)}(\boldsymbol{X}), \ldots, p^{(d)}(\boldsymbol{X})\right)}{h_q\left(q^{(1)}(\boldsymbol{X}), \ldots, q^{(d)}(\boldsymbol{X})\right)} \ ,$$

where p, q are (d, ℓ)-sumprod polynomials, $p^{(i)}, q^{(i)}$ are multilinear polynomials for all $i \in [d]$, and h_p, h_q are multilinear polynomials.

We can now define the relation for sumcheck for rational functions:

Definition A.2 (Sumcheck for rational functions)**.** *This relation $\mathscr{R}_{\mathsf{MRSC}}$ contains tuples of the form*

$$(\mathsf{i}_{\mathsf{MRSC}}, \mathsf{x}_{\mathsf{MRSC}}, \mathsf{w}_{\mathsf{MRSC}}) = ((\mathbb{F}, n, d, h_p, h_q), ([\![p_1]\!], \ldots, [\![p_d]\!], [\![q_1]\!], \ldots, [\![q_d]\!], \sigma), (p_1, \ldots, p_d, q_1, \ldots, q_d))$$

such that $\sum_{\boldsymbol{x} \in \{0,1\}^n} \hat{h}_p(\boldsymbol{x}) / \hat{h}_q(\boldsymbol{x}) = \sigma$ where $\hat{h}_p, \hat{h}_q$ are (d, ℓ)-sumprod polynomials. In particular, $\hat{h}_p(\boldsymbol{X}) = h_p\left(p^{(1)}(\boldsymbol{X}), \ldots, p^{(d)}(\boldsymbol{X})\right)$ and $\hat{h}_q(\boldsymbol{X}) = h_q\left(q^{(1)}(\boldsymbol{X}), \ldots, q^{(d)}(\boldsymbol{X})\right)$.

The following is a PIOP for $\mathscr{R}_{\mathsf{MRSC}}$:

PIOP 8: Sumcheck for (d, ℓ)-rational functions

$\langle \mathsf{P}(\mathsf{i}_{\mathsf{MRSC}}, \mathsf{x}_{\mathsf{MRSC}}, \mathsf{w}_{\mathsf{MRSC}}), \mathsf{V}(\mathsf{i}_{\mathsf{MRSC}}, \mathsf{x}_{\mathsf{MRSC}}) \rangle$:

1. P computes $f : \{0,1\}^n \to \mathbb{F}$ such that $f(\boldsymbol{X}) = \hat{h}_p(\boldsymbol{X}) \cdot \hat{h}_q(\boldsymbol{X})^{-1}$.
2. P sends $[\![\hat{f}]\!]$ to V where $\hat{f}$ is the multilinear extension of f.
3. P and V invoke a zerocheck PIOP for the polynomial $(\hat{h}_q \cdot \hat{f} - \hat{h}_p) = 0$.
4. P and V invoke a sumcheck PIOP for multilinear polynomials for the claim "$\sum_{\boldsymbol{x} \in \{0,1\}^n} \hat{f}(\boldsymbol{x}) = \sigma$".

The key idea of the above protocol is as follows:

1. Obtain the multilinear polynomial $\hat{f}(\boldsymbol{X})$ which is equal to $\hat{h}_p(\boldsymbol{X}) \cdot \hat{h}_q(\boldsymbol{X})^{-1}$ on the boolean hypercube.
2. P commits to $\hat{f}$ and sends $[\![\hat{f}]\!]$ to V.
3. In order to prove to V that $\hat{f}(\boldsymbol{x}) = p(\boldsymbol{x})/q(\boldsymbol{x})$ for all $\boldsymbol{x} \in \{0,1\}^n$, P proves that $\hat{f}(\boldsymbol{x}) \cdot q(\boldsymbol{X}) - p(\boldsymbol{X}) = 0$ for all $\boldsymbol{x} \in \{0,1\}^n$, which is achieved by engaging in a zerocheck PIOP.
4. Finally, since $\hat{f}$ is multilinear, P and V can engage in a sumcheck protocol on $\hat{f}$.

We now present a RW streaming prover for $\mathscr{R}_{\mathsf{MRSC}}$:

Read-write Streaming Algorithm 11: SUMCHECK for (d,ℓ)-rational functions

$\mathsf{P}(\mathsf{i}_{\mathsf{MRSC}}, \mathsf{x}_{\mathsf{MRSC}}, (\mathbf{R}_p^{(1)}(p^{(1)}), \ldots, \mathbf{R}_p^{(d)}(p^{(d)}), \mathbf{R}_q^{(1)}(q^{(1)}), \ldots, \mathbf{R}_q^{(d)}(q^{(d)})))$:

Obtain read-only stream of evaluations of $\hat{f}$ over $\{0,1\}^n$:

1. $\mathbf{S}_p \leftsquigarrow \mathsf{Map}(h_p) \leftsquigarrow \mathsf{Zip} \leftsquigarrow (\mathbf{R}_p^{(1)}, \ldots, \mathbf{R}_p^{(d)})$.
2. $\mathbf{S}_q \leftsquigarrow \mathsf{Map}(h_q) \leftsquigarrow \mathsf{Zip} \leftsquigarrow (\mathbf{R}_q^{(1)}, \ldots, \mathbf{R}_q^{(d)})$.
3. $\mathbf{S}_f \leftsquigarrow \mathsf{Map}(/) \leftsquigarrow \mathsf{Zip} \leftsquigarrow (\mathbf{S}_p, \mathbf{S}_q)$.
4. Create the oracle $[\![\hat{f}]\!]$ using $\mathbf{S}_f$ and send it to V.

Obtain sumcheck claim for the zerocheck using ZToS:

5. Define $t_0(X_1, \ldots, X_k, Y_1, \ldots, Y_k, Z) := h_q(Y_1, \ldots, Y_k) \cdot Z - h_p(X_1, \ldots, X_k)$.
6. $\mathsf{i}_z := (\mathbb{F}, n, d+1, t_0)$
7. $\mathsf{x}_z := ([\![p^{(1)}]\!], \ldots, [\![p^{(d)}]\!], [\![q^{(1)}]\!], \ldots, [\![q^{(d)}]\!], [\![\hat{f}]\!])$
8. $\mathsf{w}_z := (\mathbf{R}_p^{(1)}, \ldots, \mathbf{R}_p^{(d)}, \mathbf{R}_q^{(1)}, \ldots, \mathbf{R}_q^{(d)}, \mathbf{S}_f)$
9. $(\mathsf{i}_z = (\_, \_, \_, t_1), \mathsf{x}_z, \mathsf{w}_z) \leftarrow \mathsf{ZToS}(\mathsf{i}_z, \mathsf{x}_z, \mathsf{w}_z)$.

Batch the sumcheck instance above with the sumcheck over $\hat{f}$:

10. Define identity function $\mathsf{id}(X) := X$.
11. Define $\mathsf{i} := (\mathbb{F}, n, d+1, 2, \mathsf{id}, t_1)$ and $\mathsf{x} := ((\hat{[\![f]\!]}, \sigma), \mathsf{x}_z)$ and $\mathsf{w} := ((\mathbf{S}_f), \mathsf{w}_z)$
12. P and V invoke a RW streaming PIOP for a batch of sumprod polynomials for $(\mathsf{i}, \mathsf{x}, \mathsf{w})$ as defined above.

Lemma A.3. *PIOP 8 and Algorithm 11 together comprise a read-write streaming PIOP for $\mathscr{R}_{\mathsf{MRSC}}$ for n-variate (d,ℓ)-rational functions with the following properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(d\ell N)$	$O(d+\ell+\log N)$	$O(dN)$	$O(d)$	$O(dN/\lvert\mathbb{F}\rvert)$	$O(d \log N)$	$O(d)$

Proof. Time and space efficiency properties follow from the analysis of RW streaming zerocheck for (d,ℓ)-sumprod polynomials. The prover is complete by construction, and the soundness error is $O(dN/|\mathbb{F}|)$ because the degree of f is $O(dN)$. □

A.2 Split multiset-equality-check

The following PIOP is inspired from the multiset-equality-check of [Hab22]. Given a split size ν, multilinear polynomials $p^{(1)}, \ldots, p^{(\nu)}$ and multilinear polynomials $q^{(1)}, \ldots, q^{(\nu)}$, P wants to prove to V that

$$\bigcup_{j=1}^{\nu} \bigcup_{\boldsymbol{x} \in \{0,1\}^n} \left\{\!\!\left\{ p^{(j)}(\boldsymbol{x}) \right\}\!\!\right\} = \bigcup_{j=1}^{\nu} \bigcup_{\boldsymbol{x} \in \{0,1\}^n} \left\{\!\!\left\{ q^{(j)}(\boldsymbol{x}) \right\}\!\!\right\} \quad . \tag{1}$$

This is identical to the multiset-equality-check in Section 5.3.3, except that polynomials p and q have been split into ν parts. In the RW streaming setting this represents the fact they arrive in separate RW streams. Also, note that this is not the same as a *batch* multiset-equality-check where one would have ν separate equalities, and can be reduced to a sumcheck PIOP for a batch of sumprod polynomials Section 5.2.3.

This is formalized by the relation $\mathscr{R}_{\mathsf{SMC}}$, which contains tuples of the form

$$(\mathsf{i}_{\mathsf{SMC}}, \mathsf{x}_{\mathsf{SMC}}, \mathsf{w}_{\mathsf{SMC}}) = ((\mathbb{F}, n, \nu), ([\![p^{(1)}]\!], \ldots, [\![p^{(\nu)}]\!], [\![q^{(1)}]\!], \ldots, [\![q^{(\nu)}]\!]), (p^{(1)}, \ldots, p^{(\nu)}, q^{(1)}, \ldots, q^{(\nu)}))$$

such that Eq. (1) is satisfied. The following is a PIOP for $\mathscr{R}_{\mathsf{SMC}}$:

PIOP 9: SPLIT MULTISET-EQUALITY-CHECK

$\langle \mathsf{P}(\mathsf{i}_{\mathsf{SMC}}, \mathsf{x}_{\mathsf{SMC}}, \mathsf{w}_{\mathsf{SMC}}), \mathsf{V}(\mathsf{i}_{\mathsf{SMC}}, \mathsf{x}_{\mathsf{SMC}}) \rangle$:

1. V sends a random challenge $\alpha \in \mathbb{F}$ to P.
2. P and V engage in a sumcheck PIOP for rational functions for the claim "$\sum_{\boldsymbol{x} \in \{0,1\}^n} \sum_{i=1}^{\nu} (\alpha + p^{(i)}(\boldsymbol{x}))^{-1} - (\alpha + q^{(i)}(\boldsymbol{x}))^{-1} = 0$".

We describe a RW streaming prover that uses $O(\nu N)$ time and $O(\nu + \log N)$ random-access space to execute this PIOP:

Read-write Streaming Algorithm 12: SPLIT MULTISET-EQUALITY-CHECK

$\mathsf{P}(\mathsf{i}_{\mathsf{SMC}}, \mathsf{x}_{\mathsf{SMC}}, (\mathbf{R}_p^{(1)}(p^{(1)}), \ldots, \mathbf{R}_p^{(\nu)}(p^{(\nu)}), \mathbf{R}_q^{(1)}(q^{(1)}), \ldots, \mathbf{R}_q^{(\nu)}(q^{(\nu)})))$:

1. Receive a random challenge $\alpha \in \mathbb{F}$ from V.
2. Define function add: $\mathsf{add}(x) := x + \alpha$.
3. For i in $[1, \ldots, \nu]$:
4. $\quad \mathbf{S}_p^{(i)} \rightsquigarrow \mathsf{Map}(\mathsf{add}) \rightsquigarrow \mathbf{R}_p^{(i)}$.
5. $\quad \mathbf{S}_q^{(i)} \rightsquigarrow \mathsf{Map}(\mathsf{add}) \rightsquigarrow \mathbf{R}_q^{(i)}$.
6. Define $f_1(x_1, \ldots, x_\nu, y_1, \ldots, y_\nu) := \sum_{i=1}^{\nu} (-1)^{\nu-1} \prod_{j=1}^{\nu} x_j \cdot \prod_{j \neq i} y_j + \sum_{i=1}^{\nu} (-1)^{\nu} \prod_{j=1}^{\nu} y_j \cdot \prod_{j \neq i} x_j$.
7. Define $f_2(x_1, \ldots, x_\nu, y_1, \ldots, y_\nu) := \prod_{j=1}^{\nu} x_j \cdot y_j$.
8. Define $\mathsf{i} := (\mathbb{F}, n, 2\nu, f_1, f_2)$ and $\mathsf{x} := (\mathsf{x}_{\mathsf{SMC}}, 0)$ and $\mathsf{w} := (\mathbf{S}_p^{(1)}, \ldots, \mathbf{S}_p^{(\nu)}, \mathbf{S}_q^{(1)}, \ldots, \mathbf{S}_q^{(\nu)})$.
9. P and V invoke a RW streaming sumcheck PIOP for rational functions on $(\mathsf{i}, \mathsf{x}, \mathsf{w})$ as defined above.

The foregoing RW streaming prover is executing a sumcheck PIOP for the following rational function:

$$\sum_{j=1}^{\nu} \sum_{i=0}^{N-1} \frac{1}{p_i^{(j)} + \alpha} = \sum_{j=1}^{\nu} \sum_{i=0}^{N-1} \frac{1}{q_i^{(j)} + \alpha}$$

$$\iff \sum_{i=0}^{N-1} \frac{\sum_{k=1}^{\nu} \prod_{j \neq k} (p_i^{(j)} + \alpha)}{\prod_{j=1}^{\nu} (p_i^{(j)} + \alpha)} = \sum_{i=0}^{N-1} \frac{\sum_{k=1}^{\nu} \prod_{j \neq k} (q_i^{(j)} + \alpha)}{\prod_{j=1}^{\nu} (q_i^{(j)} + \alpha)}$$

At first glance, it looks like the numerator has ν terms, where each term has degree $\nu - 1$, and would therefore require $O(\nu^2)$ time to compute. However, in the sumcheck for rational functions, these terms can be computed in $O(\nu)$ time by dividing the product $\prod_{j=1}^{\nu} (p_i^{(j)} + \alpha)$ by $(p_i^{(k)} + \alpha)$ to get the kth term of the numerator.

Lemma A.4. *PIOP 9 and Algorithm 12 together comprise a read-write streaming PIOP for $\mathscr{R}_{\mathsf{SMC}}$ with the following properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(\nu N)$	$O(\nu + \log N)$	$O(\nu N)$	$O(\nu)$	$O(\nu N / \lvert\mathbb{F}\rvert)$	$O(\nu \log N)$	$O(\nu)$

Proof. All properties follow from the analysis of RW streaming sumcheck for (d, ℓ)-rational functions. □

A.3 Split permcheck

Given a split size ν, multilinear polynomials $p^{(1)}, \ldots, p^{(\nu)}, q^{(1)}, \ldots, q^{(\nu)}$, and permutations $\pi_1, \ldots, \pi^{(\nu)}$, a prover P of the split permcheck PIOP wants to prove to V that

$$p^{(i)}(\boldsymbol{x}) = q^{(i)}(\pi^{(i)}(\boldsymbol{x})) \tag{2}$$

for all $i \in [\nu]$ and $\boldsymbol{x} \in \{0,1\}^n$. This is formalized by the relation $\mathscr{R}_{\mathsf{SPC}}$, which contains tuples of the form

$$(\mathsf{i}_{\mathsf{SPC}}, \mathsf{x}_{\mathsf{SPC}}, \mathsf{w}_{\mathsf{SPC}}) = ((\mathbb{F}, n, \nu, \pi^{(1)}, \ldots, \pi^{(\nu)}), (\llbracket p^{(1)} \rrbracket, \ldots, \llbracket p^{(\nu)} \rrbracket, \llbracket q^{(1)} \rrbracket, \ldots, \llbracket q^{(\nu)} \rrbracket), \\ (p^{(1)}, \ldots, p^{(\nu)}, q^{(1)}, \ldots, q^{(\nu)}, \hat{\pi}^{(1)}, \ldots, \hat{\pi}^{(\nu)}))$$

such that Eq. (2) is satisfied where $\hat{\pi}^{(i)}(\boldsymbol{x}) = \mathsf{int}(\pi^{(i)}(\boldsymbol{x}))$ for all $i \in [\nu]$ and $\boldsymbol{x} \in \{0,1\}^n$. The PIOP for $\mathscr{R}_{\mathsf{SPC}}$ is nearly identical to the one described for the permcheck relation in Section 5.3.4, except that the polynomials p and q are split into ν parts. We therefore omit this PIOP for brevity, and present the RW streaming prover for it:

Read-write Streaming Algorithm 13: SPLIT PERMCHECK

$\mathsf{P}(\mathsf{i}_{\mathsf{SPC}}, \mathsf{x}_{\mathsf{SPC}}, (\mathbf{R}_p^{(1)}(p^{(1)}), \ldots, \mathbf{R}_p^{(\nu)}(p^{(\nu)}), \mathbf{R}_q^{(1)}(q^{(1)}), \ldots, \mathbf{R}_q^{(\nu)}(q^{(\nu)}), \mathbf{R}_\pi^{(1)}(\hat{\pi}^{(1)}), \ldots, \mathbf{R}_\pi^{(\nu)}(\hat{\pi}^{(\nu)}))):$

1. Receive a random challenge $\beta \in \mathbb{F}$ from V.
2. Define function add: $\mathsf{add}(x, y) := x + \beta \cdot y$.
3. For j in $[1, \ldots, \nu]$:
4. $\quad \mathbf{S}_p^{(j)} \rightsquigarrow \mathsf{Map}(\mathsf{add}) \rightsquigarrow \mathsf{Zip} \rightsquigarrow (\mathbf{R}_p^{(j)}, \mathbf{R}_\pi^{(j)})$.
5. $\quad \mathbf{S}_q^{(j)} \rightsquigarrow \mathsf{Map}(\mathsf{add}) \rightsquigarrow \mathsf{Zip} \rightsquigarrow (\mathbf{R}_q^{(j)}, [i]_{i=1}^n)$.
6. Define $\mathsf{i} := (\mathbb{F}, n, \nu)$
7. Define $\mathsf{x} := (\llbracket p^{(1)} \rrbracket, \ldots, \llbracket p^{(\nu)} \rrbracket, \llbracket q^{(1)} \rrbracket, \ldots, \llbracket q^{(\nu)} \rrbracket)$.
8. Define $\mathsf{w} := (\mathbf{R}_p^{(1)}(p^{(1)}), \ldots, \mathbf{R}_p^{(\nu)}(p^{(\nu)}), \mathbf{R}_q^{(1)}(q^{(1)}), \ldots, \mathbf{R}_q^{(\nu)}(q^{(\nu)}))$.
9. P and V invoke a RW streaming split multiset-equality-check PIOP on $(\mathsf{i}, \mathsf{x}, \mathsf{w})$ as defined above.

Lemma A.5. *PIOP 6 and Algorithm 13 together comprise a read-write streaming PIOP for $\mathscr{R}_{\mathsf{SPC}}$ with the following properties:*

prover time	prover space			PIOP properties		
	random-access	streaming	# streams	soundness error	communication	# queries
$O(\nu N)$	$O(\nu + \log N)$	$O(\nu N)$	$O(\nu)$	$O(\nu N/\lvert\mathbb{F}\rvert)$	$O(\nu \log N)$	$O(\nu)$

Proof. All properties follow from the analysis of RW streaming PIOP for split multiset-equality-check. □

Remark A.6 (oracles of smaller polynomials for concrete efficiency)**.** The key concrete advantage of the split multiset-equality-check and split permcheck over the vanilla ones (Algorithm 6 and Algorithm 7) is that the multilinear polynomial $\hat{f}$ sent by P at Step 4 in Algorithm 11, has $O(N)$ terms instead of $O(\nu N)$ terms, which is a dominating factor in the concrete efficiency of the protocol. This is at the cost of slightly higher communication ($O(\nu \log N)$ instead of $O(\log(\nu N))$) and more polynomial queries ($O(\nu)$ instead of $O(1)$).

A.4 Using split permcheck for the wiring constraint

Recall the HyperPlonk wiring constraint for binary circuits from Section 2.2. The circuit C is represented by three vectors $\boldsymbol{\ell}, \boldsymbol{r}, \boldsymbol{o} \in \mathbb{F}^N$ where $\boldsymbol{\ell}, \boldsymbol{r}$ are the left and right inputs to the gates, and $\boldsymbol{o}$ are the outputs of the gates. Furthermore, the wiring constraints of the circuit are represented by 2 permutation vectors $\boldsymbol{\pi}, \boldsymbol{\sigma} \in \mathbb{F}^N$ as follows: if the left input of gate i is the output of gate j, then $\pi_i = j$, and similarly, if the right input of gate i is the output of gate j then $\sigma_i = j$.

Let the multilinear extensions of these 5 vectors be $\hat{\ell}, \hat{r}, \hat{o}, \hat{\pi}, \hat{\sigma}$ respectively, and assume that P receives the stream of evaluations of these polynomials over the boolean hypercube in 5 separate RW streams. In order to prove that the wiring constraint is satisfied, P needs to do to prove that $\hat{\ell}(\boldsymbol{x}) = \hat{o}(\hat{\pi}(\boldsymbol{x}))$ and $\hat{r}(\boldsymbol{x}) = \hat{o}(\hat{\sigma}(\boldsymbol{x}))$

for all $\boldsymbol{x} \in \{0,1\}^n$. Therefore the HyperPlonk PIOP calls the split permcheck PIOP with the following arguments:

$$((\mathbb{F}, n, 2, \hat{\pi}, \hat{\sigma}), ([\![\hat{\ell}]\!], [\![\hat{r}]\!], [\![\hat{o}]\!], [\![\hat{o}]\!]), (\mathbf{R}_1(\hat{\ell}), \mathbf{R}_2(\hat{r}), \mathbf{R}_3(\hat{o}), \mathbf{R}_4(\hat{o}), \mathbf{R}_5(\hat{\pi}), \mathbf{R}_6(\hat{\sigma}))) \ ,$$

which improves the concrete efficiency of the wiring constraint check by approximately a factor of 2.

B Generalized inner product arguments

A core building block for many polynomial commitment schemes [BCCGP16; WTSTW18; Lee21; BMMTV21; BGH19; BCMS20] is an inner product argument (IPA) [BCCGP16; BBBPWM18] or its generalization [LMR19; BMMTV21].

As a stepping stone to constructing read-write streaming variants of these polynomial commitments, in this section we construct a read-write streaming prover that requires only logarithmic random access space for the generalized inner product argument (GIPA) of [BMMTV21]. At a high level, the latter is an argument that allows a prover to convince a verifier that $\langle \boldsymbol{x}, \boldsymbol{w} \rangle = v$, where $\boldsymbol{x}, \boldsymbol{w}$ are committed vectors, and v is the claimed result of a suitable inner-product between $\boldsymbol{x}$ and $\boldsymbol{w}$.

B.1 Commitment schemes

We recall the definition of doubly-homomorphic and inner-product commitment schemes [BMMTV21], along with several constructions of these that will appear in our instantiations of GIPA.

Definition B.1 (commitment scheme)**.** *A commitment scheme* $\mathsf{CM} = (\mathsf{Setup}, \mathsf{Commit})$ *over a universe of key spaces* $\{\mathcal{K}_i\}_{i\in\mathbb{N}}$ *message spaces* $\{\mathcal{M}_i\}_{i\in\mathbb{N}}$ *enables a party to generate a (perfectly) hiding and (computationally) binding commitment to a given message* $m \in \mathcal{M}$*.*

- Setup: *on input a security parameter and a description of the message space* $\mathcal{M}_i$*,* $\mathsf{CM.Setup}$ *samples a commitment key* $\mathsf{ck} \in \mathcal{K}_i$*.*
- Commit: *on input public parameters* ck*, message* $m \in \mathcal{M}_i$*, and randomness* r*,* $\mathsf{CM.Commit}$ *outputs a commitment* C_m *to* m*.*

Definition B.2 (Doubly homomorphic commitment)**.** *Let* $(\mathcal{K}, +)$*,* $(\mathcal{M}, +)$ *and* $(\mathsf{Image}(\mathsf{CM.Commit}), +)$ *define abelian groups. A commitment scheme* $\mathsf{CM} = (\mathsf{Setup}, \mathsf{Commit})$ *is doubly homomorphic if for all* $\mathsf{ck}_1, \mathsf{ck}_2 \in \mathcal{K}$ *and* $m_1, m_2 \in \mathcal{M}$ *we have*

1. $\mathsf{Commit}(\mathsf{ck}_1, m_1) + \mathsf{Commit}(\mathsf{ck}_2, m_1) = \mathsf{Commit}(\mathsf{ck}_1 + \mathsf{ck}_2, m_1)$
2. $\mathsf{Commit}(\mathsf{ck}_1, m_1) + \mathsf{Commit}(\mathsf{ck}_1, m_2) = \mathsf{Commit}(\mathsf{ck}_1, m_1 + m_2)$

In particular the above also implies $\mathsf{Commit}(\alpha \cdot \mathsf{ck}_1, m_1) = \alpha \cdot \mathsf{Commit}(\mathsf{ck}_1, m_1)$ *and* $\mathsf{Commit}(\mathsf{ck}_1, \alpha \cdot m_1) = \alpha \cdot \mathsf{Commit}(\mathsf{ck}_1, m_1)$*.*

Definition B.3 (Inner product map)**.** *A map* $\circledast : \mathcal{M}_1 \times \mathcal{M}_2 \to \mathcal{M}_3$ *from two groups of prime order* p *to a third group of order* p *is an inner product map if for all* $a, b \in \mathcal{M}_1$ *and* $c, d \in \mathcal{M}_2$ *we have that*

$$(a + b) \circledast (c + d) = a \circledast c + a \circledast d + b \circledast c + b \circledast d$$

Given an inner product $\circledast$ *between groups we define the inner product between vector spaces* $\langle \cdot, \cdot \rangle_{\circledast} : \mathcal{M}_1^N \times \mathcal{M}_2^N \to \mathcal{M}_3$ *to be* $\langle \boldsymbol{a}, \boldsymbol{b} \rangle_{\circledast} := \sum_{i=1}^{N} a_i \circledast b_i$*.*

Definition B.4 (Inner product commitment)**.** *Let* CM *be a doubly homomorphic commitment scheme with message space* $\mathcal{M} = \mathcal{M}_1^m \times \mathcal{M}_2^m \times \mathcal{M}_3$ *and key space* $\mathcal{K} = \mathcal{K}_1^m \times \mathcal{K}_2^m \times \mathcal{K}_3$ *defined for all* $m \in [2^j]_{j\in\mathbb{N}}$*, where* $|\mathcal{M}_i| = |\mathcal{K}_i| = p$ *is prime for* $i \in [3]$*. Let* $\circledast : \mathcal{M}_1 \times \mathcal{M}_2 \to \mathcal{M}_3$ *be an inner product map. We call*

$(\mathsf{CM}, \circledast)$ *an inner product commitment if there exist efficient deterministic functions* Split *and* Collapse *such that for all* $m \in [2^j]_{j\in\mathbb{N}}$, $M \in \mathcal{M}$, *and* $\mathsf{ck} \in \mathcal{K}$, *if* $\mathsf{Split}(\mathsf{ck}_i) = (\mathsf{ck}_i^L, \mathsf{ck}_i^R)$ *for* $i \in [2]$ *it holds that:*

$$\mathsf{Collapse}\left(\mathsf{CM}\left(\begin{array}{c|c} \mathsf{ck}_1 & M_1 \;||\; M_1 \\ \mathsf{ck}_2 & M_2 \;||\; M_2 \\ \mathsf{ck}_3 & M_3 \end{array}\right)\right) = \mathsf{CM}\left(\begin{array}{c|c} \mathsf{ck}_1^L + \mathsf{ck}_1^R & M_1 \\ \mathsf{ck}_2^L + \mathsf{ck}_2^R & M_2 \\ \mathsf{ck}_3 & M_3 \end{array}\right)$$

The above requirement is referred to as the collapsing property.

Constructions of inner-product commitment schemes. We recall the Pedersen commitment scheme CM_P which has message space $\{\mathbb{F}^N\}_{N\in\mathbb{N}}$, and the AFGHO commitment schemes [AFGHO16] CM_1 and CM_2, which have message spaces $\{\mathbb{G}_1^N\}_{N\in\mathbb{N}}$ and $\{\mathbb{G}_2^N\}_{N\in\mathbb{N}}$ respectively. For notational convenience, in this section we write $\bar{1}$ to denote the subscript 2, and write $\bar{2}$ to denote the subscript 1.

$\mathsf{CM}_P.\mathsf{Setup}(1^\lambda, N) \to \mathsf{ck}_P$: 1. Output $\mathsf{ck}_P := \mathbf{\Gamma}_1 \xleftarrow{\$} \mathbb{G}_1^N$.	$\mathsf{CM}_P.\mathsf{Commit}(\mathsf{ck}_P, \boldsymbol{x}) \to C_{\boldsymbol{x}}$: 1. Parse ck_P as $\mathbf{\Gamma}_1$. 2. Output $\langle \boldsymbol{x}, \mathbf{\Gamma}_1 \rangle_{\mathbb{G}}$.
$\mathsf{CM}_i.\mathsf{Setup}(1^\lambda, N) \to (\mathbf{\Gamma}_i)$: 1. Output $\mathsf{ck}_i := \mathbf{\Gamma}_{\bar{i}} \xleftarrow{\$} \mathbb{G}_{\bar{i}}^N$.	$\mathsf{CM}_i.\mathsf{Commit}(\mathsf{ck}_i, \boldsymbol{X}) \to C_{\boldsymbol{X}}$: 1. Parse ck_i as $\mathbf{\Gamma}_{\bar{i}}$. 2. If $i = 1$: output $\langle \boldsymbol{X}, \mathbf{\Gamma}_2 \rangle_e$. 3. If $i = 2$: output $\langle \mathbf{\Gamma}_1, \boldsymbol{X} \rangle_e$.

Both the Pedersen and AFGHO commitments are doubly homomorphic. Additionally, the Pedersen commitment scheme and the AFGHO commitment scheme are secure under the SXDH assumption ([AFGHO16]).

B.2 Generalized inner product arguments

We now recall the definition of generalized inner product arguments (GIPA) [BMMTV21], beginning with the relation $\mathscr{R}_{\mathsf{GIP}}$ proven by these arguments.

Definition B.5. *The indexed relation* $\mathscr{R}_{\mathsf{GIP}}$ *is the set of triples*

$$\begin{pmatrix} \mathsf{i} \\ \mathsf{x} \\ \mathsf{w} \end{pmatrix} = \begin{pmatrix} (N, \mathsf{CM}, \circledast, \mathbf{\Gamma}) \\ D \\ (\boldsymbol{V}, \boldsymbol{X}) \end{pmatrix}$$

where $N \in \mathbb{N}$ *is a power-of-two,* $(\mathsf{CM}, \circledast)$ *is an inner product commitment with key space* $\mathcal{K}$, *message space* $\mathcal{M}$, *and* $\mathbf{\Gamma} \in \mathcal{K}$, $(\boldsymbol{V}, \boldsymbol{X}, \langle \boldsymbol{V}, \boldsymbol{X} \rangle_{\circledast}) \in \mathcal{M}$ *such that* $D = \mathsf{CM}.\mathsf{Commit}(\mathbf{\Gamma}, (\boldsymbol{V}, \boldsymbol{X}, \langle \boldsymbol{V}, \boldsymbol{X} \rangle_{\circledast}))$.

B.2.1 GIPA.Reduce

In this section we present an interactive reduction of knowledge [KP23] GIPA.Reduce that reduces an instance of $\mathscr{R}_{\mathsf{GIP}}$ into a related instance of half the length using a random verifier challenge. That is, given an index $\mathsf{i} = (2^n, \mathsf{CM}, \circledast, \mathbf{\Gamma} = (\mathbf{\Gamma}_1, \mathbf{\Gamma}_2, \Gamma_3))$, GIPA.Reduce reduces the problem of checking if a tuple $(\mathsf{i}, \mathsf{x}, \mathsf{w}) \in \mathscr{R}_{\mathsf{GIP}}$ to the problem of checking if a tuple $(\mathsf{i}', \mathsf{x}', \mathsf{w}') \in \mathscr{R}_{\mathsf{GIP}}$, where $\mathsf{i}' = (2^{n-1}, \mathsf{CM}, \circledast, \mathbf{\Gamma}' = (\mathbf{\Gamma}'_1, \mathbf{\Gamma}'_2, \Gamma_3))$ with $\mathbf{\Gamma}' \in \mathcal{K}_1^{2^{n-1}} \times \mathcal{K}_2^{2^{n-1}} \times \mathcal{K}_3$.

We define two useful helper functions. The first, ExpandEven, takes as input a vector $\boldsymbol{A}$ of length N and outputs a vector $\boldsymbol{B}$ of length $2N$ whose $2i$-th entry contains the i-th entry of $\boldsymbol{A}$; the rest of $\boldsymbol{B}$'s entries are zero. The second, ExpandOdd, is similar, but sets the $2i-1$-th entry of $\boldsymbol{B}$ to contain the i-th entry of $\boldsymbol{A}$.

The full GIPA.Reduce construction is presented below.

GIPA.Reduce

$\langle \mathsf{P}(\mathsf{i},\mathsf{x},\mathsf{w}), \mathsf{V}(\mathsf{i},\mathsf{x})\rangle$:

Parse: $\mathsf{i} = (2^n, \mathsf{CM}, \circledast, \boldsymbol{\Gamma} = (\boldsymbol{\Gamma}_1, \boldsymbol{\Gamma}_2, \Gamma_3))$, $\mathsf{x} = D$ and $\mathsf{w} = (\boldsymbol{V}, \boldsymbol{X})$.

1. $\mathcal{P}$ computes the cross-inner-products $E_+ := \langle \boldsymbol{V}_O, \boldsymbol{X}_E\rangle_{\circledast}$ and $E_- := \langle \boldsymbol{V}_E, \boldsymbol{X}_O\rangle_{\circledast}$.
2. $\mathcal{P}$ computes the cross-commitments
   (a) $D_+ \leftarrow \mathsf{CM.Commit}(\boldsymbol{\Gamma}, (\mathsf{ExpandOdd}(\boldsymbol{V}_O), \mathsf{ExpandEven}(\boldsymbol{X}_E), E_+))$,
   (b) $D_- \leftarrow \mathsf{CM.Commit}(\boldsymbol{\Gamma}, (\mathsf{ExpandOdd}(\boldsymbol{V}_E), \mathsf{ExpandEven}(\boldsymbol{X}_O), E_-))$.
3. $\mathcal{P}$ sends D_+, D_- to $\mathcal{V}$.
4. $\mathcal{V}$ samples $\alpha \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$.
5. $\mathcal{P}$ and $\mathcal{V}$ fold commitment keys and commitment:
$$\begin{aligned}\boldsymbol{\Gamma}'_1 &:= \alpha\boldsymbol{\Gamma}_{2E} + \boldsymbol{\Gamma}_{2O},\\ \boldsymbol{\Gamma}'_2 &:= \alpha^{-1}\boldsymbol{\Gamma}_{2E} + \boldsymbol{\Gamma}_{2O},\\ D' &:= D + \alpha D_+ + \alpha^{-1} D_-.\end{aligned}$$
6. $\mathcal{P}$ folds the witness vectors:
$$\begin{aligned}\boldsymbol{V}' &:= \alpha^{-1}\boldsymbol{v}_E + \boldsymbol{v}_O,\\ \boldsymbol{X}' &:= \alpha\boldsymbol{X}_E + \boldsymbol{X}_O.\end{aligned}$$
7. Define $\mathsf{i}' := (2^{n-1}, \mathsf{CM.Commit}, \circledast, \boldsymbol{\Gamma}' := (\boldsymbol{\Gamma}'_1, \boldsymbol{\Gamma}'_2, \Gamma_3))$, $\mathsf{x}' := D'$ and $\mathsf{w}' := (\boldsymbol{V}', \boldsymbol{X}')$.
8. $\mathcal{P}$ receives output $(\mathsf{i}', \mathsf{x}', \mathsf{w}')$ and $\mathcal{V}$ receives output $(\mathsf{i}', \mathsf{x}')$.

Lemma B.6 ([BMMTV21, Theorem 1])**.** *Let* $\mathsf{i} = (2^n, \mathsf{CM}, \circledast, \boldsymbol{\Gamma} = (\boldsymbol{\Gamma}_1, \boldsymbol{\Gamma}_2, \Gamma_3))$ *be an index for* $\mathscr{R}_{\mathsf{GIP}}$*. Then the following is an interactive argument of knowledge for proving that* $(\mathsf{i}, \mathsf{x}, \mathsf{w}) \in \mathscr{R}_{\mathsf{GIP}}$*:*

1. *Prover and verifier invoke* GIPA.Reduce*:* $(\mathsf{i}', \mathsf{x}', \mathsf{w}') \leftarrow \mathsf{GIPA.Reduce}(\langle \mathsf{P}(\mathsf{i}, \mathsf{x}, \mathsf{w}), \mathsf{V}(\mathsf{i}, \mathsf{x})\rangle)$.
2. *Prover sends* w' *to verifier.*
3. *Verifier accepts if and only if* $(\mathsf{i}', \mathsf{x}', \mathsf{w}') \in \mathscr{R}_{\mathsf{GIP}}$.

MSB folding security implies LSB folding security. The foregoing reduction deviates from the one in [BMMTV21] as it folds the even and odd parts of vectors together as $\boldsymbol{A}' := \alpha\boldsymbol{A}_E + \boldsymbol{A}_O$ (LSB folding), instead of the left and right parts ($\boldsymbol{A}' := \alpha\boldsymbol{A}_L + \boldsymbol{A}_R$ (MSB folding)). We use LSB folding because it leads to more efficient read-write streaming prover algorithms that avoid intermediate streams, and because it simplifies exposition. This choice does not affect completeness or knowledge-soundness; completeness follows from inspection, while knowledge-soundness follows from the following lemma implicit in the work of Block et al. [BHRRS20]:

Lemma B.7. *Given a polynomial-time extractor that extracts a witness* $\boldsymbol{X}$ *for statements of the form* $((2^n, \boldsymbol{\Gamma}), D, (\boldsymbol{V}, \boldsymbol{X}))$*, we can construct a polynomial-time extractor that extracts a witness for statements of the form* $((2^n, \pi(\boldsymbol{\Gamma})), D, (\pi(\boldsymbol{V}), \pi(\boldsymbol{X})))$*, where* π *is an efficiently computable and invertible permutation.*

Knowledge-soundness follows by fixing π to be the following permutation:

$$\pi(X_i) := \begin{cases} X_{2(i-1)+1} & \text{if } i \le N/2,\\ X_{2(i-(N/2))} & \text{if } i > N/2.\end{cases}$$

B.2.2 Read-write streaming prover for GIPA.Reduce

In this section we present a streaming prover for GIPA.Reduce. We rely on the fact that the algorithms CM.Commit, ExpandOdd, and ExpandEven have efficient read-write streaming versions; this is true for all relevant instantiations of CM.Commit, since the latter is usually an inner product between the commitment key and the message. This in turn implies that Steps 2a and 2b of GIPA.Reduce can be performed in a read-write streaming manner; we call the resulting algorithm CrossCommit. We generalize the read-write streaming algorithm InnerProd to $\mathsf{InnerProd}_{\circledast}$ that computes an arbitrary inner product $\langle \cdot, \cdot \rangle_{\circledast}$ instead of the standard scalar inner product.

We are now ready to present our read-write streaming prover for GIPA.Reduce.

Read-write Streaming Algorithm 14: Prover for GIPA.REDUCE

$\mathsf{P}(\mathsf{i}, \mathsf{x}, \mathsf{w})$:

Parse: $\mathsf{i} = (2^n, \mathsf{CM}, \circledast, (\mathbf{R}_{\Gamma 1}(\boldsymbol{\Gamma}_1), \mathbf{R}_{\Gamma 2}(\boldsymbol{\Gamma}_2), \Gamma_3))$, $\mathsf{x} = D$ and $\mathsf{w} = (\mathbf{R}_V(\boldsymbol{V}), \mathbf{R}_X(\boldsymbol{X}))$.

1. Split commitment keys and witness vectors:
   - $(\mathbf{R}_{\Gamma 1E}, \mathbf{R}_{\Gamma 1O}) \leftarrow \mathsf{SplitEO}(\mathbf{R}_{\Gamma 1})$,
   - $(\mathbf{R}_{\Gamma 2E}, \mathbf{R}_{\Gamma 2O}) \leftarrow \mathsf{SplitEO}(\mathbf{R}_{\Gamma 2})$,
   - $(\mathbf{R}_{VE}, \mathbf{R}_{VO}) \leftarrow \mathsf{SplitEO}(\mathbf{R}_V)$, and
   - $(\mathbf{R}_{XE}, \mathbf{R}_{XO}) \leftarrow \mathsf{SplitEO}(\mathbf{R}_X)$.
2. Compute $E_+ \leftarrow \mathsf{InnerProd}_{\circledast}(\mathbf{R}_{VO}, \mathbf{R}_{XE})$ and $E_- \leftarrow \mathsf{InnerProd}_{\circledast}(\mathbf{R}_{VE}, \mathbf{R}_{XO})$.
3. Compute cross commitments:
   - $D_+ \leftarrow \mathsf{CrossCommit}((\mathbf{R}_{\Gamma 1}, \mathbf{R}_{\Gamma 2}, \Gamma_3), \mathbf{R}_{VO}, \mathbf{R}_{XE}, E_+)$, and
   - $D_- \leftarrow \mathsf{CrossCommit}((\mathbf{R}_{\Gamma 1}, \mathbf{R}_{\Gamma 2}, \Gamma_3), \mathbf{R}_{VE}, \mathbf{R}_{XO}, E_-)$.
4. Send D_+, D_- to $\mathcal{V}$.
5. Receive $\alpha \xleftarrow{\$} \mathbb{F}$ from $\mathcal{V}$.
6. Fold the commitment keys: $\mathsf{RWFoldInPlace}(\alpha, 1, \mathbf{R}_{\Gamma 1})$ and $\mathsf{RWFoldInPlace}(\alpha^{-1}, 1, \mathbf{R}_{\Gamma 2})$.
7. Fold the vectors: $\mathsf{RWFoldInPlace}(\alpha^{-1}, 1, \mathbf{R}_V)$ and $\mathsf{RWFoldInPlace}(\alpha, 1, \mathbf{R}_X)$.
8. Set $D' := D + \alpha D_+ + \alpha^{-1} D_-$.
9. Define $\mathsf{i}' := (2^{n-1}, \mathsf{CM}, \circledast, (\mathbf{R}_{\Gamma 1}, \mathbf{R}_{\Gamma 2}, \Gamma_3))$, $\mathsf{x}' := D'$ and $\mathsf{w}' := (\mathbf{R}_V, \mathbf{R}_X)$.
10. Store $(\mathsf{i}', \mathsf{x}', \mathsf{w}')$ for future invocations.

Lemma B.8. GIPA.Reduce *and Algorithm 14 together define a read-write streaming reduction of knowledge with the following properties:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(N)$	$O(1)$

Proof. The proof is similar to that of Lemma 6.1. □

B.2.3 The full GIPA protocol

We now present the full argument for $\mathscr{R}_{\mathsf{GIP}}$ that applies GIPA.Reduce iteratively to shrink the size of the instance to 1, and then directly checks this instance.

GIPA

$\langle \mathsf{P}(\mathsf{i}, \mathsf{x}, \mathsf{w}), \mathsf{V}(\mathsf{i}, \mathsf{x}) \rangle$:

Parse: $\mathsf{i} = (2^n, \mathsf{CM}, \circledast, \boldsymbol{\Gamma} = (\boldsymbol{\Gamma}_1, \boldsymbol{\Gamma}_2, \Gamma_3))$, $\mathsf{x} = D$ and $\mathsf{w} = (\boldsymbol{V}, \boldsymbol{X})$.

1. Define $\mathsf{i}_1 := \mathsf{i}$, $\mathsf{x}_1 := \mathsf{x}$ and $\mathsf{w}_1 := \mathsf{w}$.
2. For i in $[1, \ldots, n]$:
3. $\quad (\mathsf{i}_{i+1}, \mathsf{x}_{i+1}, \mathsf{w}_{i+1}) \leftarrow \mathsf{GIPA.Reduce}(\langle \mathsf{P}(\mathsf{i}_i, \mathsf{x}_i, \mathsf{w}_i), \mathsf{V}(\mathsf{i}_i, \mathsf{x}_i) \rangle)$.
4. Parse $\mathsf{i}_{n+1} = (1, \mathsf{CM.Commit}, \circledast, \Gamma' = (\Gamma'_1, \Gamma'_2, \Gamma'_3))$, $\mathsf{x}_{n+1} = D'$ and $\mathsf{w}_{n+1} = (V', X')$.
5. $\mathcal{P}$ sends (V', X') to $\mathcal{V}$.
6. $\mathcal{V}$ accepts if $D' = \mathsf{CM.Commit}((\Gamma'_1, \Gamma'_2, \Gamma'_3), (V', X', \langle V', X' \rangle_{\circledast}))$.

Theorem B.9. GIPA *is a read-write streaming interactive argument of knowledge for* $\mathscr{R}_{\mathsf{GIP}}$ *with the following properties:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(N)$	$O(\log N)$

Proof. Completeness and knowledge soundness follow from [BMMTV21, Theorem 1]. The existence and efficiency of a read-write streaming prover for GIPA follows from Lemma B.8. □

B.3 The MIPP Protocol

A key building block of the polynomial commitment schemes from [BMMTV21] and [Lee21] is an interactive argument for the MIPP relation from [BMMTV21], which allows a prover to convince a verifier that the multiscalar multiplication $\langle \boldsymbol{v}, \boldsymbol{X} \rangle_{\mathbb{G}} = E$, where $\boldsymbol{X} \in \mathbb{G}^N$ is a private group vector committed to in the AFGHO commitment D and $\boldsymbol{v} \in \mathbb{F}^N$ is a public field vector shared by both the prover and verifier.

Formally, the Multi-exponentiation Inner Product (MIPP) relation $\mathscr{R}_{\mathsf{MIPP}}$ is defined as follows:

Definition B.10 (MIPP relation)**.** *The indexed relation* $\mathscr{R}_{\mathsf{MIPP}}$ *is the set of triples*

$$\begin{pmatrix} \mathsf{i}, \\ \mathsf{x}, \\ \mathsf{w} \end{pmatrix} = \begin{pmatrix} (N, \boldsymbol{\Gamma}_2), \\ (D, \boldsymbol{v}, E), \\ \boldsymbol{X} \end{pmatrix}$$

where N *is an integer,* $\boldsymbol{\Gamma}_2 \in \mathbb{G}_2^N$, $D \in \mathbb{G}_T$, $\boldsymbol{v} \in \mathbb{F}^N$, $E \in \mathbb{G}_1$ *and* $\boldsymbol{X} \in \mathbb{G}_1^N$ *such that*

$$D = \mathsf{CM}_1(\boldsymbol{\Gamma}_2, \boldsymbol{X}), \quad E = \langle \boldsymbol{v}, \boldsymbol{X} \rangle_{\mathbb{G}}$$

Notice that $\mathscr{R}_{\mathsf{MIPP}}$ is a specific instantiation of $\mathscr{R}_{\mathsf{GIP}}$, with $\circledast : \mathbb{F} \times \mathbb{G}_1 \to \mathbb{G}_1$ defined to be $a \circledast B = a \cdot B$ and the inner product commitment $\mathsf{CM}_{\mathsf{MIPP}}.\mathsf{Commit}$, given commitment key $\boldsymbol{\Gamma}_2 \in \mathbb{G}_2^N$, defined to be:

$$\mathsf{CM}_{\mathsf{MIPP}}.\mathsf{Commit}((\perp, \boldsymbol{\Gamma}_2, \perp), (\boldsymbol{a}, \boldsymbol{B}, E)) = (\mathsf{CM}_1.\mathsf{Commit}(\boldsymbol{\Gamma}_2, \boldsymbol{a}, \boldsymbol{B}), E).$$

The following lemma thus follows naturally from Theorem B.9.

Lemma B.11 (Corollary of Theorem B.9)**.** MIPP *is a read-write streaming interactive argument of knowledge for* $\mathscr{R}_{\mathsf{MIPP}}$ *with the following properties:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(N)$	$O(\log N)$

Sublinear verification. Bunz et al. [BMMTV21] describe an optimization that reduces the GIPA verifier's time from $O(N)$ to $O(\log N)$ by relying on a structured commitment key. Roughly, in the GIPA protocol, the verifier's work can be rewritten so that at each round, instead of folding the commitment keys, the verifier only needs to do a small amount of work to check consistency of the folded commitments; then, at the end of the protocol, the verifier needs to check the opening of a single N-sized commitment derived form its challenges. Bunz et al. show that this commitment can be viewed as a *polynomial* commitment to a structured polynomial, and leverage the KZG polynomial commitment scheme to allow the prover to *prove* correct commitment opening. The prover's work here consists of generating, committing to, and opening this polynomial commitment, and all steps can be done in a read-write streaming manner (and some even in a read-only streaming manner [BCHO22]).

For the overall MIPP verifier to run in $O(\log N)$ time, in addition to the foregoing GIPA optimization, the verifier also needs to be able to succinctly operate over the vector $\boldsymbol{v}$. This is indeed the case in the applications we consider, namely polynomial evaluation, where $\boldsymbol{v}$ is derived from an evaluation point for a multilinear polynomial.

B.4 The FIP protocol

In this section, we present another important building block of the polynomial commitment schemes from [BMMTV21] and [Lee21], the FIP protocol, which at a high level allows a prover to convince a verifier of the multiscalar multiplication $\langle \boldsymbol{v}, \boldsymbol{y} \rangle_{\mathbb{G}} = e$, where $\boldsymbol{y} \in \mathbb{F}^N$ is a private field vector committed to in the Pedersen commitment D and $\boldsymbol{v} \in \mathbb{F}^N$ is a public field vector shared by both the prover and verifier.

Formally, the Field Inner Product (FIP) relation $\mathscr{R}_{\mathsf{FIP}}$ is defined analogously to $\mathscr{R}_{\mathsf{MIPP}}$ as follows:

Definition B.12 (FIP relation)**.** *The indexed relation $\mathscr{R}_{\mathsf{FIP}}$ is the set of triples*

$$\begin{pmatrix} \mathsf{i}, \\ \mathsf{x}, \\ \mathsf{w} \end{pmatrix} = \begin{pmatrix} (N, \boldsymbol{\Gamma}_1), \\ (D, \boldsymbol{v}, e), \\ \boldsymbol{y} \end{pmatrix}$$

where N is an integer, $\boldsymbol{\Gamma}_1 \in \mathbb{G}_1^N$, $D \in \mathbb{G}_1$, $\boldsymbol{v} \in \mathbb{F}^N$, $e \in \mathbb{F}$ and $\boldsymbol{y} \in \mathbb{F}^N$ such that

$$D = \mathsf{CM}_P(\boldsymbol{\Gamma}_1, \boldsymbol{y}), \quad e = \langle \boldsymbol{v}, \boldsymbol{y} \rangle_{\mathbb{F}}$$

Notice that $\mathscr{R}_{\mathsf{FIP}}$ is a specific instantiation of $\mathscr{R}_{\mathsf{GIP}}$, with $\circledast : \mathbb{F} \times \mathbb{F} \to \mathbb{F}$ defined to be $a \circledast b = a \cdot b$ and the inner product commitment $\mathsf{CM}_{\mathsf{FIP}}.\mathsf{Commit}$, given commitment key $\boldsymbol{\Gamma}_1 \in \mathbb{G}_1^N$, defined to be:

$$\mathsf{CM}_{\mathsf{FIP}}.\mathsf{Commit}((\boldsymbol{\Gamma}_1, \perp, \perp), (\boldsymbol{a}, \boldsymbol{b}, e)) = (\mathsf{CM}_P.\mathsf{Commit}(\boldsymbol{\Gamma}_1, \boldsymbol{b}), \boldsymbol{a}, E).$$

The following lemma thus follows naturally from Theorem B.9.

Theorem B.13 (Corollary of Theorem B.9)**.** FIP *is a read-write streaming interactive argument of knowledge for $\mathscr{R}_{\mathsf{FIP}}$ with the following properties:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(N)$	$O(\log N)$

A similar verifier efficiency optimization as in the MIPP protocol can be applied to the FIP protocol; we refer the reader to Bünz et al. [BMMTV21] for details.

C Constructing polynomial commitment schemes with square-root SRS

Wahby et al. [WTSTW18] proposed a novel recipe for constructing polynomial commitment schemes using inner product arguments, where the size of the commitment key is sublinear in the size of the polynomial being committed to. This blueprint is also used in the PC scheme of Bünz et al. [BMMTV21] and was formalized using the notion of vector-matrix-vector product arguments by Dory [Lee21]. The blueprint proceeds by viewing an n-variate multilinear polynomial $p(\boldsymbol{X})$, represented by its evaluation over the boolean hypercube $\boldsymbol{p}$, as a matrix $\mathbf{M} \in \mathbb{F}^{\sqrt{N} \times \sqrt{N}}$ defined as follows:

$$M_{ij} := p_k \text{ for } k = i \cdot 2^m + j.$$

where $N := 2^n$ and $m := n/2$. Without loss of generality, we can assume that for the n-variate multilinear polynomials we consider, n is even. This is because if n is odd, we can just add a 'dummy' variable that is not included in the polynomial, incurring a cost overhead of at most a multiplicative factor of 2.[24]

Their key observation is that evaluating $p(X_1, \ldots, X_n)$ at a point $\boldsymbol{z} = (z_1, z_2, \ldots, z_n)$ is equivalent to computing the vector-matrix-vector (VMV) product $\boldsymbol{\ell}^\top \mathbf{M} \boldsymbol{r}$, where $\boldsymbol{\ell} := [\mathrm{eq}(\boldsymbol{z}_L, \mathsf{bin}_n(i))]_{i=0}^{\sqrt{N}-1} = \otimes_{i=1}^{m}(1 - z_i, z_i)$ and $\boldsymbol{r} := [\mathrm{eq}(\boldsymbol{z}_R, \mathsf{bin}_n(i))]_{i=0}^{\sqrt{N}-1} = \otimes_{i=m+1}^{n}(1 - z_i, z_i)$. This can be done via 'vector-matrix-vector product' arguments.

VMV arguments. A vector-matrix-vector product (VMV) argument, first defined in [Lee21][25], is a tuple of algorithms $\mathsf{VMV} = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Eval})$ with the following syntax.

- $\mathsf{VMV.Setup}(1^\lambda, N) \to (\mathsf{ck}, \mathsf{vk})$. On input a security parameter λ (in unary), and a maximum dimension bound $N \in \mathbb{N}$, $\mathsf{VMV.Setup}$ samples committer and verifier keys ck and vk.
- $\mathsf{VMV.Commit}(\mathsf{ck}, \mathbf{M}) \to C_{\mathbf{M}}$. On input ck and a matrix $\mathbf{M} \in \mathbb{F}^{\sqrt{N} \times \sqrt{N}}$, $\mathsf{VMV.Commit}$ outputs a commitment $C_{\mathbf{M}}$ to $\mathbf{M}$.
- $\mathsf{VMV.Eval}(\langle \mathcal{P}(\mathsf{ck}, (C_{\mathbf{M}}, \boldsymbol{\ell}, \boldsymbol{r}, y), \mathbf{M}), \mathcal{V}(\mathsf{vk}, (C_{\mathbf{M}}, \boldsymbol{\ell}, \boldsymbol{r}, y)) \rangle)$. $\mathcal{P}$ and $\mathcal{V}$ engage in an interactive protocol that convinces $\mathcal{V}$ that $C_{\mathbf{M}}$ commits to a matrix $\mathbf{M}$ satisfying the claim "$\boldsymbol{\ell}^\top \cdot \mathbf{M} \cdot \boldsymbol{r} = y$".

A vector-matrix-vector product argument VMV must satisfy completeness and extractability properties analogous to those of a polynomial commitment scheme. We show how to use a VMV argument to obtain a PC scheme in the next section, but first we make a comment about our presentation for the rest of the paper.

Presentation. For the subsequent polynomial commitment schemes, for ease of exposition we present the evaluation protocol as an interactive protocol $\mathsf{PC.Eval}$. The standard non-interactive notions $\mathsf{PC.Open}$ and $\mathsf{PC.Check}$ correspond to the prover and verifier respectively of the non-interactive version of $\mathsf{PC.Eval}$ obtained via the Fiat-Shamir transform [FS86].

C.1 Constructing VMV arguments

As a prerequisite, we define the following instantiations of $\mathscr{R}_{\mathsf{GIP}}$ and their associated arguments:

- $\mathscr{R}_{\mathsf{in}}$ is $\mathscr{R}_{\mathsf{GIP}}$ specialized to a commitment scheme $\mathsf{CM}_{\mathsf{in}}$ with message space $\mathbb{F}^{\sqrt{N}}$ and commitment space $\mathbb{G}_1$, and inner product map $\circledast : \mathbb{F} \times \mathbb{F} \to \mathbb{F}$ defined to be $a \circledast b = a \cdot b$. $\mathsf{Arg}_{\mathsf{in}}$ is an interactive argument for $\mathscr{R}_{\mathsf{in}}$.
- $\mathscr{R}_{\mathsf{out}}$ is $\mathscr{R}_{\mathsf{GIP}}$ specialized to a commitment scheme $\mathsf{CM}_{\mathsf{out}}$ with message space $\mathbb{G}_1^{\sqrt{N}}$, and an inner product map $\star : \mathbb{F} \times \mathbb{G} \to \mathbb{G}$ defined to be $a \star B = a \cdot B$. $\mathsf{Arg}_{\mathsf{out}}$ is an interactive argument for $\mathscr{R}_{\mathsf{out}}$.

[24] We can in fact handle an odd-number of variables without incurring this overhead by using a non-square matrix, but we omit that discussion for ease of exposition.

[25] One minor difference between our definition and that of [Lee21] is that we assume the evaluation value y is given in the clear, while they assume that it is committed to in a commitment C_y.

The blueprint is as follows: commit to a matrix $\mathbf{M} \in \mathbb{F}^{\sqrt{N} \times \sqrt{N}}$ by first commiting to the rows using $\mathsf{CM}_{\mathsf{in}}$ to obtain $C_i \leftarrow \mathsf{CM}_{\mathsf{in}}.\mathsf{Commit}(\mathsf{ck}_{\mathsf{in}}, \boldsymbol{M}_i)$ for each $i \in [0, 1, \ldots, \sqrt{N} - 1]$, and then committing to $\boldsymbol{C} := [C_i]_{i=0}^{\sqrt{N}-1}$ using $\mathsf{CM}_{\mathsf{out}}$ to obtain $C_{\mathbf{M}} \leftarrow \mathsf{CM}_{\mathsf{out}}.\mathsf{Commit}(\mathsf{ck}_{\mathsf{out}}, \boldsymbol{C})$.

To show that the VMV product $\boldsymbol{\ell}^\top \mathbf{M} \boldsymbol{r} = y$ for the matrix $\mathbf{M}$ committed to in $C_{\mathbf{M}}$, the prover and verifier engage in an interactive argument $\mathsf{Arg}_{\mathsf{out}}(\langle \mathcal{P}(\mathsf{ck}_{\mathsf{out}}, (C_{\mathbf{M}}, \boldsymbol{\ell}, C'), \boldsymbol{C}), \mathcal{V}(\mathsf{vk}_{\mathsf{out}}, (C_{\mathbf{M}}, \boldsymbol{\ell}, C')) \rangle)$ to show that $\sum_{i=0}^{\sqrt{N}-1} \ell_i \cdot C_i = C'$. This implies that C' must be a commitment to $\boldsymbol{a} := \boldsymbol{\ell}^\top \mathbf{M}$ under the commitment scheme $\mathsf{CM}_{\mathsf{in}}$. The prover is left to convince the verifier that $\langle \boldsymbol{a}, \boldsymbol{r} \rangle_{\mathbb{F}} = y$, which is done using $\mathsf{Arg}_{\mathsf{in}}$. The full construction is as follows:

$\mathsf{VMV.Setup}(1^\lambda, N) \to (\mathsf{ck}, \mathsf{vk})$:
1. Sample $(\mathsf{ck}_{\mathsf{in}}, \mathsf{vk}_{\mathsf{in}}) \leftarrow \mathsf{CM}_{\mathsf{in}}.\mathsf{Setup}(1^\lambda, \sqrt{N})$.
2. Sample $(\mathsf{ck}_{\mathsf{out}}, \mathsf{vk}_{\mathsf{out}}) \leftarrow \mathsf{CM}_{\mathsf{out}}.\mathsf{Setup}(1^\lambda, \sqrt{N})$.
3. Output $(\mathsf{ck} := (\mathsf{ck}_{\mathsf{in}}, \mathsf{ck}_{\mathsf{out}}), \mathsf{vk} := (\mathsf{vk}_{\mathsf{in}}, \mathsf{vk}_{\mathsf{out}}))$

$\mathsf{VMV.Commit}(\mathsf{ck}, \mathbf{M}) \to C_{\mathbf{M}}$:
1. Parse ck as $(\mathsf{ck}_{\mathsf{in}}, \mathsf{ck}_{\mathsf{out}})$.
2. For all $i \in [0, 1, \ldots, \sqrt{N} - 1]$: Commit to the i-th row as $C_i \leftarrow \mathsf{CM}_{\mathsf{in}}.\mathsf{Commit}(\mathsf{ck}_{\mathsf{in}}, \mathbf{M}_i)$.
3. Output $C_{\mathbf{M}} \leftarrow \mathsf{CM}_{\mathsf{out}}.\mathsf{Commit}(\mathsf{ck}_{\mathsf{out}}, \boldsymbol{C})$.

$\mathsf{VMV.Eval}(\langle \mathcal{P}(\mathsf{ck}, \mathsf{x}, \mathbf{M}), \mathcal{V}(\mathsf{vk}, \mathsf{x}) \rangle)$:
Parse: $\mathsf{ck} = (\mathsf{ck}_{\mathsf{in}}, \mathsf{ck}_{\mathsf{out}})$, $\mathsf{vk} = (\mathsf{vk}_{\mathsf{in}}, \mathsf{vk}_{\mathsf{out}})$ and $\mathsf{x} = (C_{\mathbf{M}}, \boldsymbol{\ell}, \boldsymbol{r}, y)$.
1. For all $i \in [0, 1, \ldots, \sqrt{N} - 1]$: $\mathcal{P}$ computes $C_i \leftarrow \mathsf{CM}_{\mathsf{in}}.\mathsf{Commit}(\mathsf{ck}_{\mathsf{in}}, \mathbf{M}_i)$.
2. $\mathcal{P}$ sets
$$\begin{aligned} \boldsymbol{X} &:= \boldsymbol{C}, & \boldsymbol{a} &:= \boldsymbol{\ell}^\top \mathbf{M}, \\ D_1 &:= C_{\mathbf{M}}, & D_2 &:= \mathsf{CM}_{\mathsf{in}}.\mathsf{Commit}(\mathsf{ck}_{\mathsf{in}}, \boldsymbol{a}), \\ E_1 &:= \langle \boldsymbol{\ell}, \boldsymbol{X} \rangle_{\mathbb{G}}, & e_2 &:= \langle \boldsymbol{r}, \boldsymbol{a} \rangle_{\mathbb{F}}. \end{aligned}$$
3. $\mathcal{P}$ sends E_1 to $\mathcal{V}$.
4. $\mathcal{V}$ sets $D_1 := C_{\mathbf{M}}$, $D_2 := E_1$ and $e_2 := y$.
5. Set $\mathsf{i}_1 := (\sqrt{N}, \mathsf{ck}_{\mathsf{out}})$, $\mathsf{x}_1 := (D_1, \boldsymbol{\ell}, E_1)$, $\mathsf{i}_2 := (\sqrt{N}, \mathsf{ck}_{\mathsf{in}})$ and $\mathsf{x}_2 := (D_2, \boldsymbol{r}, e_2)$.
6. $\mathcal{V}$ accepts if: $\langle \mathcal{P}_{\mathsf{out}}(\mathsf{i}_1, \mathsf{x}_1, \boldsymbol{X}), \mathcal{V}_{\mathsf{out}}(\mathsf{i}_1, \mathsf{x}_1) \rangle = 1$ and $\langle \mathcal{P}_{\mathsf{in}}(\mathsf{i}_2, \mathsf{x}_2, \boldsymbol{a}), \mathcal{V}_{\mathsf{in}}(\mathsf{i}_2, \mathsf{x}_2) \rangle = 1$.

Clearly $\mathsf{VMV.Commit}$ has a read-write streaming algorithm if $\mathsf{CM}_{\mathsf{in}}.\mathsf{Commit}$ and $\mathsf{CM}_{\mathsf{out}}.\mathsf{Commit}$ have read-write streaming algorithms; this is true for all known instantiations (see Sections C.3 and C.4). We are hence left to show that $\mathsf{VMV.Eval}$ has a read-write streaming prover. Our algorithm for the latter relies on the following helper functions, all of which require $O(\log N)$ random access space:

- $\mathsf{ExtractRow}$, given as input a read-stream for a matrix specified in row-major order, outputs the rows of a matrix one at a time;
- $\mathsf{CommitMatrix}$ computes commitments to these rows;
- VMProd computes vector matrix products of the form $\boldsymbol{\ell}^\top \mathbf{M}$; and
- $\mathsf{TensorProd}$, which compute tensor products of the form $\boldsymbol{c} := \otimes_{i=1}^{n}(a_i, b_i)$.

$\mathsf{ExtractRow}(N, \mathbf{R}_M) \to \mathbf{S}_R$:
1. For i in $[1, \ldots, \sqrt{N}]$, emit $m_i \leftarrow \mathbf{R}_M.\mathsf{read}()$.

$\mathsf{CommitMatrix}(N, \mathbf{R}_{\mathsf{ck}}, \mathbf{R}_M) \to \mathbf{S}_C$:
1. For i in $[1, \ldots, \sqrt{N}]$, emit
$D \rightsquigarrow \mathsf{CM}_{\mathsf{in}}.\mathsf{Commit}(\mathbf{R}_{\mathsf{ck}}) \rightsquigarrow \mathsf{ExtractRow}(N, \mathbf{R}_M)$.

$\mathsf{VMProd}(N, \mathbf{R}_\ell, \mathbf{R}_M) \to \mathbf{W}_S$:
1. Initialize the read stream $\mathbf{R}_S([0]_{i=0}^{\sqrt{N}-1})$.
2. For i in $[0, \ldots, \sqrt{N}-1]$:
3. $\quad \ell \leftarrow \mathbf{R}_\ell.\mathsf{read}()$.
4. $\quad \mathbf{R}_R \leftarrow \mathsf{ExtractRow}(N, \mathbf{R}_M)$.
5. $\quad \mathbf{R}_S \leftarrow \mathsf{LinComb}(1, \ell, \mathbf{R}_S, \mathbf{R}_R)$.
6. Output $\mathbf{W}_S \leftarrow \mathbf{R}_S.\mathsf{swapmode}()$.

$\mathsf{TensorProd}(n, \mathbf{R}_a, \mathbf{R}_b) \to \mathbf{W}_c$:
1. Initialize the read stream $\mathbf{R}_c(1)$.
2. For i in $[1, \ldots, n]$:
3. $\quad a \leftarrow \mathbf{R}_a.\mathsf{read}()$, $b \leftarrow \mathbf{R}_b.\mathsf{read}()$.
4. $\quad \mathbf{R}_{ia} \rightsquigarrow \mathsf{Map}(x \mapsto a \cdot x) \rightsquigarrow \mathbf{R}_c$.
5. $\quad \mathbf{R}_{ib} \rightsquigarrow \mathsf{Map}(x \mapsto b \cdot x) \rightsquigarrow \mathbf{R}_c$.
6. $\quad$ Set $\mathbf{R}_c \leftarrow \mathsf{Concat}(\mathbf{R}_{ia}, \mathbf{R}_{ib})$.
7. Output $\mathbf{W}_c \leftarrow \mathbf{R}_c.\mathsf{swapmode}()$.

Read-write streaming prover for VMV.Eval. We now present the full construction of a read-write streaming prover for VMV.Eval. We assume $\mathsf{Arg}_{\mathsf{in}}$ and $\mathsf{Arg}_{\mathsf{out}}$ all have read-write streaming provers, which is the case for all our instantiations (see Sections C.3 and C.4).

Read-write Streaming Algorithm 15: prover for VMV.Eval

$\mathsf{P}(\mathsf{ck}, \mathsf{x}, \mathbf{R}_M.\mathsf{init}(\mathbf{M}))$:
Parse: $\mathsf{ck} = (\mathbf{R}_{\mathsf{ckIn}}.\mathsf{init}(\mathsf{ck}_{\mathsf{in}}), \mathbf{R}_{\mathsf{ckOut}}.\mathsf{init}(\mathsf{ck}_{\mathsf{out}}))$ and $\mathsf{x} = (C_{\mathbf{M}}, \mathbf{R}_\ell.\mathsf{init}(\boldsymbol{\ell}), \mathbf{R}_r.\mathsf{init}(\boldsymbol{r}), y)$.
1. Compute commitments to the rows of $\mathbf{M}$: $\mathbf{R}_X \leftarrow \mathsf{CommitMatrix}(N, \mathbf{R}_{\mathsf{ckIn}}, \mathbf{R}_M)$.
2. Compute $\boldsymbol{\ell}^\top \mathbf{M}$: $\mathbf{R}_a \leftarrow \mathsf{VMProd}(N, \mathbf{R}_\ell, \mathbf{R}_M)$.
3. Compute $D_2 \leftarrow \mathsf{CM}_{\mathsf{in}}.\mathsf{Commit}(\mathbf{R}_{\mathsf{ckIn}}, \mathbf{R}_a)$.
4. Compute $E_1 \leftarrow \mathsf{InnerProd}(\mathbf{R}_\ell, \mathbf{R}_X)$.
5. Compute $e_2 \leftarrow \mathsf{InnerProd}(\mathbf{R}_r, \mathbf{R}_a)$.
6. Send E_1 to $\mathcal{V}$.
7. Set $\mathsf{i}_1 := (\sqrt{N}, \mathbf{R}_{\mathsf{ckOut}})$, $\mathsf{x}_1 := (C_{\mathbf{M}}, \mathbf{R}_\ell, E_1)$, $\mathsf{i}_2 := (\sqrt{N}, \mathbf{R}_{\mathsf{ckIn}})$, $\mathsf{x}_2 := (D_2, \mathbf{R}_\ell, e_2)$.
8. Run: $\mathsf{Arg}_{\mathsf{out}}(\langle \mathcal{P}(\mathsf{i}_1, \mathsf{x}_1, \mathbf{R}_X), \mathcal{V}(\mathsf{i}_1, \mathsf{x}_1)\rangle)$ and $\mathsf{Arg}_{\mathsf{in}}(\langle \mathcal{P}(\mathsf{i}_2, \mathsf{x}_2, \mathbf{R}_a), \mathcal{V}(\mathsf{i}_2, \mathsf{x}_2)\rangle)$.

C.2 Constructing PC schemes from VMV arguments

Given a vector-matrix-vector product argument VMV, the blueprint to construct a polynomial commitment scheme PC for n-variate multilinear polynomials is as follows.

$\mathsf{PC.Setup}(1^\lambda, n) \to (\mathsf{ck}, \mathsf{vk})$:
1. Define $N := 2^n$.
2. Output $(\mathsf{ck}, \mathsf{vk}) \leftarrow \mathsf{VMV.Setup}(1^\lambda, N)$.

$\mathsf{PC.Commit}(\mathsf{ck}, \boldsymbol{p}) \to C_{\mathbf{M}}$:
1. Define $\mathbf{M} \in \mathbb{F}^{\sqrt{N} \times \sqrt{N}}$ with $M_{ij} := p_{i \cdot \sqrt{N} + j}$.
2. Output $C_{\mathbf{M}} \leftarrow \mathsf{VMV.Commit}(\mathsf{ck}, \mathbf{M})$.

$\mathsf{PC.Eval}(\langle \mathcal{P}(\mathsf{ck}, \mathsf{x}, \boldsymbol{p}), \mathcal{V}(\mathsf{vk}, \mathsf{x})\rangle)$:
Parse: $\mathsf{x} = (C_{\mathbf{M}}, \boldsymbol{z}, y)$.
1. $\mathcal{P}$ defines $\mathbf{M}$ with $M_{ij} := p_{i \cdot \sqrt{N} + j}$.
2. $\mathcal{P}$ and $\mathcal{V}$ set $\boldsymbol{\ell} := \otimes_{i=1}^{n/2}(1 - z_i, z_i)$ and $\boldsymbol{r} := \otimes_{i=n/2+1}^{n}(1 - z_i, z_i)$.
3. Define $\mathsf{x}_{\mathsf{VMV}} := (C_{\mathbf{M}}, \boldsymbol{\ell}, \boldsymbol{r}, y)$ and $\mathsf{w}_{\mathsf{VMV}} := \mathbf{M}$.
4. Output $\langle \mathcal{P}_{\mathsf{VMV}}(\mathsf{ck}, \mathsf{x}_{\mathsf{VMV}}, \mathsf{w}_{\mathsf{VMV}}), \mathcal{V}_{\mathsf{VMV}}(\mathsf{vk}, \mathsf{x}_{\mathsf{VMV}})\rangle$.

Since all the helper functions run in $O(N)$ time, the foregoing PC scheme inherits the efficiency and RW streaming prover of the underlying VMV argument.

C.3 The Hyrax PC scheme

The Hyrax PC scheme from [WTSTW18] is obtained via the aforementioned blueprint, with $\mathsf{CM}_{\mathsf{in}}$ and $\mathsf{Arg}_{\mathsf{in}}$ corresponding to FIP, and $\mathsf{CM}_{\mathsf{out}}.\mathsf{Commit}(\bot, \boldsymbol{X}) := \boldsymbol{X}$ being the identity commitment, and $\mathsf{Arg}_{\mathsf{out}}$ being the naive argument, where the verifier directly checks that $\sum_{i=0}^{N-1} \ell_i \cdot X_i = X'$ in $O(\sqrt{N})$ time. The following lemma follows from [WTSTW18, Lemma 5] and the fact that all of the aforementioned schemes have read-write streaming provers.

Lemma C.1. Hyrax *is a read-write streaming polynomial commitment scheme for n-variate multilinear polynomials with the following efficiency:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(\sqrt{N})$	$O(N)$	$O(\log N)$	$O(N)$	$O(\sqrt{N})$	$O(\log N)$

C.4 The PC scheme implicit in BMMTV21

Bünz et al. [BMMTV21] construct a univariate PC scheme that achieves square-root SRS size, linear prover time, and logarithmic verifier time. Their scheme can be adapted to the multilinear setting without affecting performance by adapting the VMV argument implicit in their scheme. Their VMV scheme is obtained via the aforementioned blueprint, with $\mathsf{CM}_{\mathsf{in}}$ and $\mathsf{Arg}_{\mathsf{in}}$ corresponding to FIP, and $\mathsf{CM}_{\mathsf{out}}$ and $\mathsf{Arg}_{\mathsf{out}}$ corresponding to MIPP. As an optimization, the scheme uses a structured commitment key to obtain a sublinear verifier as explained in Sections B.3 and B.4. The following lemma follows from the fact that all of the aforementioned components have read-write streaming provers.

Lemma C.2. *The PC scheme implicit in [BMMTV21] is a read-write streaming polynomial commitment scheme for n-variate multilinear polynomials with the following efficiency:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(\sqrt{N})$	$O(N)$	$O(\log N)$	$O(N)$	$O(\log N)$	$O(\log N)$

D Vector-matrix-vector product arguments from Dory

In this section we describe the VMV argument from Dory [Lee21], which departs slightly from the blueprint presented in Section C.1, by essentially running a MIPP and FIP in parallel with modified versions MIPP.Reduce and FIP.Reduce to allow the commitment keys for each round to be fixed a priori (and not obtained by folding the commitment key from the prior round).

This allows the verifier to know the commitment key for the final round without having to fold it itself, thus avoiding the need to have either an $O(N)$ verification time overhead or the need to have the prover provide a proof of correct folding like in [BMMTV21], which would require a trusted setup. At a high level, Dory's VMV argument works as follows:[26]

Setup. As part of its commitment key, Setup samples the the unstructured commitment keys for round 1 of the FIP and MIPP protocols, $\boldsymbol{\Gamma}_{1,1} \xleftarrow{\$} \mathbb{G}_1^N$ and $\boldsymbol{\Gamma}_{2,1} \xleftarrow{\$} \mathbb{G}_2^N$. For each round $i \in [2, 3, \ldots, n]$, Setup sets the commitment keys for round i to be $\boldsymbol{\Gamma}_{1,i} := (\boldsymbol{\Gamma}_{1,i-1})_L$ and $\boldsymbol{\Gamma}_{2,i} := (\boldsymbol{\Gamma}_{2,i-1})_L$. In addition, the commitment key contains certain preprocessed information required for the modified reductions for FIP and MIPP.

Commit. Dory commits to a matrix $\mathbf{M} \in \mathbb{F}^{N \times N}$ in the standard way: by first Pedersen commiting to the rows to obtain $C_i \leftarrow \mathsf{CM}_P.\mathsf{Commit}(\boldsymbol{\Gamma}_{1,1}, \boldsymbol{M}_i)$ for each $i \in [0, 1, \ldots, N-1]$, and then AFGHO committing to $\boldsymbol{C}$ to obtain $C_{\mathbf{M}} \leftarrow \mathsf{CM}_1.\mathsf{Commit}(\boldsymbol{\Gamma}_{2,1}, \boldsymbol{C})$.

Eval. To show that the VMV product $\boldsymbol{\ell}^T \mathbf{M} \boldsymbol{r} = y$, for the matrix $\mathbf{M}$ committed to in $C_{\mathbf{M}}$, prior schemes have first used an MIPP argument to show that $\sum_{i=0}^{N-1} \ell_i \cdot C_i = C'$, where C' must be the commitment to $\boldsymbol{a} := \boldsymbol{\ell}^T \mathbf{M}$. Then, to convince the verifier that $\langle \boldsymbol{a}, \boldsymbol{r} \rangle_{\mathbb{F}} = y$, the prover would then use an FIP argument.

Dory essentially runs the MIPP and FIP in parallel but runs the FIP 'in $\mathbb{G}_2$' by lifting the vector $\boldsymbol{a} \in \mathbb{F}^N$ to $\boldsymbol{A} := \boldsymbol{a} \cdot \Gamma_{2,\mathsf{fin}} \in \mathbb{G}_2^N$ using a generator $\Gamma_{2,\mathsf{fin}} \xleftarrow{\$} \mathbb{G}_2$. This is done because by dealing with the group vectors $\boldsymbol{C} \in \mathbb{G}_1^N$ and $\boldsymbol{A} \in \mathbb{G}_2^N$, one can fold the commitment keys $\boldsymbol{\Gamma}_1$ and $\boldsymbol{\Gamma}_2$ into $\boldsymbol{C}$ and $\boldsymbol{A}$ respectively, enabling Dory to switch to a completely new set of commitment keys $\boldsymbol{\Gamma}'_1 \in \mathbb{G}_1^{N/2}$ and $\boldsymbol{\Gamma}'_2 \in \mathbb{G}_2^{N/2}$ (that do not need to depend on $\boldsymbol{\Gamma}_1$ and $\boldsymbol{\Gamma}_2$) in the every round.

Scalar pairing product. We first define the scalar pairing product relation $\mathscr{R}_{\mathsf{SPP}}$, as defined in [Lee21] and present their interactive argument Dory.InnerProd for $\mathscr{R}_{\mathsf{SPP}}$. Then in Section D.4 we show how to use Dory.InnerProd to construct a VMV argument.

Definition D.1 (scalar pairing product relation)**.** *The indexed relation $\mathscr{R}_{\mathsf{SPP}}$ is the set of triples*

$$\begin{pmatrix} \mathsf{i}, \\ \mathsf{x}, \\ \mathsf{w} \end{pmatrix} = \begin{pmatrix} (N, \boldsymbol{\Gamma}_1, \boldsymbol{\Gamma}_2), \\ (C, D_1, D_2, E_1, E_2, \boldsymbol{\ell}, \boldsymbol{r}), \\ (\boldsymbol{X}, \boldsymbol{Y}) \end{pmatrix}$$

where N is an integer, $\boldsymbol{\Gamma}_1 \in \mathbb{G}_1^N$, $\boldsymbol{\Gamma}_2 \in \mathbb{G}_2^N$, $C, D_1, D_2 \in \mathbb{G}_T$, $E_1 \in \mathbb{G}_1$, $E_2 \in \mathbb{G}_2$, $\boldsymbol{\ell}, \boldsymbol{r} \in \mathbb{F}^N$, $\boldsymbol{X} \in \mathbb{G}_1^N$ and $\boldsymbol{Y} \in \mathbb{G}_2^N$ such that

$$\begin{aligned} &C = \langle \boldsymbol{X}, \boldsymbol{Y} \rangle_e, \\ &D_1 = \mathsf{CM}_1.\mathsf{Commit}(\boldsymbol{\Gamma}_2, \boldsymbol{X}), \quad D_2 = \mathsf{CM}_2.\mathsf{Commit}(\boldsymbol{\Gamma}_1, \boldsymbol{Y}), \\ &E_1 = \langle \boldsymbol{\ell}, \boldsymbol{X} \rangle_{\mathbb{G}}, \qquad\qquad E_2 = \langle \boldsymbol{r}, \boldsymbol{Y} \rangle_{\mathbb{G}}. \end{aligned}$$

[26] For ease of exposition, we consider committing to and evaluating VMV products over matrices of dimension $N \times N$.

D.1 Dory.Reduce

In this section we present an interactive reduction of knowledge Dory.Reduce [27] that reduces an instance of $\mathscr{R}_{\mathsf{SPP}}$ into a related instance of half the length using a random verifier challenge. That is, given an index $\mathsf{i} = (2^n, \boldsymbol{\Gamma}_1, \boldsymbol{\Gamma}_2)$, and commitment keys $\boldsymbol{\Gamma}'_1 \xleftarrow{\$} \mathbb{G}_1^{2^{n-1}}$ and $\boldsymbol{\Gamma}'_2 \xleftarrow{\$} \mathbb{G}_2^{2^{n-1}}$ that are fixed a priori but randomly sampled, Dory.Reduce reduces the problem of checking if a tuple $(\mathsf{i}, \mathsf{x}, \mathsf{w}) \in \mathscr{R}_{\mathsf{IPP}}$ to the problem of checking if $(\mathsf{i}', \mathsf{x}', \mathsf{w}') \in \mathscr{R}_{\mathsf{GIP}}$, where $\mathsf{i}' = (2^{n-1}, \boldsymbol{\Gamma}'_1, \boldsymbol{\Gamma}'_2)$.

Dory.Reduce

$\langle \mathcal{P}((\boldsymbol{\Gamma}'_1, \boldsymbol{\Gamma}'_2), \mathsf{i}, \mathsf{x}, \mathsf{w}), \mathcal{V}(\mathsf{i}, \mathsf{x}) \rangle$:
Parse: $\mathsf{i} = (2^n, \boldsymbol{\Gamma}_1, \boldsymbol{\Gamma}_2)$, $\mathsf{x} = (C, D_1, D_2, E_1, E_2, \boldsymbol{\ell}, \boldsymbol{r})$ and $\mathsf{w} = (\boldsymbol{X}, \boldsymbol{Y})$.
Precompute: $\Delta_{1E} := \langle \boldsymbol{\Gamma}_{1E}, \boldsymbol{\Gamma}'_{2} \rangle_e$, $\Delta_{1O} := \langle \boldsymbol{\Gamma}_{1O}, \boldsymbol{\Gamma}'_{2} \rangle_e$, $\Delta_{2E} := \langle \boldsymbol{\Gamma}'_{1}, \boldsymbol{\Gamma}_{2E} \rangle_e$, $\Delta_{2O} := \langle \boldsymbol{\Gamma}'_{1}, \boldsymbol{\Gamma}_{2O} \rangle_e$ and $\chi := \langle \boldsymbol{\Gamma}_1, \boldsymbol{\Gamma}_2 \rangle_e$.

The prover computes and sends the necessary cross-products to the verifier:
1. $\mathcal{P}$ computes $D_{1E} := \langle \boldsymbol{X}_E, \boldsymbol{\Gamma}'_2 \rangle_e$, $D_{1O} := \langle \boldsymbol{X}_O, \boldsymbol{\Gamma}'_2 \rangle_e$, $D_{2E} := \langle \boldsymbol{\Gamma}'_1, \boldsymbol{Y}_E \rangle_e$ and $D_{2O} := \langle \boldsymbol{\Gamma}'_1, \boldsymbol{Y}_O \rangle_e$.
2. $\mathcal{P}$ also computes $E_{1\beta} := \langle \boldsymbol{\ell}, \boldsymbol{\Gamma}_1 \rangle_{\mathbb{G}}$, $E_{2\beta} := \langle \boldsymbol{r}, \boldsymbol{\Gamma}_2 \rangle_{\mathbb{G}}$.
3. $\mathcal{P}$ sends $D_{1E}, D_{1O}, D_{2E}, D_{2O}, E_{1\beta}, E_{2\beta}$ to $\mathcal{V}$.

4. $\mathcal{V}$ samples $\beta \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$.

The prover folds the commitment key into its witness with respect to the challenge β:
5. $\mathcal{P}$ sets $\boldsymbol{X} = \boldsymbol{X} + \beta \boldsymbol{\Gamma}_1$ and $\boldsymbol{Y} = \boldsymbol{Y} + \beta^{-1} \boldsymbol{\Gamma}_2$.
6. $\mathcal{P}$ computes $C_+ := \langle \boldsymbol{X}_E, \boldsymbol{Y}_O \rangle_e$ and $C_- := \langle \boldsymbol{X}_O, \boldsymbol{Y}_E \rangle_e$.
7. $\mathcal{P}$ sets $E_{1+} := \langle \boldsymbol{\ell}_O, \boldsymbol{X}_E \rangle_{\mathbb{G}}$, $E_{1-} := \langle \boldsymbol{\ell}_E, \boldsymbol{X}_O \rangle_{\mathbb{G}}$, $E_{2+} := \langle \boldsymbol{r}_E, \boldsymbol{Y}_O \rangle_{\mathbb{G}}$, $E_{2-} := \langle \boldsymbol{r}_O, \boldsymbol{Y}_E \rangle_{\mathbb{G}}$.
8. $\mathcal{P}$ sends $C_+, C_-, E_{1+}, E_{1-}, E_{2+}, E_{2-}$ to $\mathcal{V}$.

9. $\mathcal{V}$ samples $\alpha \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$.

The prover and verifier fold their instance and witness in half with respect to the challenge α:
10. $\mathcal{P}$ sets $\boldsymbol{X}' := \alpha \boldsymbol{X}_E + \boldsymbol{X}_O$ and $\boldsymbol{Y}' := \alpha^{-1} \boldsymbol{Y}_E + \boldsymbol{Y}_O$.
11. $\mathcal{P}$ and $\mathcal{V}$ set

$$
\begin{aligned}
C' &:= C + \chi + \beta D_2 + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_-, \\
D'_1 &:= \alpha D_{1E} + D_{1O} + \alpha\beta \Delta_{1E} + \beta \Delta_{1O}, \\
D'_2 &:= \alpha^{-1} D_{2E} + D_{2O} + \alpha^{-1}\beta^{-1} \Delta_{2E} + \beta^{-1} \Delta_{2O}, \\
E'_1 &:= E_1 + \beta E_{1\beta} + \alpha E_{1+} + \alpha^{-1} E_{1-}, \\
E'_2 &:= E_2 + \beta^{-1} E_{2\beta} + \alpha E_{2+} + \alpha^{-1} E_{2-}, \\
\boldsymbol{\ell}' &:= \alpha^{-1} \boldsymbol{\ell}_E + \boldsymbol{\ell}_O, \\
\boldsymbol{r}' &:= \alpha \boldsymbol{r}_E + \boldsymbol{r}_O
\end{aligned}
$$

12. Define $\mathsf{i}' := (2^{n-1}, \boldsymbol{\Gamma}'_1, \boldsymbol{\Gamma}'_2)$, $\mathsf{x}' := (C', D'_1, D'_2, E'_1, E'_2, \boldsymbol{\ell}', \boldsymbol{r}')$ and $\mathsf{w}' := (\boldsymbol{X}', \boldsymbol{Y}')$.
13. $\mathcal{P}$ receives output $(\mathsf{i}', \mathsf{x}', \mathsf{w}')$ and $\mathcal{V}$ receives output $(\mathsf{i}', \mathsf{x}')$.

Lemma D.2 ([Lee21, Theorem 6]). *Let $\boldsymbol{\Gamma}'_1 \xleftarrow{\$} \mathbb{G}_1^{2^{n-1}}$, $\boldsymbol{\Gamma}'_2 \xleftarrow{\$} \mathbb{G}_2^{2^{n-1}}$ and $\mathsf{i} = (2^n, \boldsymbol{\Gamma}_1, \boldsymbol{\Gamma}_2)$ be an index for $\mathscr{R}_{\mathsf{SPP}}$. Then the following is an interactive argument of knowledge for proving that $(\mathsf{i}, \mathsf{x}, \mathsf{w}) \in \mathscr{R}_{\mathsf{SPP}}$:*
1. Prover and verifier run: $(\mathsf{i}', \mathsf{x}', \mathsf{w}') \leftarrow \mathsf{Dory.Reduce}(\langle \mathcal{P}((\boldsymbol{\Gamma}'_1, \boldsymbol{\Gamma}'_2), \mathsf{i}, \mathsf{x}, \mathsf{w}), \mathcal{V}((\boldsymbol{\Gamma}'_1, \boldsymbol{\Gamma}'_2), \mathsf{i}, \mathsf{x}) \rangle)$.
2. Prover sends w' to verifier.

[27] We note that in [Lee21], this protocol is actually called Dory.ReduceExt, but we will denote it Dory.Reduce for brevity.

3. Verifier accepts if and only if $(\mathsf{i}', \mathsf{x}', \mathsf{w}') \in \mathscr{R}_{\mathsf{SPP}}$.

D.2 Read-write streaming prover for Dory.Reduce

We now present a read-write streaming prover for Dory.Reduce.

Read-write Streaming Algorithm 16: prover for Dory.Reduce

$\mathsf{P}((\mathbf{R}_{\Gamma 1'}.\mathsf{init}(\boldsymbol{\Gamma}_1'), \mathbf{R}_{\Gamma 2'}.\mathsf{init}(\boldsymbol{\Gamma}_2')), \mathsf{i}, \mathsf{x}, \mathsf{w})$:
Parse: $\mathsf{i} = (2^n, \mathbf{R}_{\Gamma 1}.\mathsf{init}(\boldsymbol{\Gamma}_1), \mathbf{R}_{\Gamma 2}.\mathsf{init}(\boldsymbol{\Gamma}_2))$, $\mathsf{x} = (C, D_1, D_2, E_1, E_2, \mathbf{R}_\ell.\mathsf{init}(\boldsymbol{\ell}), \mathbf{R}_r.\mathsf{init}(\boldsymbol{r}))$ and $\mathsf{w} = (\mathbf{R}_X.\mathsf{init}(\boldsymbol{X}), \mathbf{R}_Y.\mathsf{init}(\boldsymbol{Y}))$.
Precompute: $\Delta_{1E} := \langle \boldsymbol{\Gamma}_{1E}, \boldsymbol{\Gamma}_2' \rangle_e$, $\Delta_{1O} := \langle \boldsymbol{\Gamma}_{1O}, \boldsymbol{\Gamma}_2' \rangle_e$, $\Delta_{2E} := \langle \boldsymbol{\Gamma}_1', \boldsymbol{\Gamma}_{2E} \rangle_e$, $\Delta_{2O} := \langle \boldsymbol{\Gamma}_1', \boldsymbol{\Gamma}_{2O} \rangle_e$ and $\chi := \langle \boldsymbol{\Gamma}_1, \boldsymbol{\Gamma}_2 \rangle_e$.

Compute and send the necessary cross-products to the verifier:
1. Obtain $(\mathbf{R}_{XE}, \mathbf{R}_{XO}) \leftarrow \mathsf{SplitEO}(\mathbf{R}_X)$ and $(\mathbf{R}_{YE}, \mathbf{R}_{YO}) \leftarrow \mathsf{SplitEO}(\mathbf{R}_Y)$.
2. Obtain $(\mathbf{R}_{\ell E}, \mathbf{R}_{\ell O}) \leftarrow \mathsf{SplitEO}(2^n, \mathbf{R}_\ell)$ and $(\mathbf{R}_{rE}, \mathbf{R}_{rO}) \leftarrow \mathsf{SplitEO}(2^n, \mathbf{R}_r)$.
3. Compute $D_{1E} := \mathsf{InnerProd}(\mathbf{R}_{XE}, \mathbf{R}_{\Gamma 2'})$, $D_{1O} := \mathsf{InnerProd}(\mathbf{R}_{XO}, \mathbf{R}_{\Gamma 2'})$, $D_{2E} := \mathsf{InnerProd}(\mathbf{R}_{\Gamma 1'}, \mathbf{R}_{YE})$ and $D_{2O} := \mathsf{InnerProd}(\mathbf{R}_{\Gamma 1'}, \mathbf{R}_{YO})$.
4. Compute $E_{1\beta} := \mathsf{InnerProd}(\mathbf{R}_\ell, \mathbf{R}_{\Gamma 1})$, $E_{2\beta} := \mathsf{InnerProd}(\mathbf{R}_r, \mathbf{R}_{\Gamma 2})$.
5. Send $D_{1E}, D_{1O}, D_{2E}, D_{2O}, E_{1\beta}, E_{2\beta}$ to $\mathcal{V}$.

6. Receive $\beta \xleftarrow{\$} \mathbb{F}$ from $\mathcal{V}$.

Fold the commitment key into the witness with respect to the challenge β:
7. $\mathsf{LinComb}(1, \beta, \mathbf{R}_X, \mathbf{R}_{\Gamma 1})$ and $\mathsf{LinComb}(1, \beta^{-1}, \mathbf{R}_Y, \mathbf{R}_{\Gamma 2})$.
8. Compute $C_+ := \mathsf{InnerProd}(\mathbf{R}_{XE}, \mathbf{R}_{YO})$ and $C_- := \mathsf{InnerProd}(\mathbf{R}_{XO}, \mathbf{R}_{YE})$.
9. Compute $E_{1+} := \mathsf{InnerProd}(\mathbf{R}_{\ell O}, \mathbf{R}_{XE})$, $E_{1-} := \mathsf{InnerProd}(\mathbf{R}_{\ell E}, \mathbf{R}_{XO})$.
10. Compute $E_{2+} := \mathsf{InnerProd}(\mathbf{R}_{rE}, \mathbf{R}_{YO})$, $E_{2-} := \mathsf{InnerProd}(\mathbf{R}_{rO}, \mathbf{R}_{YE})$.
11. Send $C_+, C_-, E_{1+}, E_{1-}, E_{2+}, E_{2-}$ to $\mathcal{V}$.

12. Receive $\alpha \xleftarrow{\$} \mathbb{F}$ from $\mathcal{V}$.

Fold the instance and witness in half with respect to the challenge α:
13. $\mathsf{RWFoldInPlace}(2^n, \alpha, 1, 1, \mathbf{R}_X)$ and $\mathsf{RWFoldInPlace}(2^n, \alpha^{-1}, 1, 1, \mathbf{R}_Y)$.
14. $\mathsf{RWFoldInPlace}(2^n, \alpha^{-1}, 1, 1, \mathbf{R}_\ell)$ and $\mathsf{RWFoldInPlace}(2^n, \alpha, 1, 1, \mathbf{R}_r)$.
15. Set

$$
\begin{aligned}
C' &:= C + \chi + \beta D_2 + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_- \\
D_1' &:= \alpha D_{1E} + D_{1O} + \alpha\beta \Delta_{1E} + \beta \Delta_{1O} \\
D_2' &:= \alpha^{-1} D_{2E} + D_{2O} + \alpha^{-1}\beta^{-1} \Delta_{2E} + \beta^{-1} \Delta_{2O}, \\
E_1' &:= E_1 + \beta E_{1\beta} + \alpha E_{1+} + \alpha^{-1} E_{1-}, \\
E_2' &:= E_2 + \beta^{-1} E_{2\beta} + \alpha E_{2+} + \alpha^{-1} E_{2-},
\end{aligned}
$$

16. Define $\mathsf{i}' := (2^{n-1}, \mathbf{R}_{\Gamma 1'}, \mathbf{R}_{\Gamma 2'})$, $\mathsf{x}' := (C', D_1', D_2', E_1', E_2', \mathbf{R}_\ell, \mathbf{R}_r)$ and $\mathsf{w}' := (\mathbf{R}_X, \mathbf{R}_Y)$.
17. Retain output $(\mathsf{i}', \mathsf{x}', \mathsf{w}')$.

Lemma D.3. Dory.Reduce *and Algorithm 16 together define a read-write streaming reduction of knowledge with the following properties:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(\sqrt{N})$	$O(N)$	$O(\log N)$	$O(N)$	$O(\log N)$	$O(\log N)$

The proof of this lemma is similar to that of Lemma 6.1.

D.3 Dory-InnerProduct

Just like the GIPA protocol from Section B.2.3, the full argument for $\mathscr{R}_{\mathsf{SPP}}$, Dory.InnerProd, applies Dory.Reduce iteratively to shrink the size of the scalar pairing product instance to length 1, and then checks the instance directly.

Lemma D.4. Dory.InnerProd *is a read-write streaming interactive argument of knowledge for* $\mathscr{R}_{\mathsf{SPP}}$ *with the following properties:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(N)$	$O(N)$	$O(\log N)$	$O(N)$	$O(N)$	$O(\log N)$

Proof. Completeness and knowledge soundness follow from [Lee21, Theorem 7]. The existence and efficiency of a read-write streaming prover for Dory.InnerProd follows from Lemma D.3. □

D.4 VMV from Dory-Innerproduct

In this section we describe how to build a VMV protocol from Dory.InnerProd. Notice that VMV.Setup precomputes all the necessary terms for Dory.InnerProd, and that VMV.Open lifts $\boldsymbol{a} := \boldsymbol{\ell}^T\mathbf{M}$ into a vector in $\mathbb{G}_2$ using $\Gamma_{2,\mathsf{fin}}$ (this is highlighted in blue).

$\mathsf{VMV.Setup}(1^\lambda, N^2 = 2^{2n}) \to (\mathsf{ck}, \mathsf{vk})$:

1. Sample $\mathbf{\Gamma}_{1,1} \xleftarrow{\$} \mathbb{G}_1^N$ and $(\mathbf{\Gamma}_{2,1}, \Gamma_{2,\mathsf{fin}}) \xleftarrow{\$} \mathbb{G}_2^N \times \mathbb{G}_2$.
2. For i in $[1, \ldots, n]$ set:
3. $\quad \mathbf{\Gamma}_{1,i+1} := (\mathbf{\Gamma}_{1,i})_L$ and $\mathbf{\Gamma}_{2,i+1} := (\mathbf{\Gamma}_{2,i})_L$.
4. $\quad \chi_i := \langle \mathbf{\Gamma}_{1,i}, \mathbf{\Gamma}_{2,i} \rangle_e$.
5. $\quad \Delta_{1E,i} := \langle (\mathbf{\Gamma}_{1,i})_E, \mathbf{\Gamma}_{2,i+1} \rangle_e$, $\Delta_{1O,i} := \langle (\mathbf{\Gamma}_{1,i})_O, \mathbf{\Gamma}_{2,i+1} \rangle_e$, $\Delta_{2E,i} := \langle \mathbf{\Gamma}_{1,i+1}, (\mathbf{\Gamma}_{2,i})_E \rangle_e$ and $\Delta_{2O,i} := \langle \mathbf{\Gamma}_{1,i+1}, (\mathbf{\Gamma}_{2,i})_O \rangle_e$.
6. Compute $\chi_{n+1} := e(\Gamma_{1,n+1}, \Gamma_{2,n+1})$.
7. Set $\mathsf{precomp} := ([\chi_i]_{i=1}^{n+1}, [\Delta_{1E,i}]_{i=1}^n, [\Delta_{1O,i}]_{i=1}^n, [\Delta_{2E,i}]_{i=1}^n, [\Delta_{2O,i}]_{i=1}^n, \Gamma_{2,\mathsf{fin}})$.
8. Set $\mathsf{ck} := ([\mathbf{\Gamma}_{1,i}]_{i=1}^{n+1}, [\mathbf{\Gamma}_{2,i}]_{i=1}^{n+1}, \mathsf{precomp})$.
9. Set $\mathsf{vk} := (\Gamma_{1,n+1}, \Gamma_{2,n+1}, \mathsf{precomp})$.
10. Output $(\mathsf{ck}, \mathsf{vk})$.

$\mathsf{VMV.Commit}(\mathsf{ck}, \mathbf{M}) \to C_{\mathbf{M}}$:

1. Parse ck as $([\mathbf{\Gamma}_{1,i}]_{i=1}^{n+1}, [\mathbf{\Gamma}_{2,i}]_{i=1}^{n+1}, \mathsf{precomp})$.
2. For all $i \in [0, 1, \ldots, N-1]$: Commit to the i-th row as $C_i \leftarrow \mathsf{CM}_P.\mathsf{Commit}(\mathbf{\Gamma}_{1,1}, \mathbf{M}_i)$.
3. Output $C_{\mathbf{M}} \leftarrow \mathsf{CM}_1.\mathsf{Commit}(\mathbf{\Gamma}_{2,1}, \boldsymbol{C})$.

$\mathsf{VMV.Open}(\langle \mathcal{P}(\mathsf{ck}, \mathsf{x}, \mathbf{M}), \mathcal{V}(\mathsf{vk}, \mathsf{x}) \rangle)$:
Parse: $\mathsf{ck} = ([\mathbf{\Gamma}_{1,i}]_{i=1}^{n+1}, [\mathbf{\Gamma}_{2,i}]_{i=1}^{n+1}, \mathsf{precomp})$, $\mathsf{vk} = (\Gamma_{1,n+1}, \Gamma_{2,n+1}, \mathsf{precomp})$ and $\mathsf{x} = (C_{\mathbf{M}}, \boldsymbol{\ell}, \boldsymbol{r}, y)$.
1. For all $i \in [0, 1, \ldots, N-1]$: $\mathcal{P}$ computes $C_i := \mathsf{CM}_P.\mathsf{Commit}(\mathbf{\Gamma}_{1,1}, \mathbf{M}_i)$.
2. $\mathcal{P}$ sets $\boldsymbol{X} := \boldsymbol{C}$, $D_1 := C_{\mathbf{M}}$.
3. $\mathcal{P}$ computes $\boldsymbol{a} := \boldsymbol{\ell}^T \mathbf{M}$, and $\boldsymbol{Y} := \boldsymbol{a} \cdot \Gamma_{2,\mathsf{fin}}$.
4. $\mathcal{P}$ sets
$$C := \langle \boldsymbol{X}, \boldsymbol{Y} \rangle_e,$$
$$D_1 := C_{\mathbf{M}}, \qquad D_2 := \mathsf{CM}_2(\mathbf{\Gamma}_{1,1}, \boldsymbol{Y}),$$
$$E_1 := \langle \boldsymbol{\ell}, \boldsymbol{X} \rangle_{\mathbb{G}}, \quad E_2 := \langle \boldsymbol{r}, \boldsymbol{Y} \rangle_{\mathbb{G}}.$$
5. $\mathcal{P}$ sends C, D_2, E_1, E_2 to $\mathcal{V}$.
6. $\mathcal{V}$ checks that $E_2 = y \cdot \Gamma_{2,\mathsf{fin}}$.
7. $\mathcal{V}$ checks that $e(E_1, \Gamma_{2,\mathsf{fin}}) = D_2$.
8. Set $\mathsf{i}_{\mathsf{SPP}} := (N, \mathbf{\Gamma}_{1,1}, \mathbf{\Gamma}_{2,1})$, $\mathsf{x}_{\mathsf{SPP}} := (C, D_1, D_2, E_1, E_2, \boldsymbol{\ell}, \boldsymbol{r})$ and $\mathsf{w}_{\mathsf{SPP}} := (\boldsymbol{X}, \boldsymbol{Y})$.
9. $\mathcal{P}$ and $\mathcal{V}$ run $\mathsf{Dory.InnerProd}(\langle \mathcal{P}(\mathsf{i}_{\mathsf{SPP}}, \mathsf{x}_{\mathsf{SPP}}, \mathsf{w}_{\mathsf{SPP}}), \mathcal{V}(\mathsf{i}_{\mathsf{SPP}}, \mathsf{x}_{\mathsf{SPP}}) \rangle)$.

Lemma D.5. VMV *is a read-write streaming VMV argument for 'random evaluation vectors'*[28] *with the following efficiency:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(\sqrt{N})$	$O(N)$	$O(\log N)$	$O(N)$	$O(\log N)$	$O(\log N)$

Proof. The completeness and soundness claims follows from [Lee21, Theorem 9], while the efficiency claims follow from the efficiency of our read-write streaming algorithm for Dory.InnerProd. □

Using VMV to construct a PC scheme as described in Section C.2 yields the following lemma.

Lemma D.6. *The* Dory *PC scheme [Lee21] is a read-write streaming polynomial commitment scheme for n-variate multilinear polynomials with the following efficiency:*

SRS size	prover time	prover space		check time	proof size
		random-access	streaming		
$O(\sqrt{N})$	$O(N)$	$O(\log N)$	$O(N)$	$O(\log N)$	$O(\log N)$

E Read-write streaming and external memory

The external memory model [AV88] and its generalization, the parallel disk model (PDM) [VS94], were defined to enable reasoning about the efficiency of algorithms whose state is stored in external memory. At a high level, they do so by developing a cost model where accessing a continuous block of data in external memory is much cheaper than accessing an equal number of individual locations. We recall the definition of the PDM below.

[28] The current scheme is only sound for $\boldsymbol{\ell} := \otimes_{i=1}^{n/2}(1 - z_i, z_i)$ and $\boldsymbol{r} := \otimes_{i=n/2+1}^{n}(1 - z_i, z_i)$ constructed using a random $\boldsymbol{z} \xleftarrow{\$} \mathbb{F}^n$. [Lee21] shows how to remedy this by repeating the VMV argument twice, once with random evaluation vectors and once with the required evaluation vectors.

Definition E.1 ([VS94])**.** *A parallel data model (PDM) algorithm $\mathcal{A}$ is parameterized by an input size N, an internal memory size M, a block transfer size B, and a number of independent disks D, such that $B \ll M \ll N$. In a single I/O step, $\mathcal{A}$ can read/write a block of length B from each of the D disks simultaneously, and in a computation step it can perform a computation over the data stored in internal memory. The three main performance measures of PDM algorithms are the number of (parallel) I/O operations, the amount of internal and disk memory required for a problem of size N, and the number of computation steps (i.e., execution time).*

It is easy to see that the RW streaming model is a restricted case of the PDM, where the block transfer size is $B = 1$ and read-write accesses must be performed in a streaming manner. On the other hand, every "good" RW streaming algorithm also implies a similarly "good" PDM algorithm that simply reads a batch of items in each access (i.e., sets $B > 1$). We formalize this intuition in the following lemma.

Lemma E.2. *Let $\mathcal{A}$ be an RW streaming algorithm that on inputs of size N has time complexity $t(N)$, random-access space complexity $m(N)$, allocates at most D streams, and has streaming space complexity $s(N)$. Then, for all $B > 1$, there exists a corresponding PDM algorithm $\mathcal{B}$ with block transfer size B which, for problem size N,*

1. *uses $m(N)$ internal memory,*
2. *uses D disks with total disk memory $s(N)$,*
3. *has time complexity $t(N)$, and*
4. *has I/O complexity $\mathsf{io}(N)/B$, where $\mathsf{io}(N)$ is the number of read and write operations performed by $\mathcal{A}$.*

Proof. The PDM algorithm $\mathcal{B}$ emulates the RW streaming algorithm $\mathcal{A}$, except that it sequentially reads and writes blocks of size B instead of one element at a time. If $\mathcal{A}$ wants to read $< B$ elements from a stream, $\mathcal{B}$ still reads a full block of size B and ignores the rest of the elements. It is easy to inspect that $\mathcal{B}$ satisfies the stated efficiency claims. □

This lemma enables us to analyze our algorithms in the simpler RW streaming model.

F Restrictions of our RW streaming model

Allocation of output streams. While the model dictates that $\mathcal{A}(\mathbf{I}) \mapsto \mathbf{O}$ does not allocate memory for $\mathbf{O}$, our pseudocode will occasionally let $\mathcal{A}$ initialize $\mathbf{O}$ for clarity.

Immutable input streams. Let $\mathcal{A}(\mathbf{I}) \mapsto \mathbf{O}$ be a RW streaming algorithm. To simplify analysis, it is convenient to assume that input streams $\mathbf{I}$ are immutable. On the other hand, writing pseudocode is easier when $\mathcal{A}$ can mutate the contents of $\mathbf{I}$. We resolve this issue by rewriting $\mathcal{A}$ as follows: $\mathcal{A}$ allocates an RW stream $\mathbf{S}$, copies over the contents of $\mathbf{I}$ to $\mathbf{S}$ with one pass, and uses $\mathbf{S}$ wherever it would have used $\mathbf{I}$.

Single input and output streams. For simplicity of modeling, Definition 4.2 restricts algorithms to have single input and output streams, but pseudocode is simpler without this restriction. As above, we can get the best of both worlds by automatically rewriting any algorithm with multiple input/output streams to instead have single input/output streams by concatenating the multiple streams into single ones. This does not worsen asymptotic efficiency.

G Comparing `arkworks` versions

The baselines we compared Scribe against use an older version of `arkworks`. Specifically, while Scribe uses version 0.5, the baselines all use version 0.4. To illustrate that the performance differences reported in

Section 8 do not arise because of this version difference, we ran relevant benchmarks from the benchmark suite of `arkworks`. We report the results in the following table. Note that all MSMs are of size 2^{16}, and the sparse MSM uses scalars of size 32 bits. The takeaway is that field arithmetic performs identically, while MSMs are actually faster in v0.4. Hence, if anything, the version difference benefits the baselines.

benchmark	v0.4	v0.5	speedup
$\mathbb{F}_r$ addition	2.5ns	2.5ns	0.99
$\mathbb{F}_r$ double	2.5ns	2.5ns	1.00
$\mathbb{F}_r$ inverse	3.5ns	3.5ns	1.00
$\mathbb{F}_r$ multiplication	12.2ns	12.5ns	0.97
$\mathbb{F}_r$ negation	2.8ns	2.7ns	1.00
$\mathbb{F}_r$ square	11.4ns	11.5ns	1.00
$\mathbb{F}_r$ subtraction	3.2ns	3.2ns	0.98
$\mathbb{G}_1$ MSM	844.5ms	975.4ms	0.87
$\mathbb{G}_1$ sparse MSM	140.6ms	155.8ms	0.90

References

[AFGHO16] M. Abe, G. Fuchsbauer, J. Groth, K. Haralambiev, and M. Ohkubo. "Structure-Preserving Signatures and Commitments to Group Elements". In: *Journal of Cryptology* 29.2 (2016), pp. 363–421.

[AST24] A. Arun, S. T. V. Setty, and J. Thaler. "Jolt: SNARKs for Virtual Machines via Lookups". In: *Proceedings of the 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '24. 2024, pp. 3–33.

[AV88] A. Aggarwal and J. S. Vitter. "The Input/Output Complexity of Sorting and Related Problems". In: *Communications of the ACM* 31.9 (1988), pp. 1116–1127.

[Baw+25] A. Baweja, A. Chiesa, E. Fedele, G. Fenzi, P. Mishra, T. Mopuri, and A. Zitek-Estrada. "Time-Space Trade-Offs for Sumcheck". In: *Proceedings of the 23rd Theory of Cryptography Conference*. TCC '25. 2025.

[BBBPWM18] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. "Bulletproofs: Short Proofs for Confidential Transactions and More". In: *Proceedings of the 39th IEEE Symposium on Security and Privacy*. S&P '18. 2018, pp. 315–334.

[BBHV22] L. Bangalore, R. Bhadauria, C. Hazay, and M. Venkitasubramaniam. "On Black-Box Constructions of Time and Space Efficient Sublinear Arguments from Symmetric-Key Primitives". In: *Proceedings of the 20th Theory of Cryptography Conference*. TCC '22. 2022, pp. 417–446.

[BC12] N. Bitansky and A. Chiesa. "Succinct Arguments from Multi-Prover Interactive Proofs and their Efficiency Benefits". In: *Proceedings of the 32nd Annual International Cryptology Conference*. CRYPTO '12. 2012, pp. 255–272.

[BCCGP16] J. Bootle, A. Cerulli, P. Chaidos, J. Groth, and C. Petit. "Efficient Zero-Knowledge Arguments for Arithmetic Circuits in the Discrete Log Setting". In: *Proceedings of the 35th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT '16. 2016, pp. 327–357.

[BCCT12] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. "From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again". In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. ITCS '12. 2012, pp. 326–349.

[BCCT13] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. "Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data". In: *Proceedings of the 45th ACM Symposium on the Theory of Computing*. STOC '13. 2013, pp. 111–120.

[BCGJM18] J. Bootle, A. Cerulli, J. Groth, S. K. Jakobsen, and M. Maller. "Arya: Nearly Linear-Time Zero-Knowledge Proofs for Correct Program Execution". In: *Proceedings of the 24th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT '18. 2018, pp. 595–626.

[BCHO22] J. Bootle, A. Chiesa, Y. Hu, and M. Orrù. "Gemini: Elastic SNARKs for Diverse Environments". In: *Proceedings of the 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '22. 2022.

[BCLMS21] B. Bünz, A. Chiesa, W. Lin, P. Mishra, and N. Spooner. "Proof-Carrying Data Without Succinct Arguments". In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO '21. 2021, pp. 681–710.

[BCMS20] B. Bünz, A. Chiesa, P. Mishra, and N. Spooner. "Proof-Carrying Data from Accumulation Schemes". In: *Proceedings of the 18th Theory of Cryptography Conference*. TCC '20. 2020.

[BCS21] J. Bootle, A. Chiesa, and K. Sotiraki. "Sumcheck Arguments and Their Applications". In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO '21. 2021, pp. 742–773.

[BCTV14] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. "Scalable Zero Knowledge via Cycles of Elliptic Curves". In: *Proceedings of the 34th Annual International Cryptology Conference*. CRYPTO '14. 2014, pp. 276–294.

[BDFG21] D. Boneh, J. Drake, B. Fisch, and A. Gabizon. "Halo Infinite: Proof-Carrying Data from Additive Polynomial Commitments". In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO '21. 2021, pp. 649–680.

[BFS20] B. Bünz, B. Fisch, and A. Szepieniec. "Transparent SNARKs from DARK Compilers". In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '20. 2020, pp. 677–706.

[BGH19] S. Bowe, J. Grigg, and D. Hopwood. "Halo: Recursive Proof Composition without a Trusted Setup". IACR ePrint Report 2019/1021. 2019.

[BHRRS20] A. R. Block, J. Holmgren, A. Rosen, R. D. Rothblum, and P. Soni. "Public-Coin Zero-Knowledge Arguments with (almost) Minimal Time and Space Overheads". In: *Proceedings of the 18th Theory of Cryptography Conference*. TCC '20. 2020.

[BHRRS21] A. R. Block, J. Holmgren, A. Rosen, R. D. Rothblum, and P. Soni. "Time- and Space-Efficient Arguments from Groups of Unknown Order". In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO '21. 2021, pp. 123–152.

[BJR07] P. Beame, T. S. Jayram, and A. Rudra. "Lower Bounds for Randomized Read/Write Stream Algorithms". In: *Proceedings of the 39th Annual ACM Symposium on Theory of Computing*. STOC '07. 2007.

[BMMTV21] B. Bünz, M. Maller, P. Mishra, N. Tyagi, and P. Vesely. "Proofs for Inner Pairing Products and Applications". In: *Proceedings of the 27th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT '21. 2021, pp. 65–97.

[CBBZ23] B. Chen, B. Bünz, D. Boneh, and Z. Zhang. "HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates". In: *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '23. 2023, pp. 499–530.

[CHMMVW20] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward. "Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS". In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '20. 2020.

[CM24] J. Cook and D. Moshkovitz. "Explicit Time and Space Efficient Encoders Exist Only with Random Access". In: *Proceedings of the 39th Computational Complexity Conference*. CCC '24. 2024, 5:1–5:54.

[con22] arkworks contributors. `arkworks` *zkSNARK ecosystem*. 2022.

[FOSS17] X. Fan, A. Otemissov, F. Sica, and A. Sidorenko. "Multiple point compression on elliptic curves". In: *Des. Codes Cryptogr.* 83.3 (2017), pp. 565–588.

[FS86] A. Fiat and A. Shamir. "How to prove yourself: practical solutions to identification and signature problems". In: *Proceedings of the 6th Annual International Cryptology Conference*. CRYPTO '86. 1986, pp. 186–194.

[GKS05] M. Grohe, C. Koch, and N. Schweikardt. "Tight Lower Bounds for Query Processing on Streaming and External Memory Data". In: *Proceedings of the 32nd International Colloquium on Automata, Languages and Programming*. ICALP '05. 2005, pp. 1076–1088.

[GLSTW23] A. Golovnev, J. Lee, S. T. V. Setty, J. Thaler, and R. S. Wahby. "Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS". In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO '23. 2023, pp. 193–226.

[GW19] A. Gabizon and Z. J. Williamson. "The turbo-plonk program syntax for specifying snark programs". Preprint. 2019.

[Hab22] U. Haböck. "Multivariate lookups based on logarithmic derivatives". IACR ePrint Report 2022/1530. 2022.

[HLP24] U. Haböck, D. Levit, and S. Papini. "Circle STARKs". IACR ePrint Report 2024/278. 2024.

[KP23] A. Kothapalli and B. Parno. "Algebraic Reductions of Knowledge". In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO '23. 2023, pp. 669–701.

[KST22] A. Kothapalli, S. T. V. Setty, and I. Tzialla. "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes". In: *Proceedings of the 42nd Annual International Cryptology Conference*. CRYPTO '22. 2022, pp. 359–388.

[KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. "Constant-Size Commitments to Polynomials and Their Applications". In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT '10. 2010, pp. 177–194.

[Lee21] J. Lee. "Dory: Efficient, Transparent Arguments for Generalised Inner Products and Polynomial Commitments". In: *Proceedings of the 19th Theory of Cryptography Conference*. TCC '21. 2021, pp. 1–34.

[LFKN92] C. Lund, L. Fortnow, H. J. Karloff, and N. Nisan. "Algebraic Methods for Interactive Proof Systems". In: *Journal of the ACM* 39.4 (1992), pp. 859–868.

[LMR19] R. W. F. Lai, G. Malavolta, and V. Ronge. "Succinct Arguments for Bilinear Group Arithmetic: Practical Structure-Preserving Cryptography". In: *Proceedings of the 26th ACM Conference on Computer and Communications Security*. CCS '19. 2019, pp. 2057–2074.

[NDCTB24] W. Nguyen, T. Datta, B. Chen, N. Tyagi, and D. Boneh. "Mangrove: A Scalable Framework for Folding-based SNARKs". In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO '24. 2024.

[NTZ25] V. Nair, J. Thaler, and M. Zhu. "Proving CPU Executions in Small Space". IACR ePrint Report 2025/611. 2025.

[Pip80] N. Pippenger. "On the Evaluation of Powers and Monomials". In: *SIAM Journal on Computing* 9.2 (1980), pp. 230–250.

[Pol] Polygon. *Plonky3*. URL: https://github.com/plonky3/plonky3.

[PP24] C. Pappas and D. Papadopoulos. "Sparrow: Space-Efficient zkSNARK for Data-Parallel Circuits and Applications to Zero-Knowledge Decision Trees". In: *Proceedings of the 31st ACM Conference on Computer and Communications Security*. CCS '24. 2024.

[PP25] C. Pappas and D. Papadopoulos. "Hobbit: Space-Efficient zkSNARK with Optimal Prover Time". In: *Proceedings of the 34th USENIX Security Symposium*. USENIX Security '25. 2025.

[PST13] C. Papamanthou, E. Shi, and R. Tamassia. "Signatures of Correct Computation". In: *Proceedings of the 10th Theory of Cryptography Conference*. TCC '13. 2013, pp. 222–242.

[PY14] P. A. Papakonstantinou and G. Yang. "Cryptography with Streaming Algorithms". In: *Proceedings of the 34th Annual Cryptology Conference*. CRYPTO '14. 2014, pp. 55–70.

[Set20] S. Setty. "Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup". In: *Proceedings of the 40th Annual International Cryptology Conference*. CRYPTO '20. 2020, pp. 704–737.

[SL20] S. T. V. Setty and J. Lee. "Quarks: Quadruple-efficient transparent zkSNARKs". IACR ePrint Report 2020/1275. 2020.

[Tha13] J. Thaler. "Time-Optimal Interactive Proofs for Circuit Evaluation". In: *Proceedings of the 33rd Annual International Cryptology Conference*. CRYPTO '13. 2013, pp. 71–89.

[Tha22] J. Thaler. "Proofs, Arguments, and Zero-Knowledge". In: *Found. Trends Priv. Secur.* 4.2-4 (2022), pp. 117–660.

[VS94] J. S. Vitter and E. A. M. Shriver. "Algorithms for Parallel Memory, I: Two-level Memories". In: *Algorithmica* 12.2 (1994), pp. 110–147.

[Whi18] B. WhiteHat. "roll_up: A Scalable Zero Knowledge Roll Up". Accessed: 2024-02-10. 2018.

[WHV24] R. Wang, C. Hazay, and M. Venkitasubramaniam. "Ligetron: Lightweight Scalable End-to-End Zero-Knowledge Proofs. Post-Quantum ZK-SNARKs on a Browser". In: *Proceedings of the 45th IEEE Symposium on Security and Privacy*. IEEE S&P '24. 2024.

[WTSTW18] R. S. Wahby, I. Tzialla, A. Shelat, J. Thaler, and M. Walfish. "Doubly-Efficient zkSNARKs Without Trusted Setup". In: *Proceedings of the 39th IEEE Symposium on Security and Privacy*. IEEE S&P '18. 2018, pp. 926–943.

[Xie+22] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song. "zkBridge: Trustless Cross-chain Bridges Made Practical". In: *Proceedings of the 29th ACM Conference on Computer and Communications Security*. CCS '22. 2022, pp. 3003–3017.

[XZZPS19] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, and D. Song. "Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation". In: *Proceedings of the 39th Annual International Cryptology Conference*. CRYPTO '19. 2019, pp. 733–764.

[ZCLKZ24] S. Zhang, D. Cai, Y. Li, H. Kan, and L. Zhang. "Epistle: Elastic Succinct Arguments for Plonk Constraint System". IACR ePrint Report 2024/872. 2024.

[ZGKPP18] Y. Zhang, D. Genkin, J. Katz, D. Papadopoulos, and C. Papamanthou. "vRAM: Faster Verifiable RAM with Program-Independent Preprocessing". In: *Proceedings of the 39th IEEE Symposium on Security and Privacy*. IEEE S&P '18. 2018, pp. 908–925.